授業受講者以外への再配布禁止です
Redistribution to anyone who don't take this lecture is prohibited

2018.5.14

# 第4回
# 航空宇宙設計最適化
## Design optimization in aerospace engineering

大山　聖
Akira Oyama

# スケジュール　Schedule

| | | |
|---|---|---|
| 4/16 | [1] 設計最適化問題とは？ | What is design optimization? |
| 4/23 | [2] 設計最適化手法（その１） | Design optimization methods (part 1) |
| 5/7 | [3] 設計最適化手法（その２） | Design optimization methods (part 2) |
| 5/14 | [4] 設計最適化手法（その３） | Design optimization methods (part 3) |
| 5/21 | [5] 多目的設計探査 | Multiobjective design exploration |
| 5/28 | 休講 | |
| 6/4 | [6] 飛行機の設計 | aircraft design |
| | JAXA第三研究ユニット長 嶋英志氏 | |
| 6/11 | [7]【活用事例】Solar-Dミッションの軌道計画 | |
| | | Trajectory design of Solar-D mission |
| 6/18 | [8]【活用事例】H-III ロケットのターボポンプ設計 | |
| | | H-III Rocket turbopump design |
| 6/25 | [9] ロバスト設計・不確定性定量評価 | robust design /uq |
| | 東北大学流体科学研究所准教授　下山幸治氏 | |
| 7/2 | [10]【活用事例】火星飛行機＆自動車設計 | |
| | | Mars airplane and car structure design |
| 7/9 | [11] 超音速機設計 | supersonic aircraft design |
| | JAXA航空プログラムグループ超音速機チーム牧野好和氏 | |

# 今日のアウトライン
# outline

## （単目的）進化アルゴリズム
(single-objective) evolutionary algorithms

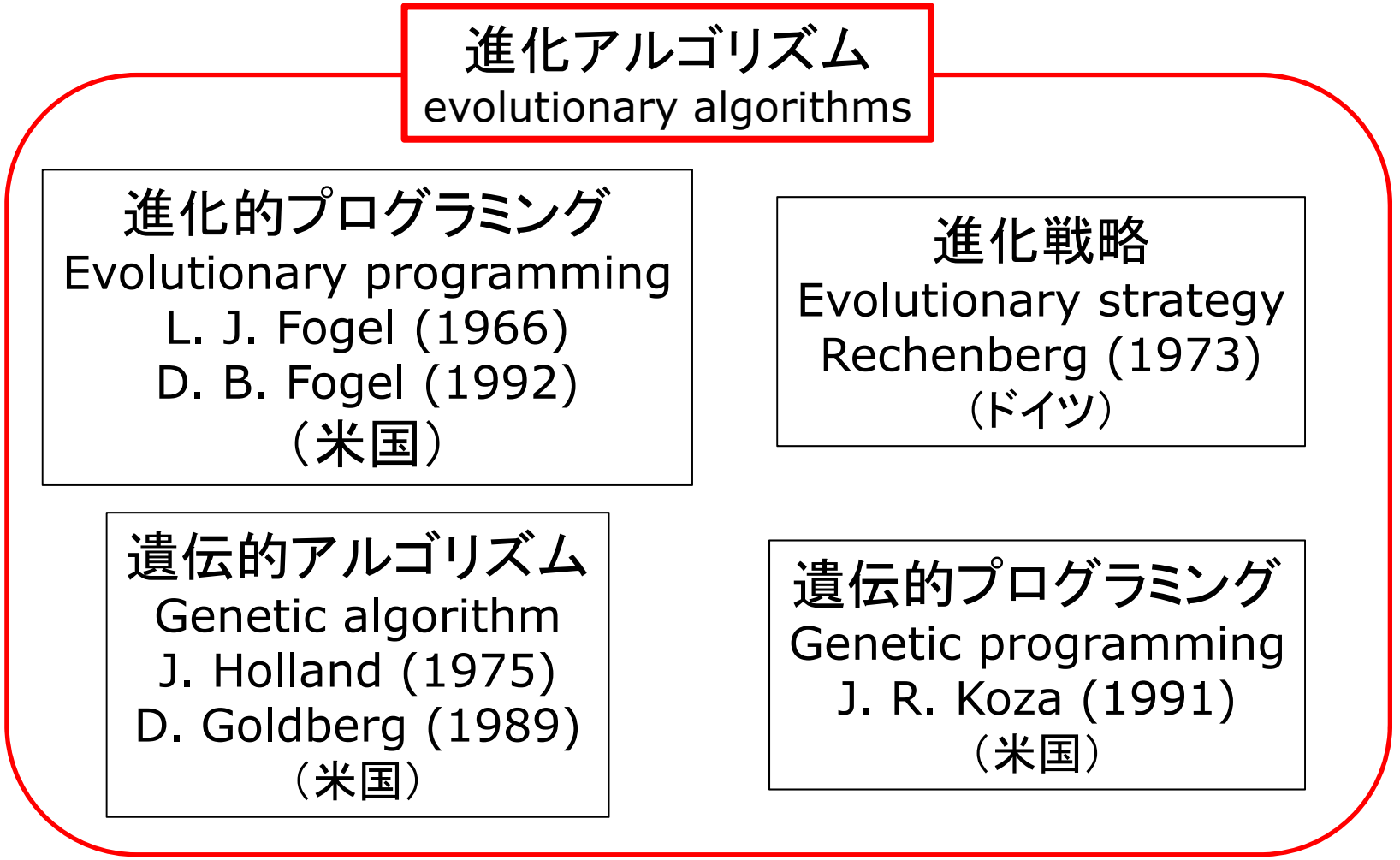
## 多目的進化アルゴリズム
multiobjective evolutionary algorithms

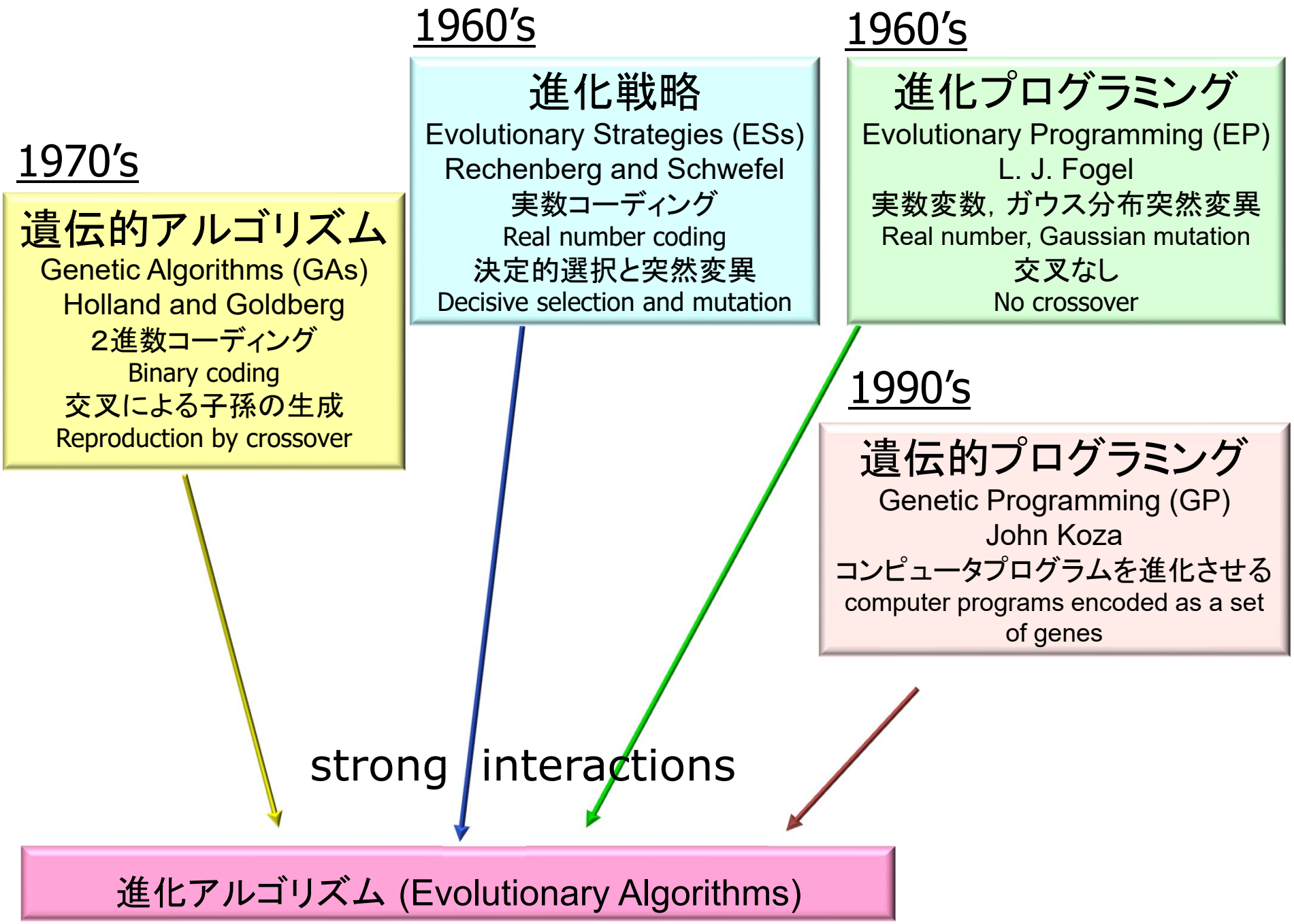
# 進化アルゴリズム

Evolutionary algorithms

(EAs)

# 生物の進化にヒントを得た大域的探査手法
Global optimization algorithms inspired by natural evolution




1960's
進化戦略
Evolutionary Strategies (ESs)
Rechenberg and Schwefel
実数コーディング
Real number coding
決定的選択と突然変異
Decisive selection and mutation
1960's
進化プログラミング
Evolutionary Programming (EP)
L. J. Fogel
実数変数，ガウス分布突然変異
Real number, Gaussian mutation
交叉なし
No crossover
1970's
遺伝的アルゴリズム
Genetic Algorithms (GAs)
Holland and Goldberg
2進数コーディング
Binary coding
交叉による子孫の生成
Reproduction by crossover
1990's
遺伝的プログラミング
Genetic Programming (GP)
John Koza
コンピュータプログラムを進化させる
computer programs encoded as a set of genes
strong interactions
進化アルゴリズム (Evolutionary Algorithms)

生物界にヒントを得た大域的探査手法
Global optimization algorithms inspired by nature

# 進化計算
# Evolutionary computation

## 進化アルゴリズム
## Evolutionary algorithms

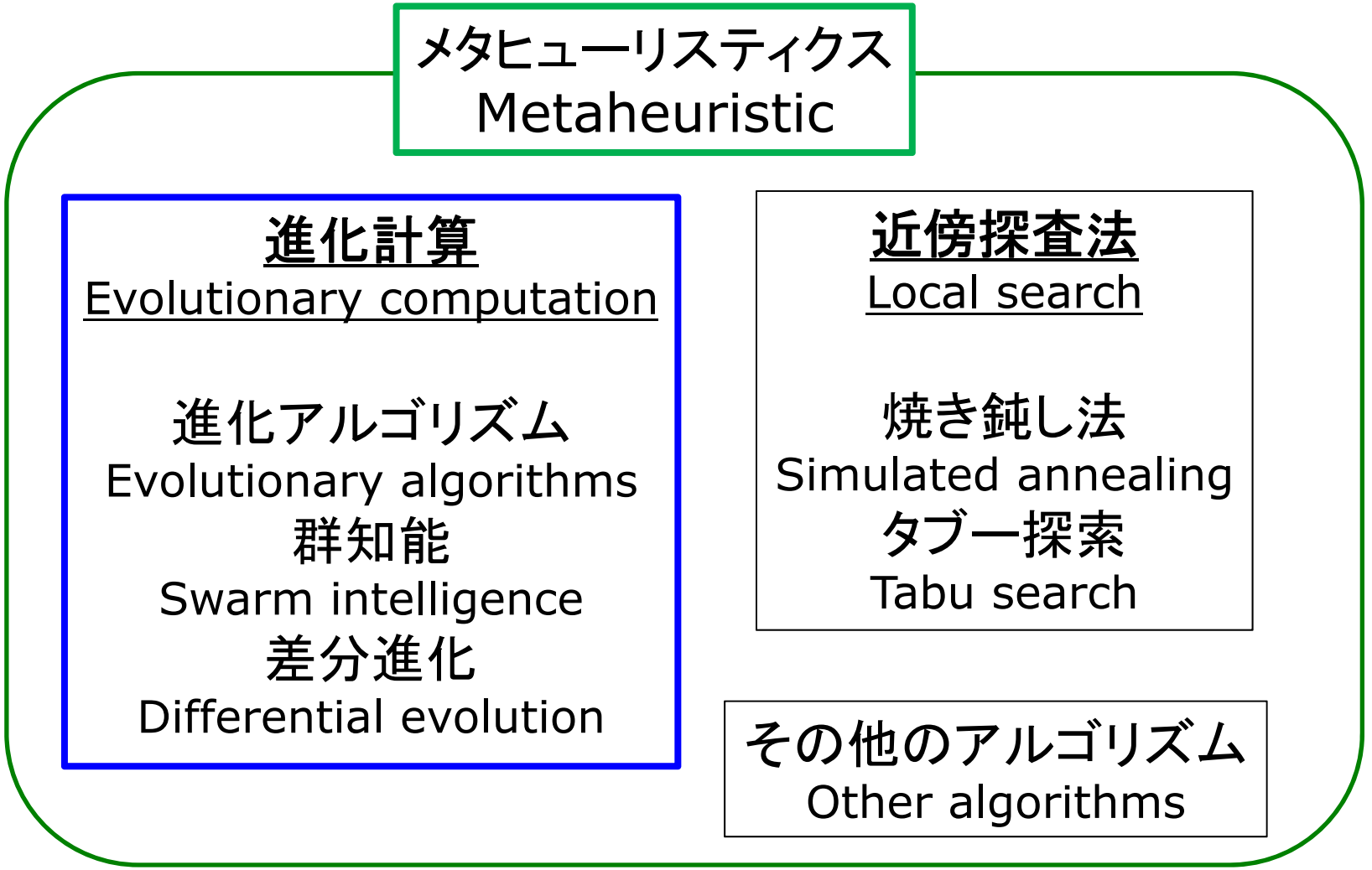
遺伝的アルゴリズム
Genetic algorithm
進化戦略
Evolutionary strategy
進化的プログラミング
Evolutionary programming
遺伝的プログラミング
Genetic programming

## 群知能
## Swarm intelligence

粒子群最適化
Particle swarm optimization
アリコロニー最適化
Ant colony optimization

## 微分進化
## Differential evolution

特定の問題に限定されない，汎用性の高い最適化アルゴリズム
General purpose optimization algorithms



# ノーフリーランチ定理?
No free lunch theorem (NFLT)?

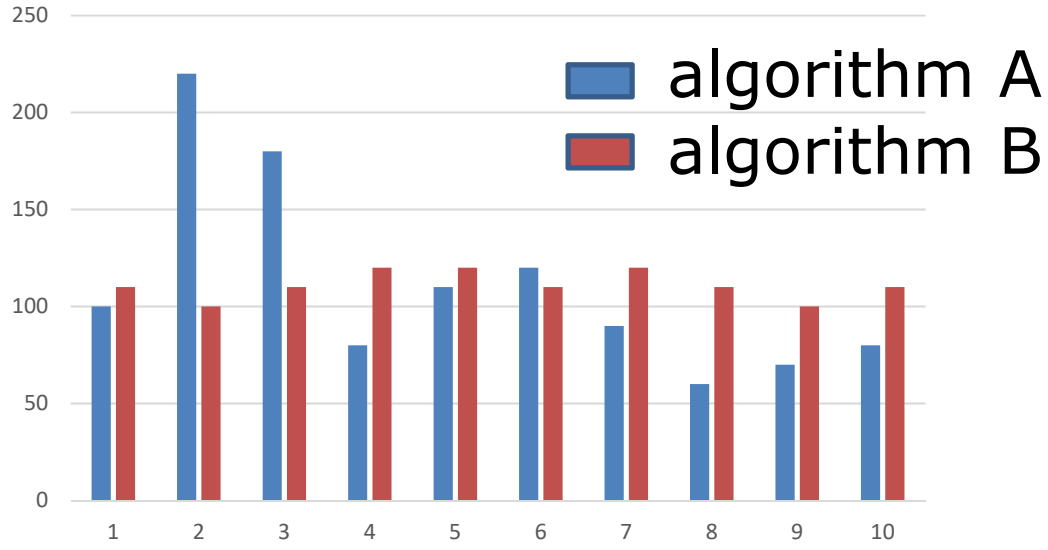
# ノーフリーランチ定理
## No free lunch theorem (NFLT)

David H. WolpertとWilliam G. Macreadyによって紹介された最適化に関する定理

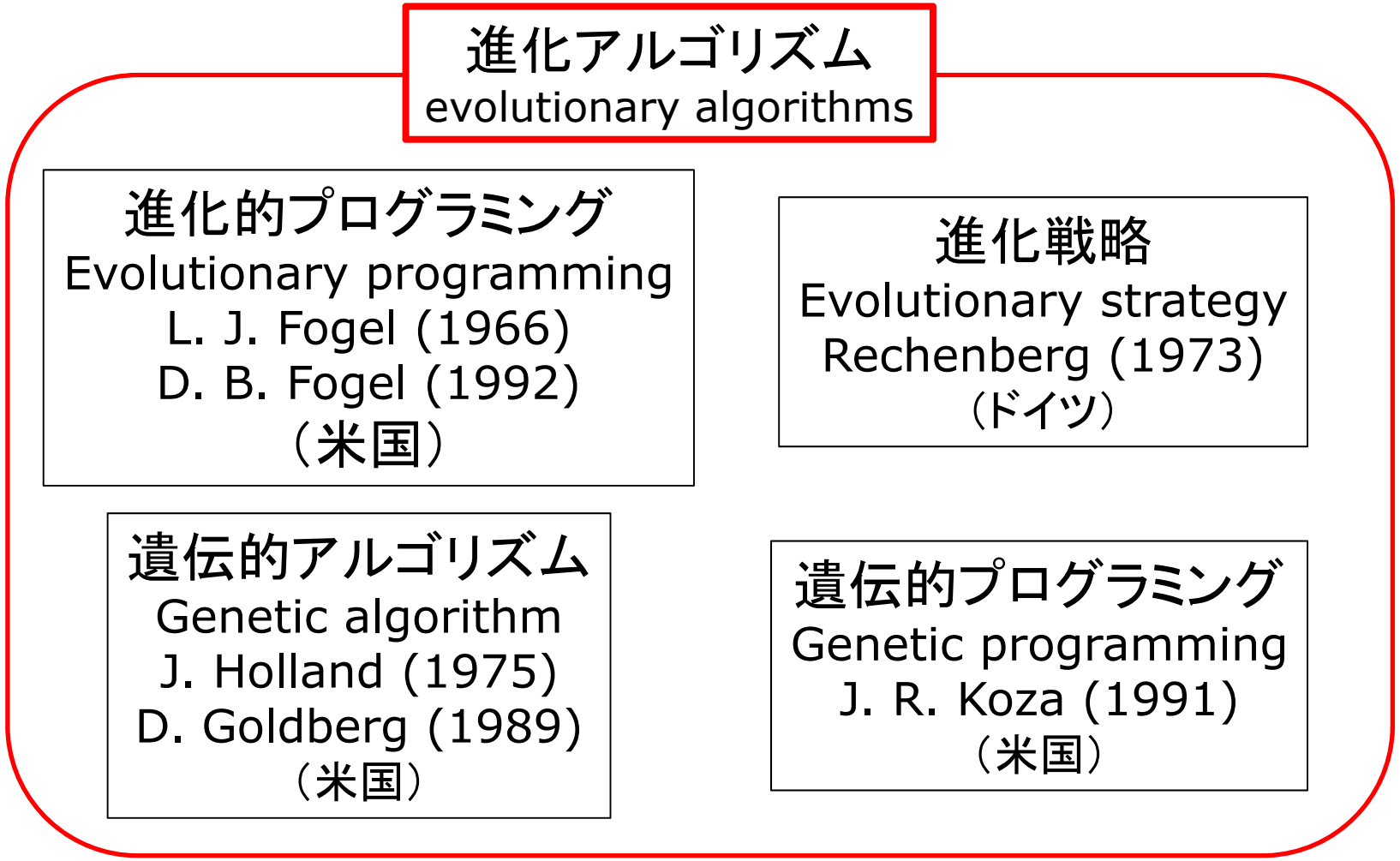
NFLT was introduced by David Wolpert and William G. Macready in connection with the problems of search and optimization

コスト関数の極値を探索するあらゆるアルゴリズムは、全ての可能なコスト関数に適用した結果を平均すると同じ性能となる

the computational cost of finding an optimum solution, averaged over all problems in the class, is the same for any solution method.



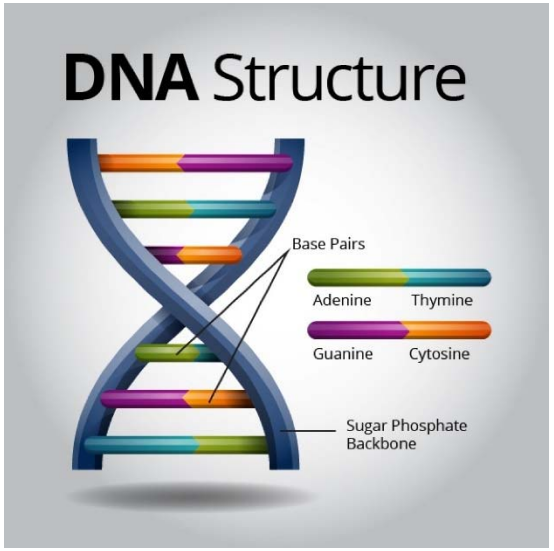
# 生物の進化にヒントを得た大域的探査手法
Global optimization algorithms inspired by natural evolution

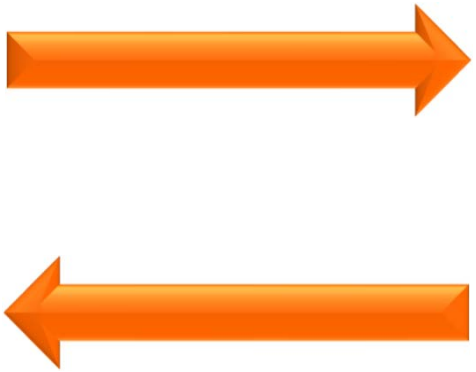


# ダーウィンの進化論

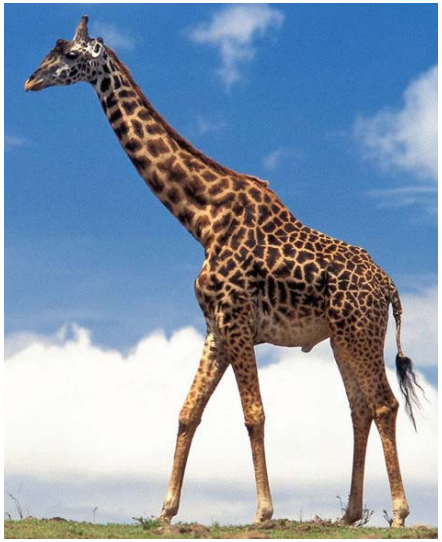
Theory of evolution
by
Charles R Darwin

## 生物の情報：遺伝子
## Biological information：genes



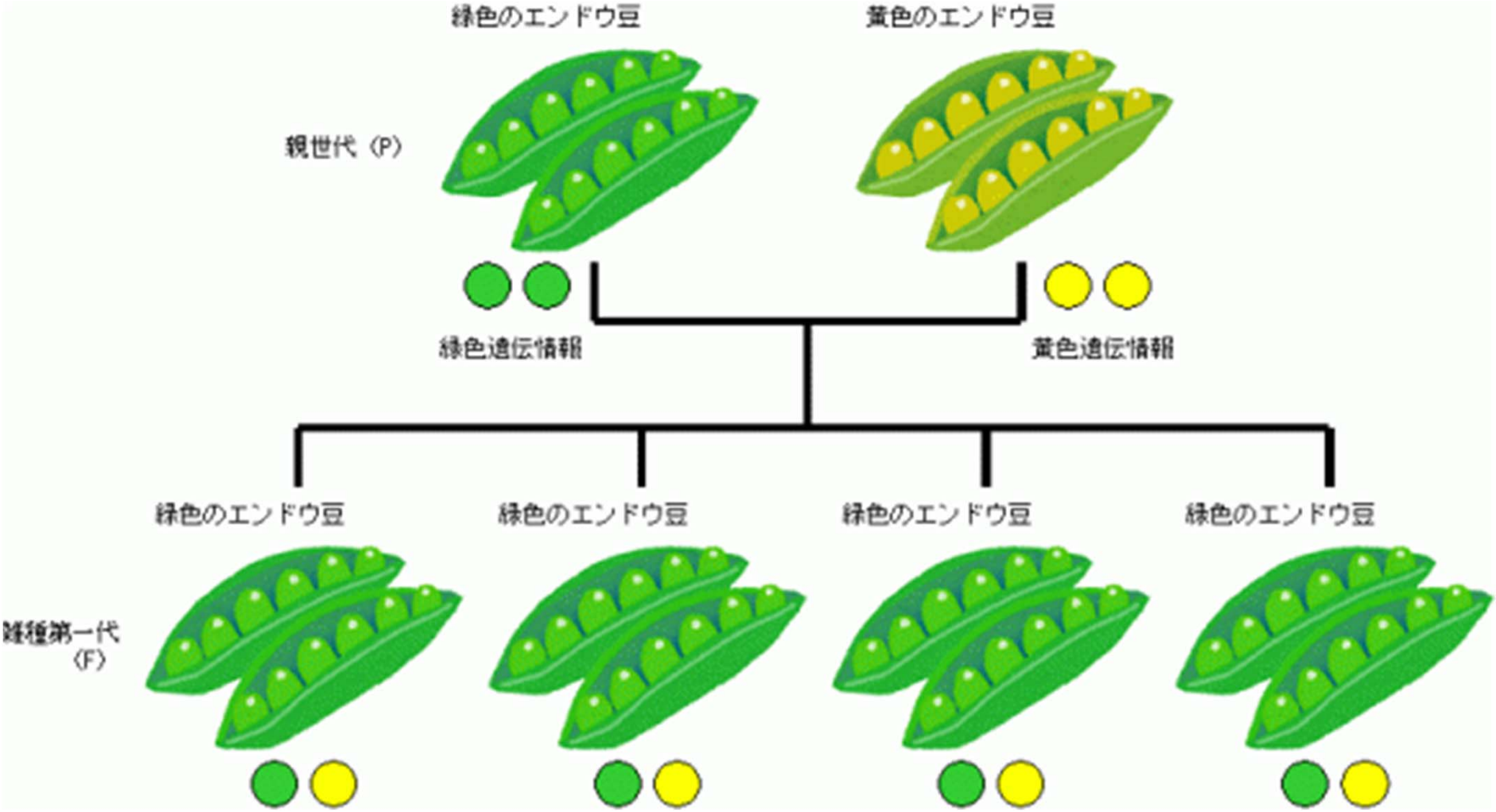



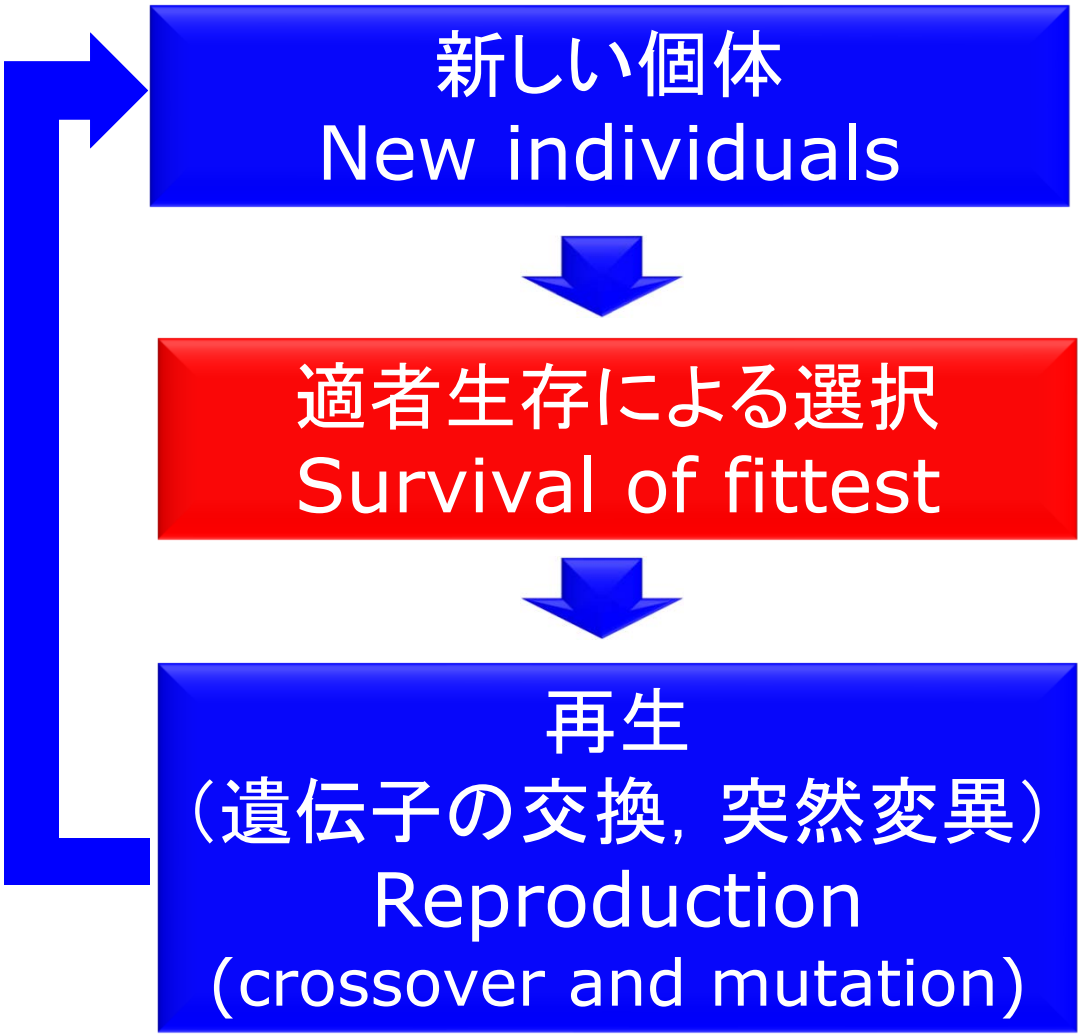
## 生物の表現形
## Phenotype of organism



| A | G | T | C | C | A | T | G |
|---|---|---|---|---|---|---|---|

# メンデルの法則 Mendel's law

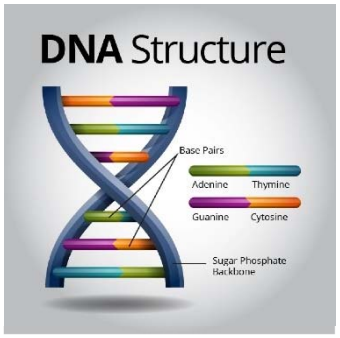




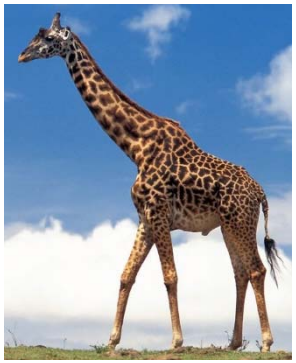

環境に適した生物が生き残っていく

# 進化アルゴリズムでは．．．

In evolutionary algorithm …




| A | G | T | C | C | A | T | G |
|---|---|---|---|---|---|---|---|



設計の情報：遺伝子型

Genotype of a design : string

Binary number string

| 1 | 0 | 1 | 1 | 0 | 1 | 0 |
|---|---|---|---|---|---|---|

Real number string

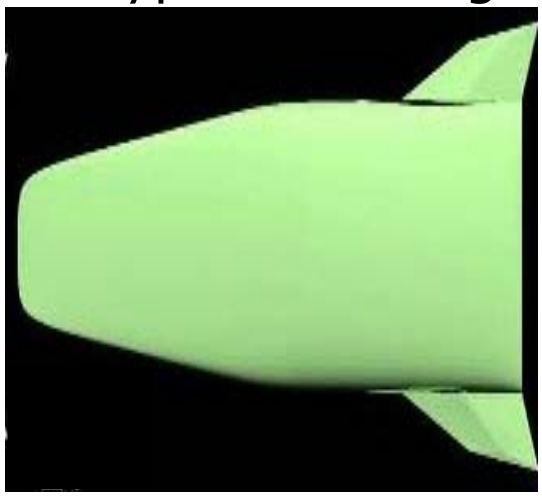
| 0.27 | 0.81 | 0.63 |
|---|---|---|

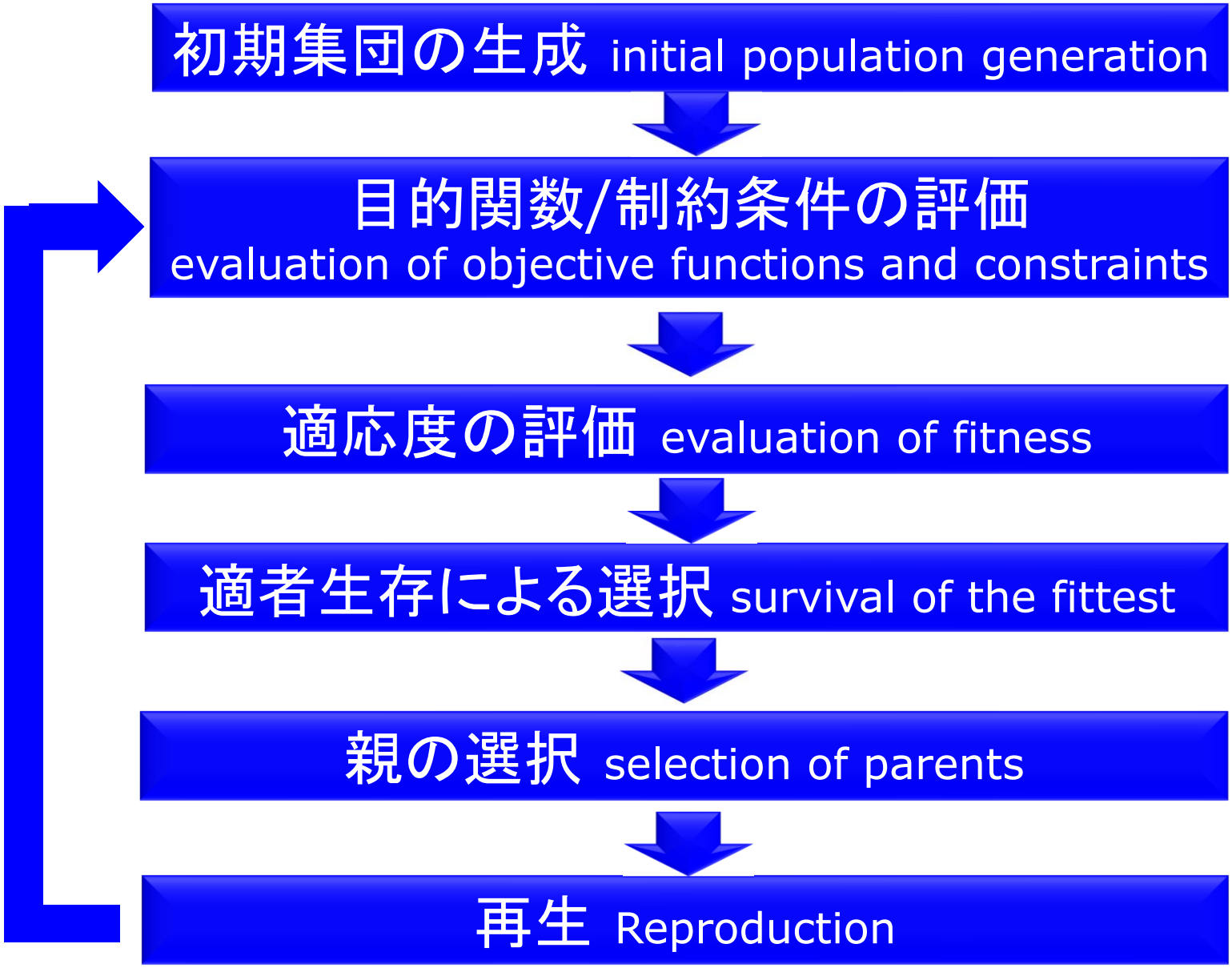
デコード化

コード化

設計の表現型

Phenotype of a design




初期集団の生成 initial population generation
目的関数/制約条件の評価
evaluation of objective functions and constraints
適応度の評価 evaluation of fitness
適者生存による選択 survival of the fittest
親の選択 selection of parents
再生 Reproduction

# 進化アルゴリズム

（1）初期集団を定める

Give initial population of designs

（2）集団の中のそれぞれの個体の目的関数値/制約条件値を評価し，その値をもとに適応度を計算

Evaluate objective / constraint function values of each design and compute fitness of each design

（3）それぞれの個体の適応度をもとに優れた個体群を選択

Selection of the designs based on their fitness

（4）交叉・突然変異の遺伝子操作で新しい集団を作成

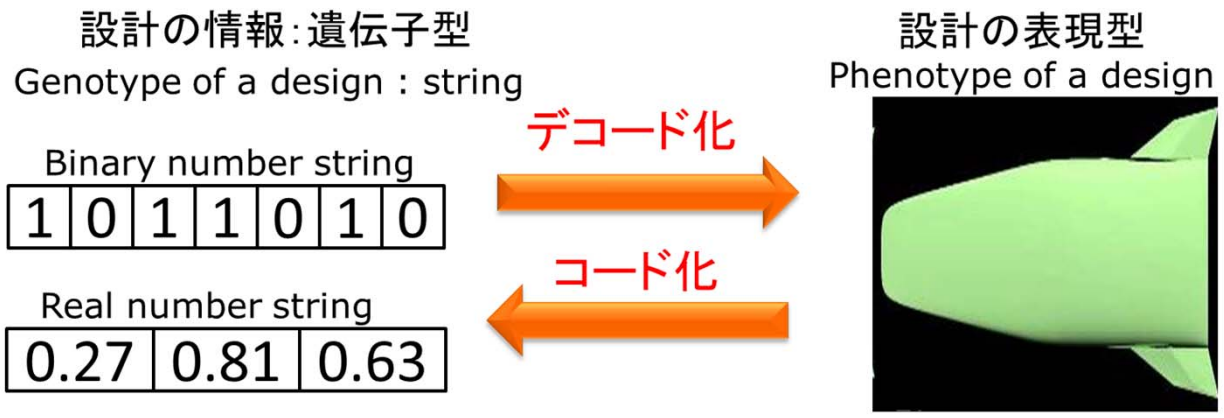
Generation of new designs based on crossover/mutation operators

（5）ステップ（2）にもどる

Go to step (2)

# 設計変数のコーディング方法

coding of design parameter



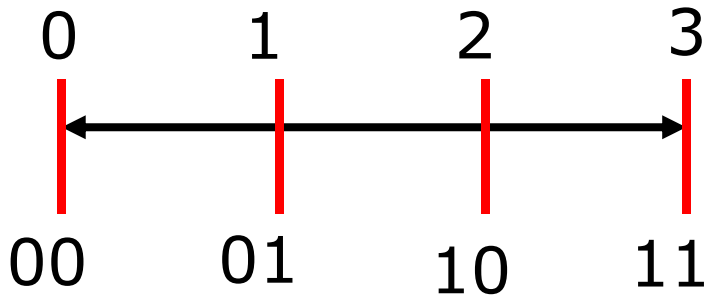
# 2進数コーディング Binary coding

設計変数 design parameter　　実数 real number

コード化された設計変数 encoded design parameter　　2進数 binary number

$$x = x_{\min} + \frac{x_{\max} - x_{\min}}{2^l - 1} \sum_{i=1}^{l} b_i 2^{i-1}$$

if　$x_{\min} = 0$　$x_{\max} = 3$　$l = 2$　⇒　$x = 2b_2 + b_1$

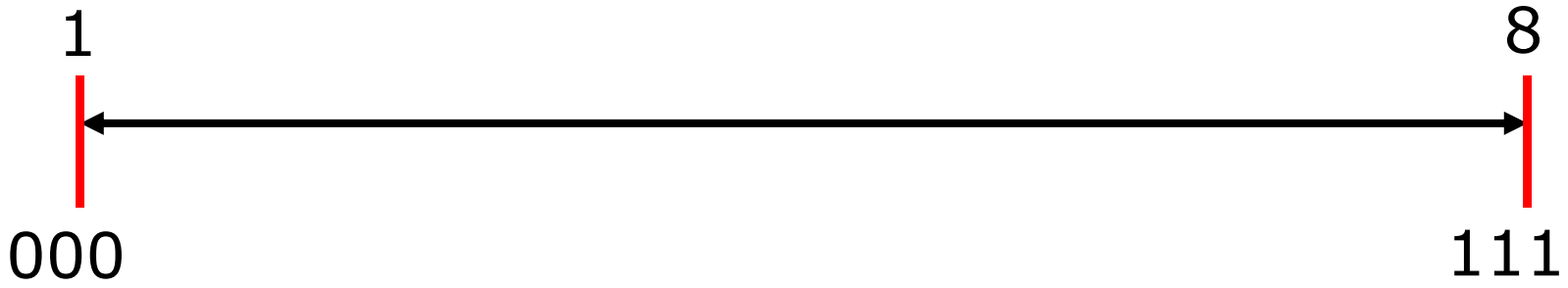



| | | |
|---|---|---|
| 0 | 0 | 0 |
| 1 | 0 | 1 |
| 2 | 1 | 0 |
| 3 | 1 | 1 |

# 2進数コーディング Binary coding

$$x = x_{\min} + \frac{x_{\max} - x_{\min}}{2^l - 1} \sum_{i=1}^{l} b_i 2^{i-1}$$

設計空間が1から8の場合に，3桁の2進数で表現するとどう表されるか？
Express relation when a design parameter is coded by 3-digit binary number where the design space is [1,8]



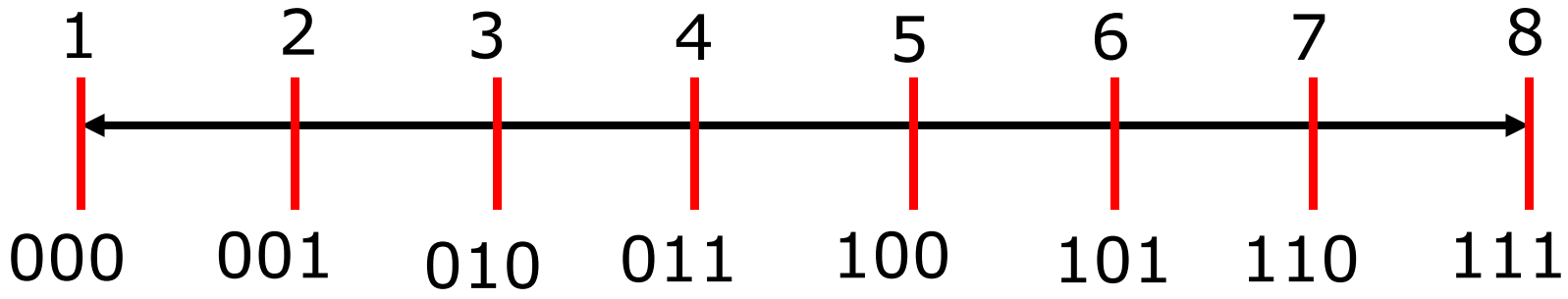
# 2進数コーディング Binary coding

$$x = x_{\min} + \frac{x_{\max} - x_{\min}}{2^l - 1} \sum_{i=1}^{l} b_i 2^{i-1}$$

設計空間が1から8の場合に，3桁の2進数で表現するとどう表されるか？
Express relation when a design parameter is coded by 3-digit binary number where the design space is [1,8]

$$x_{\max} = 8 \quad x_{\min} = 1 \quad l = 3$$

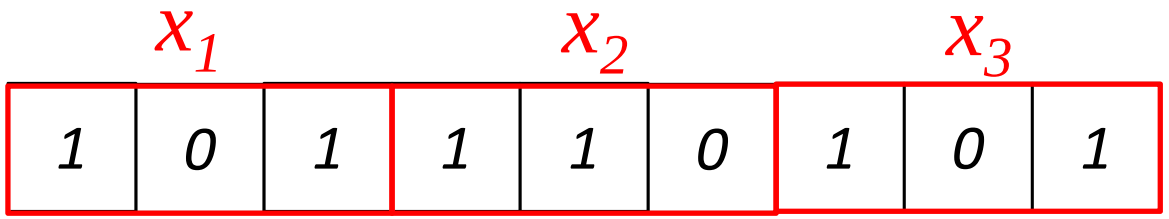
$$x = 1 + b_1 + 2b_2 + 4b_3$$



# 2進数コーディング Binary coding

設計変数が複数ある場合は

When number of design parameters is multiple



# 実数コーディング real-number coding

設計変数 design parameter　　実数 real number



コード化された設計変数 encoded design parameter　　実数 real number [0,1]

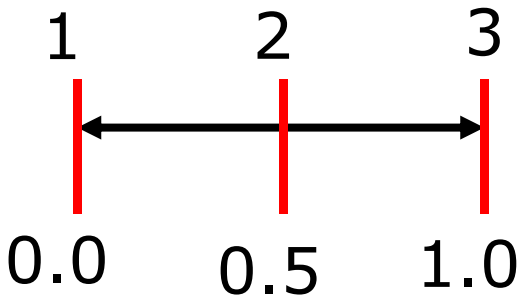
$$x = r(x_{\max} - x_{\min}) + x_{\min}$$

if　$x_{\min} = 1$　$x_{\max} = 3$

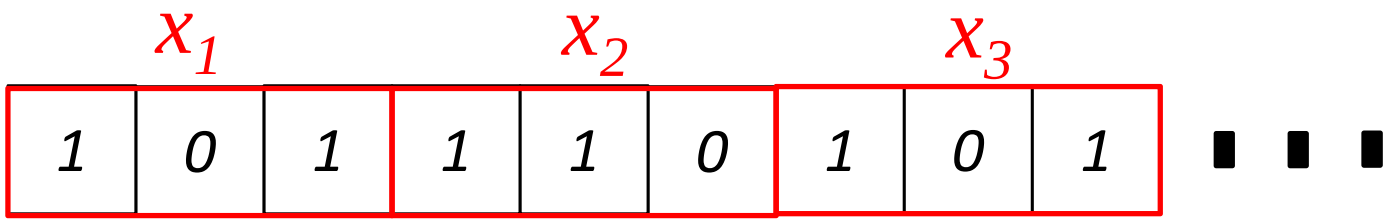


$$x = 2r + 1$$

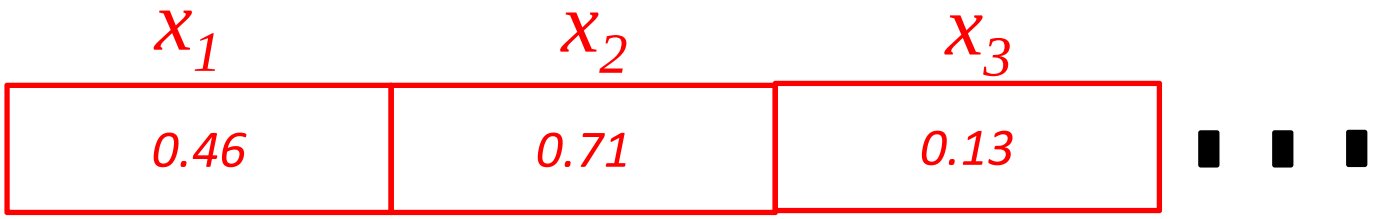
| | |
|---|---|
| 1 | 0.0 |
| 2 | 0.5 |
| 3 | 1.0 |



## 2進数コーディング binary coding



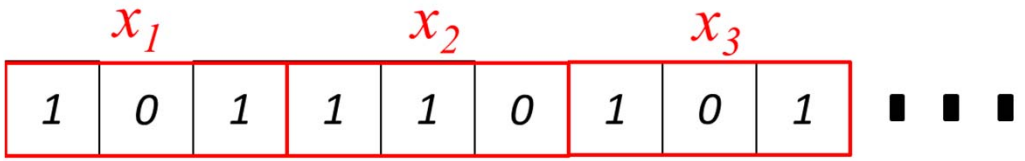

## 実数コーディング real-number coding



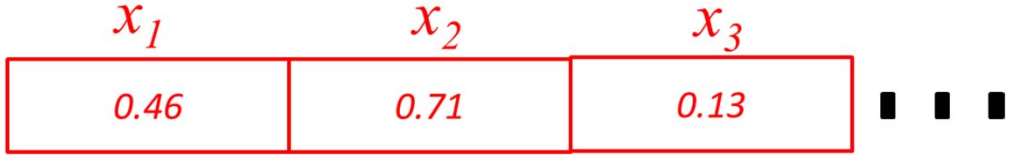
# コーディング方法の選択 selection of coding

実数最適化問題においては，2進数コーディングは探査効率が悪くなるので，通常，実数コーディングを用いる
In real number optimization problem, real-coding is usually used because optimization efficiency of EA with binary coding is poor

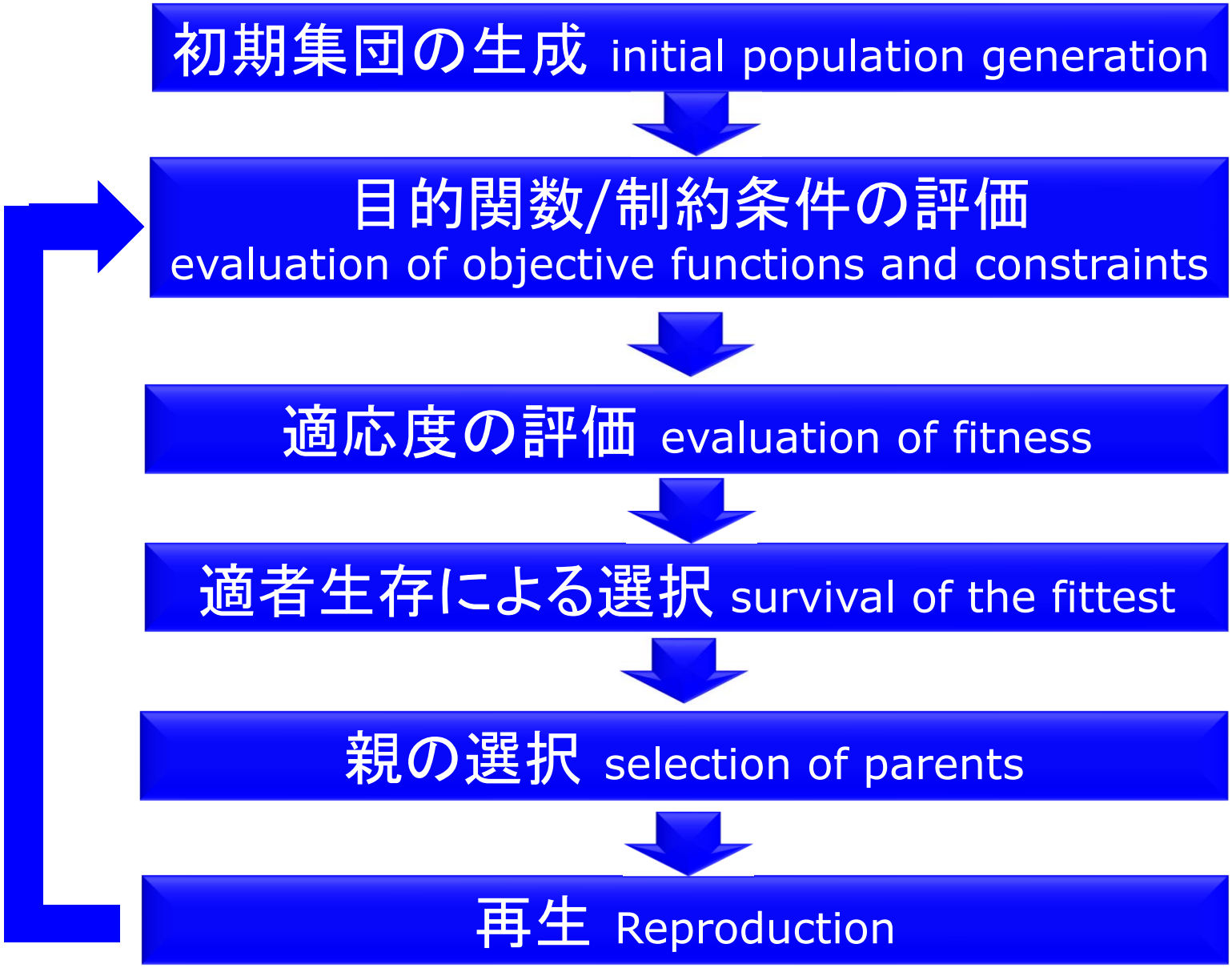
## 2進数コーディング binary coding



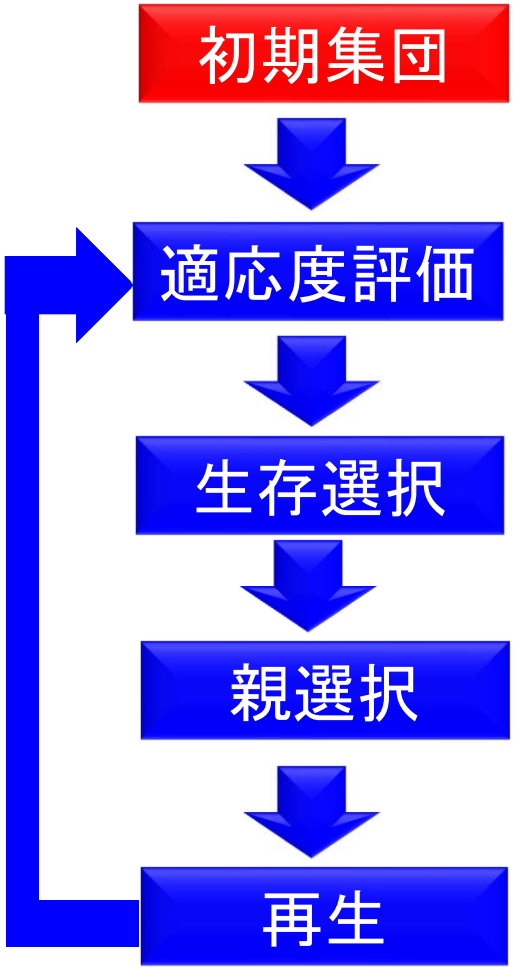

## 実数コーディング real-number coding

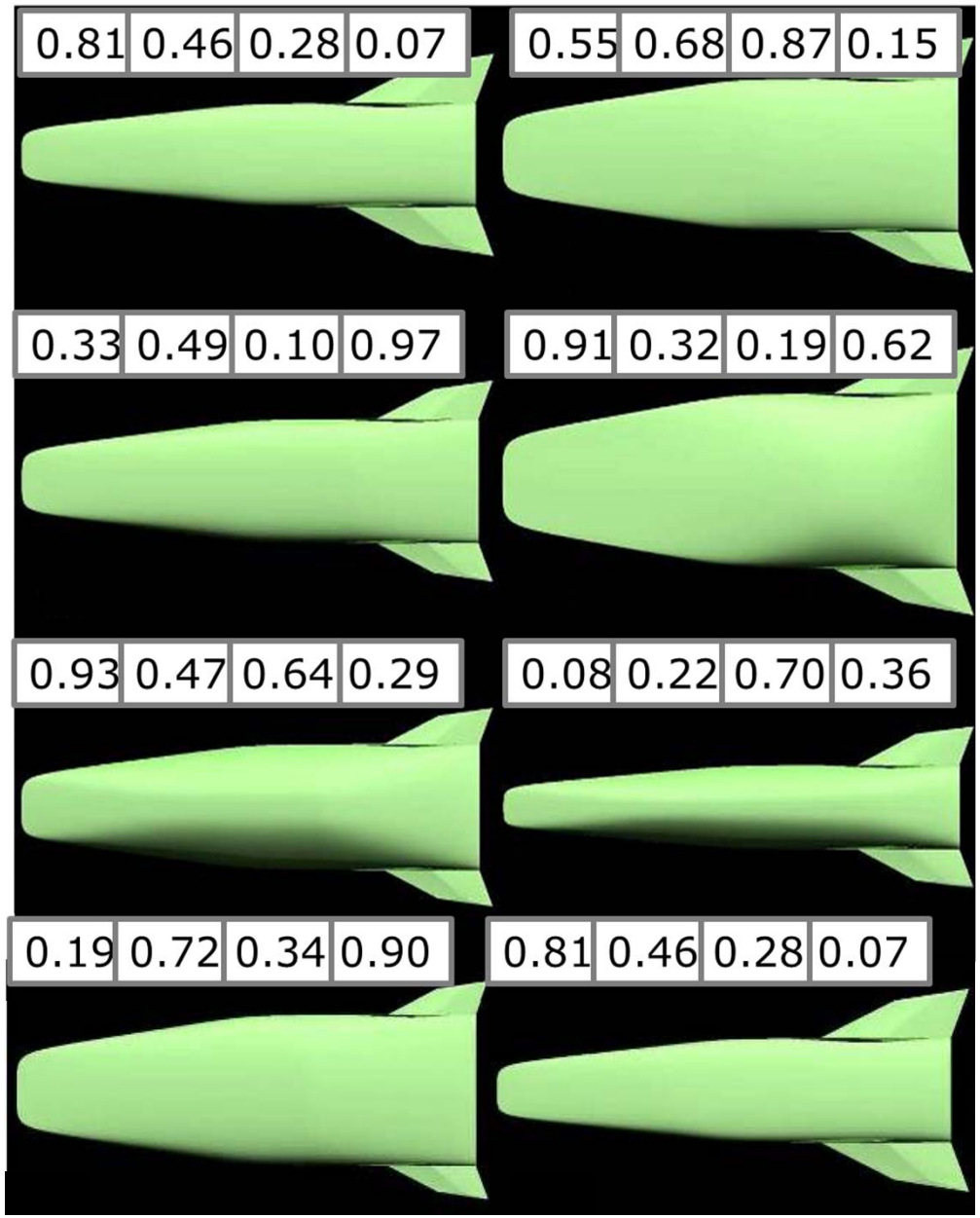



The reasons will be explained later …

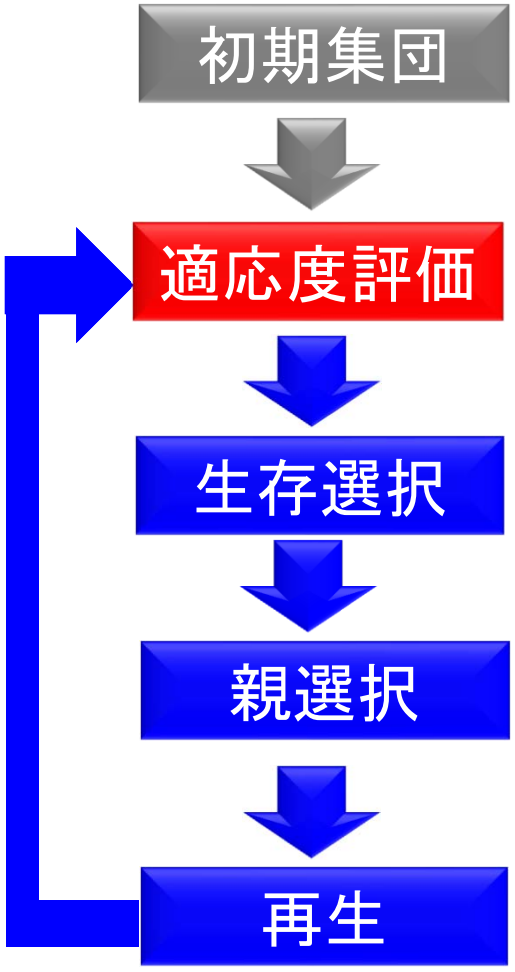

初期集団の生成 initial population generation
目的関数/制約条件の評価
evaluation of objective functions and constraints
適応度の評価 evaluation of fitness
適者生存による選択 survival of the fittest
親の選択 selection of parents
再生 Reproduction

# 乱数などで初期集団を生成
Initial population is generated randomly



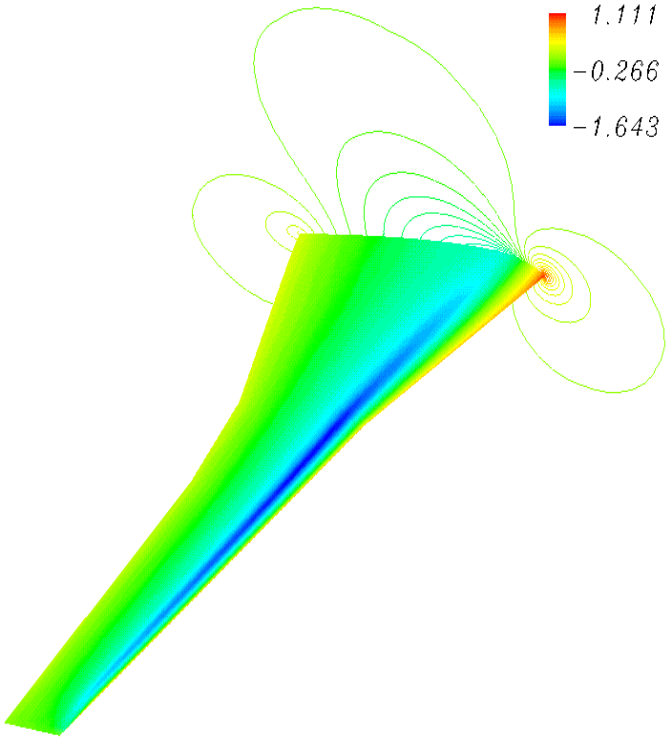
# 適応度の評価 evaluation of fitness





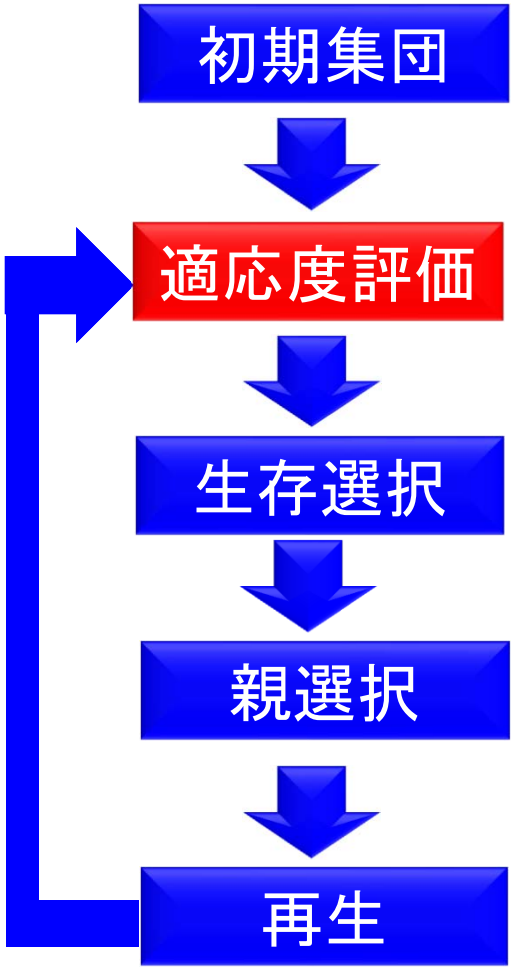


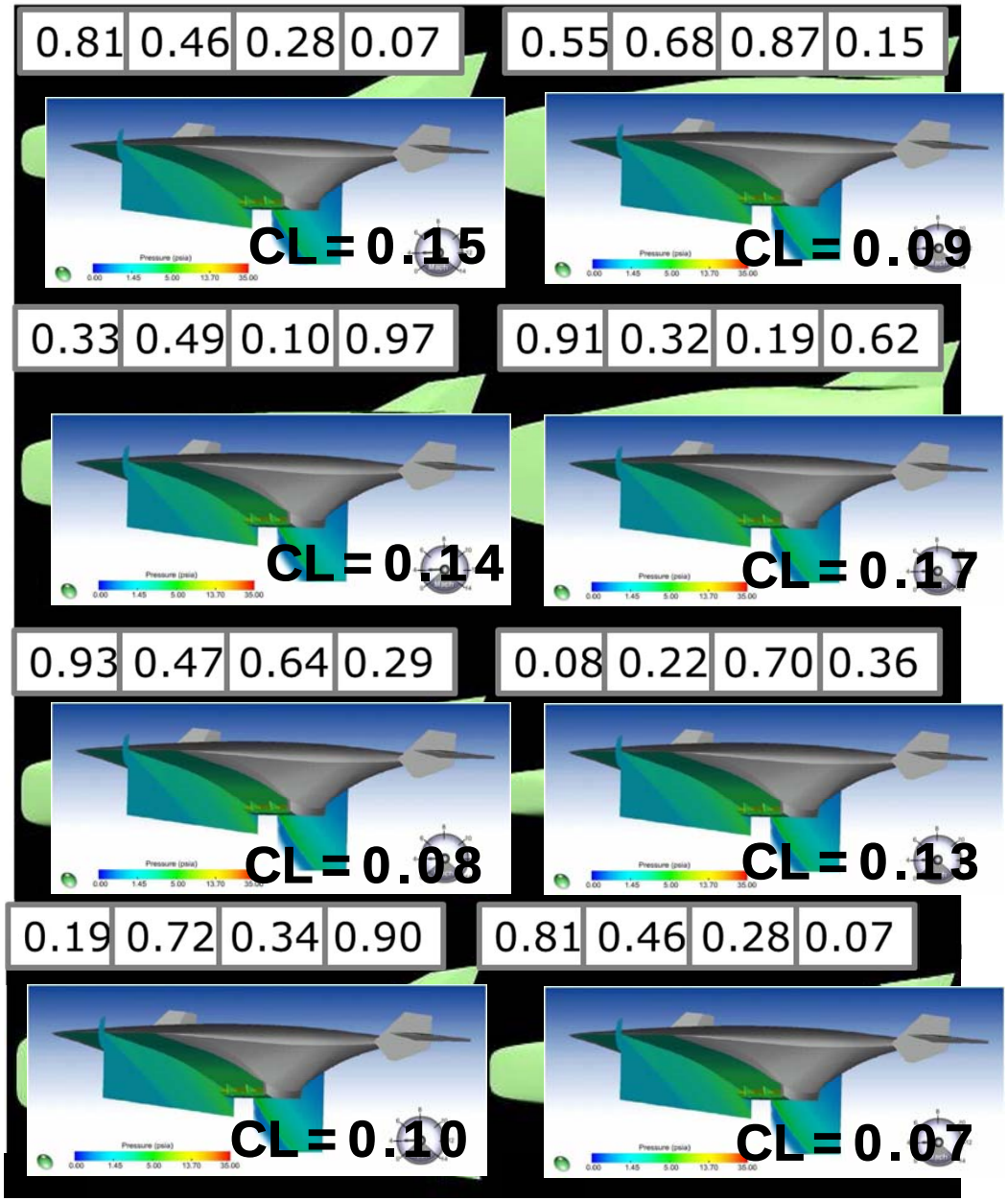
自然界
In nature



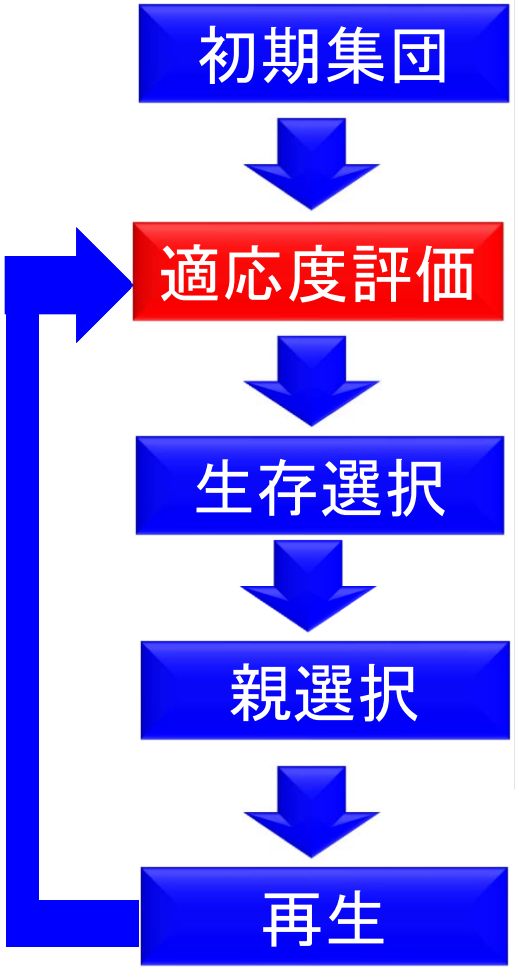

進化アルゴリズム
In evolutionary algorithm

# 適応度の評価 evaluation of fitness



# 適応度の評価 evaluation of fitness

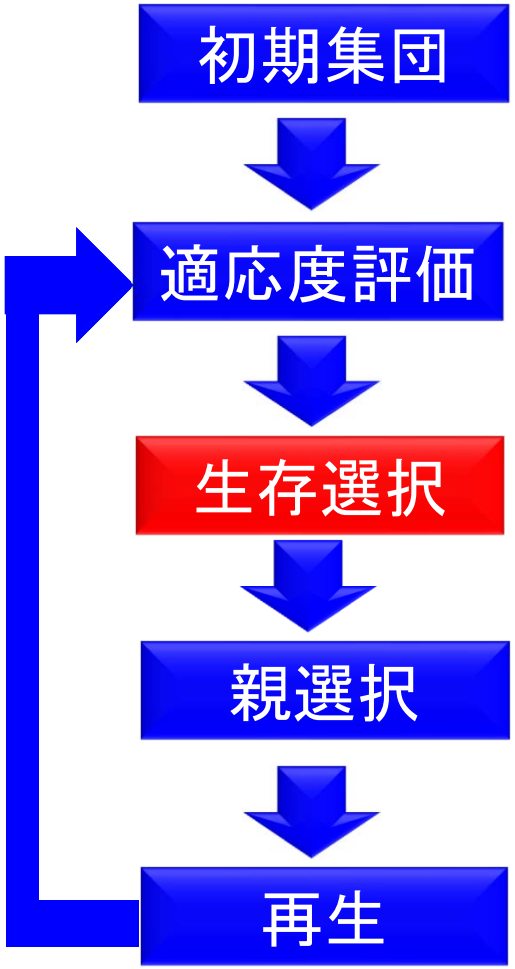



## 単目的最適化問題の場合
In case of single-objective optimization problem

最も簡単な定義 simple definition

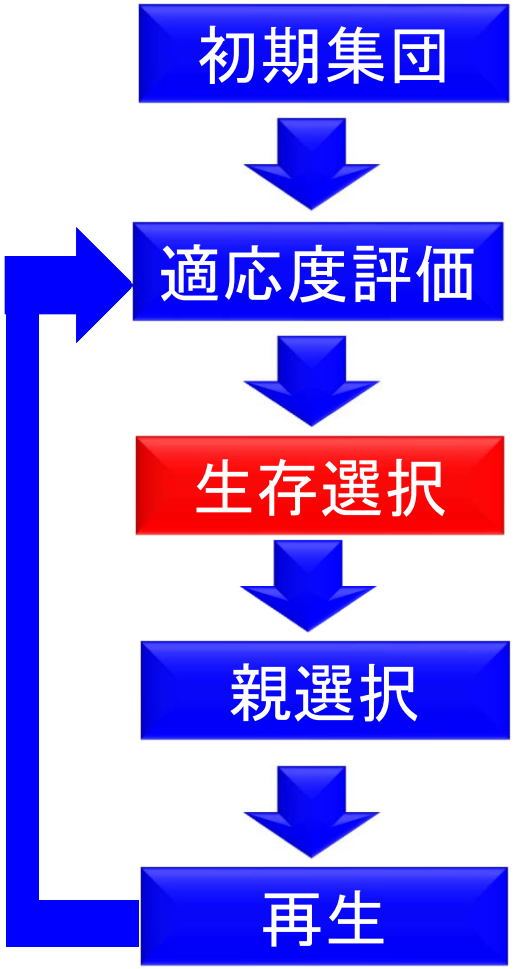
| | |
|---|---|
| 最大化<br>Maximization | 適応度＝目的関数値<br>Fitness = objective function |
| 最小化<br>Minimization | 適応度＝(-1)x目的関数値<br>Fitness = (-1)xobjective function |

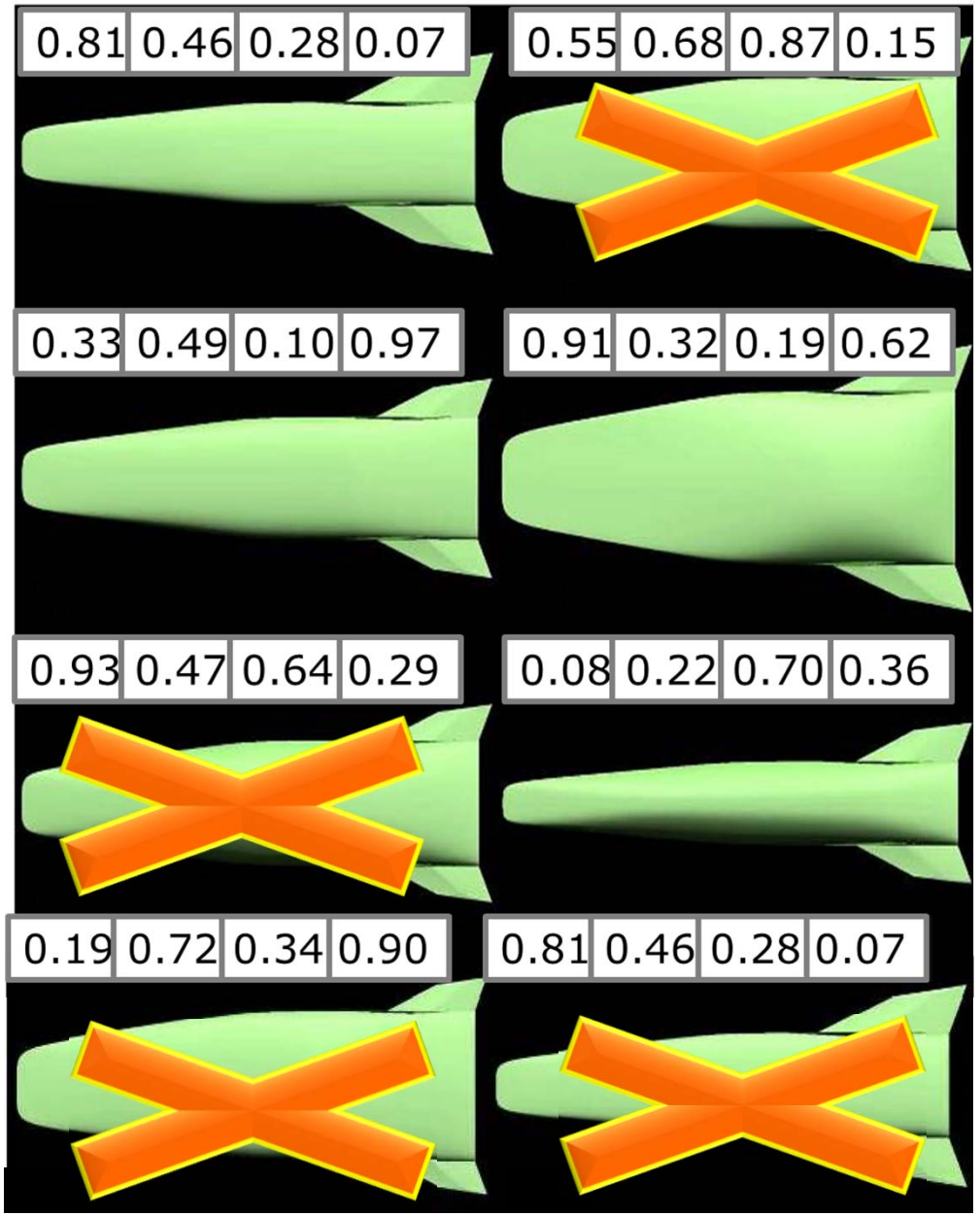
# 生存選択 survival of the fittest



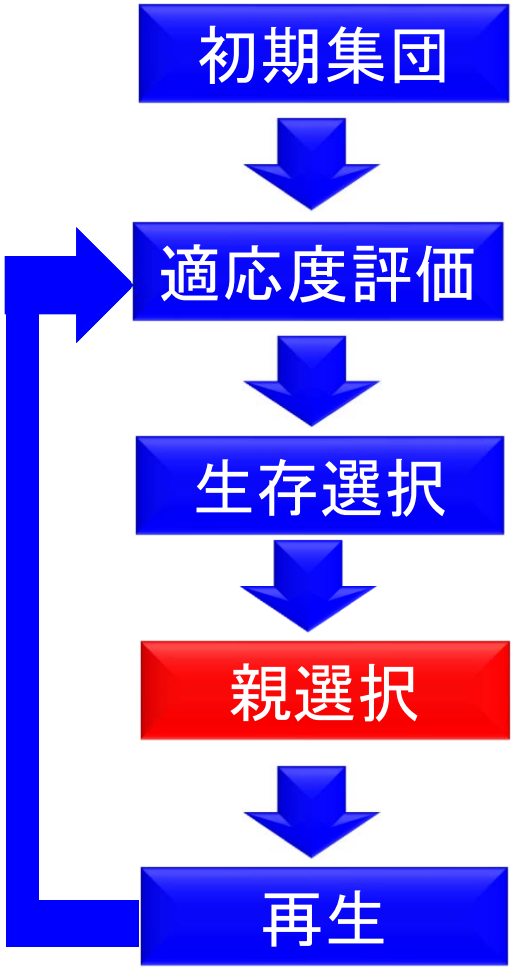

| | fitness | rank | |
|---|---|---|---|
| individual 1 | 200 | 1 | survive |
| individual 2 | 120 | 8 | |
| individual 3 | 140 | 6 | |
| Individual 4 | 180 | 3 | survive |
| individual 5 | 160 | 4 | survive |
| individual 6 | 130 | 7 | |
| individual 7 | 190 | 2 | survive |
| individual 8 | 150 | 5 | |

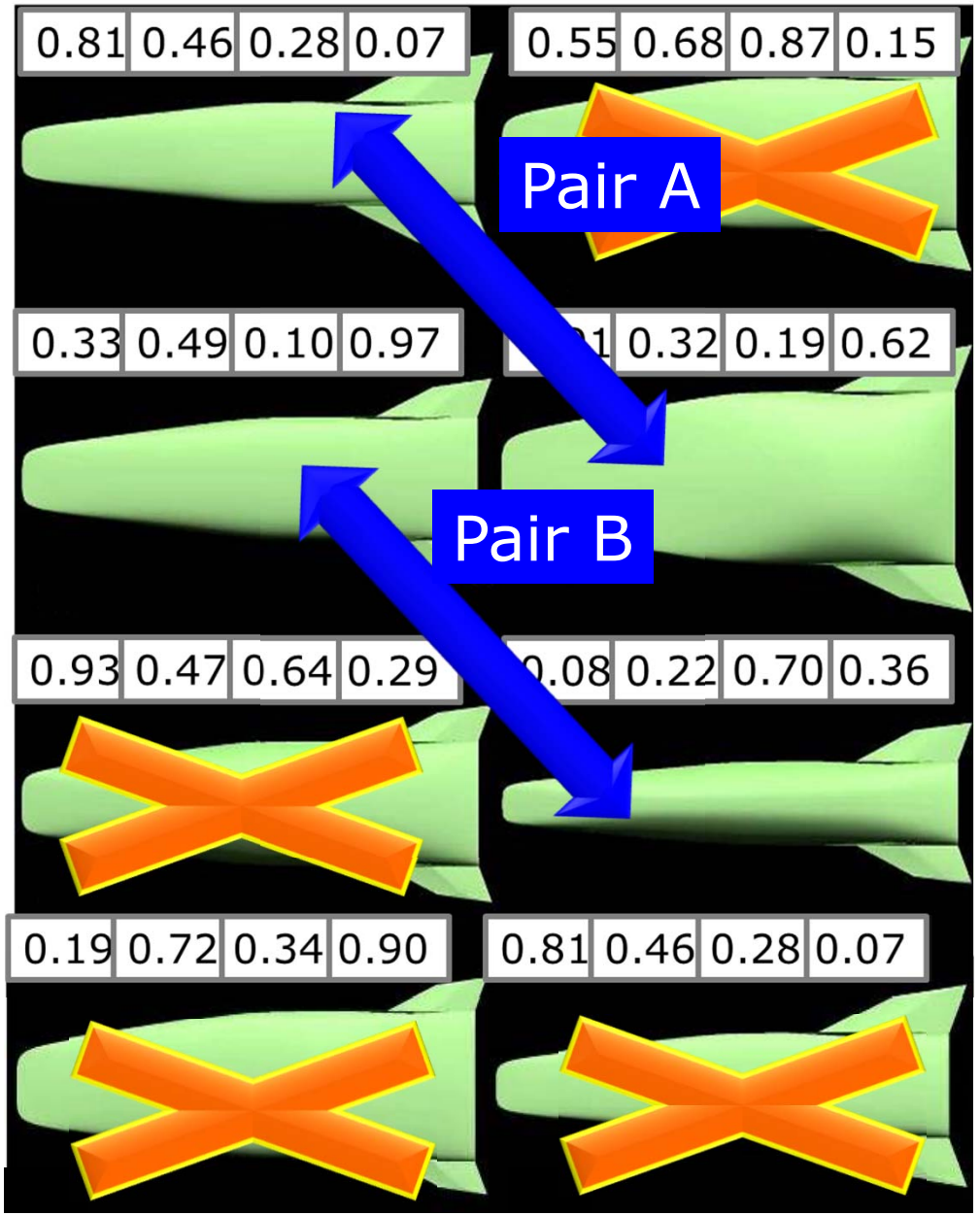
# 適応度が高い設計が生き残る



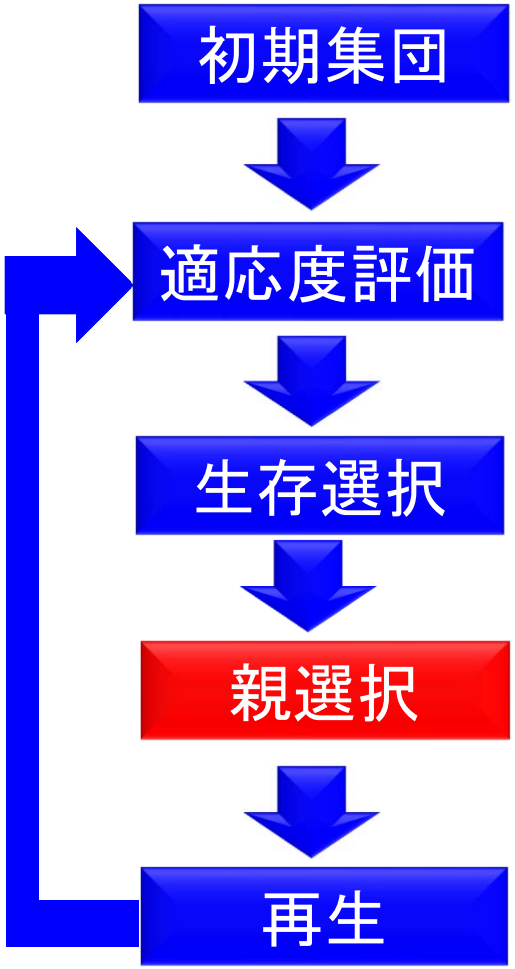



# 親の選択 Parent selection



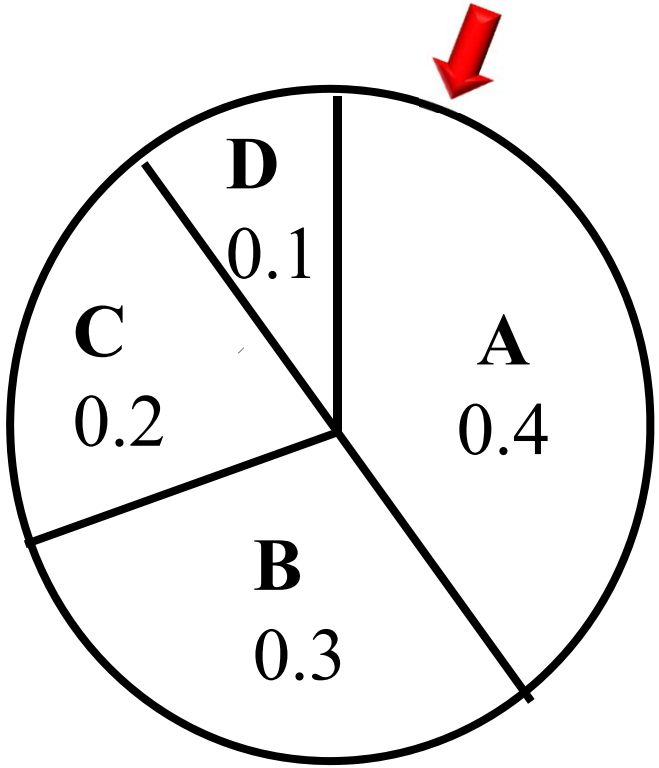



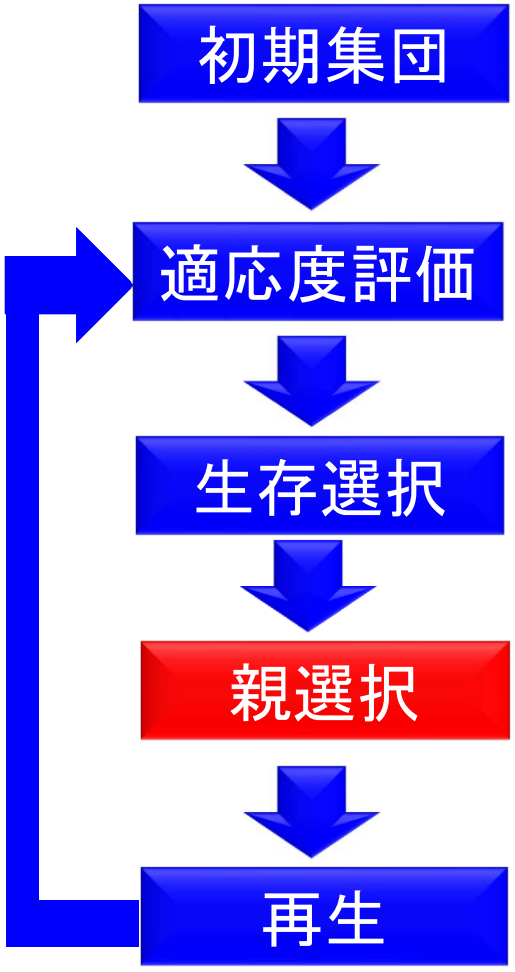
# 親選択 Parent selection

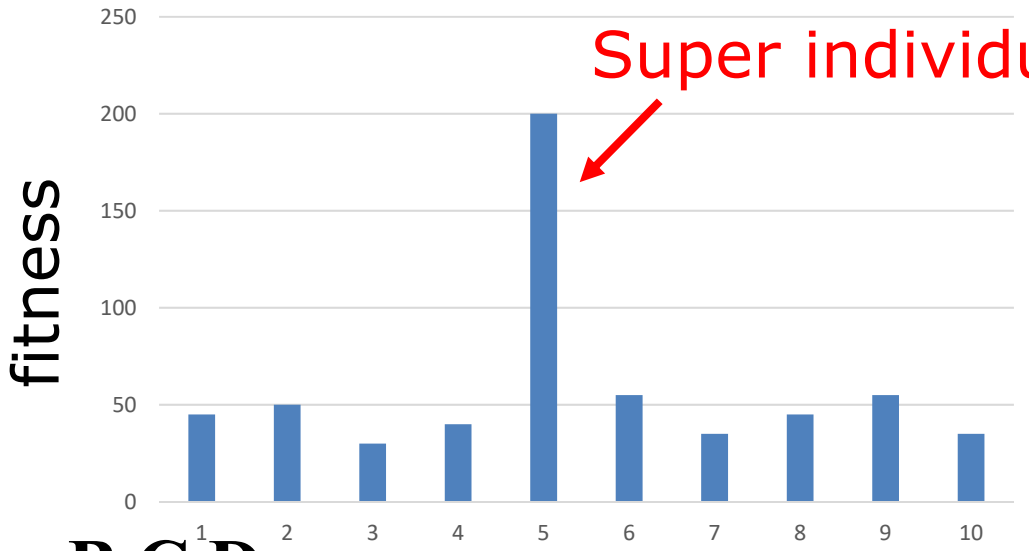


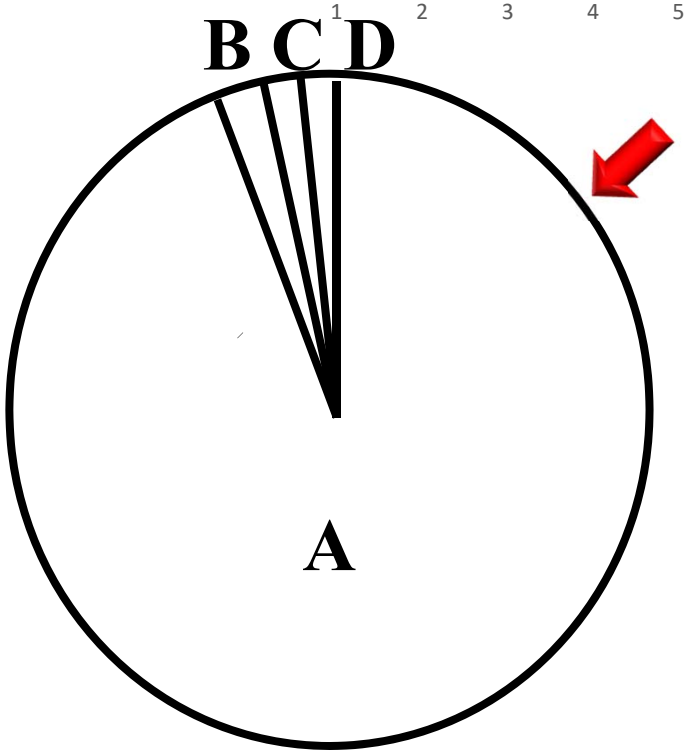

## 一例：ルーレット選択
An example: Roulette selection

| 個体番号 ID | 適応度 Fitness |
|---|---|
| A | 0.4 |
| B | 0.3 |
| C | 0.2 |
| D | 0.1 |





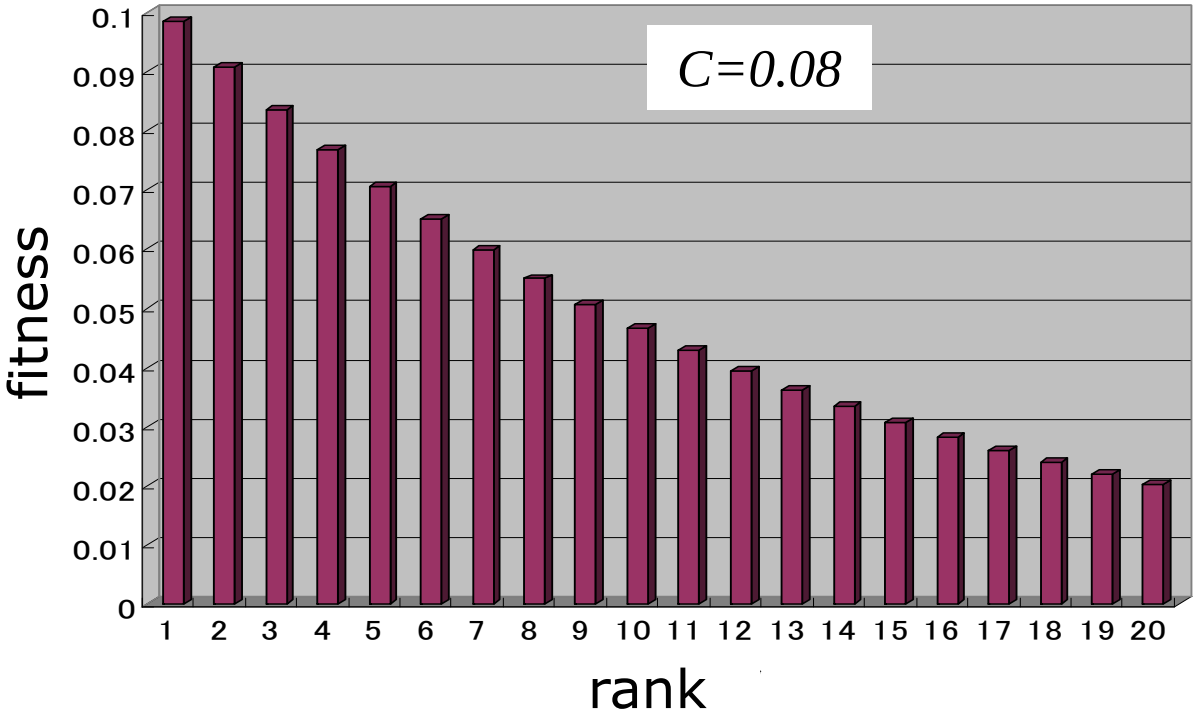

Aばかり選ばれてしまう可能性がある
Only A may be chosen

局所解に収束してしまう
Convergence to local optima

# Nonlinear function of Michalewicz

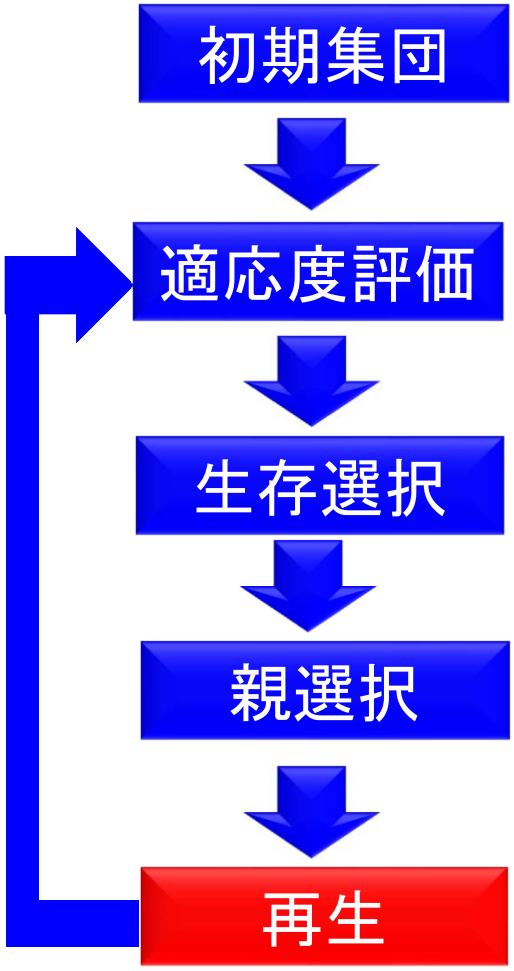
$$fitness = c \cdot (1-c)^{rank-1}$$

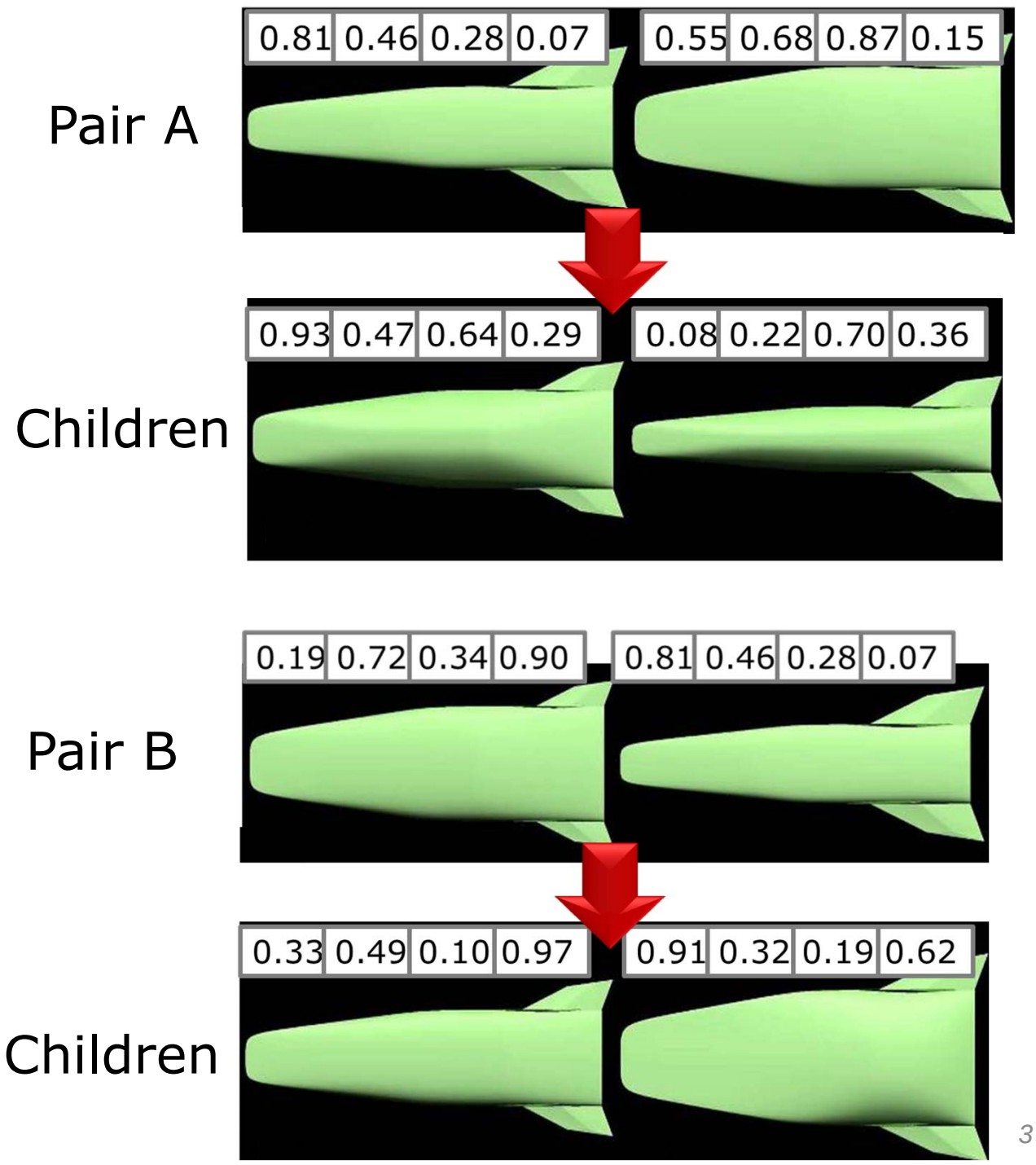
$rank$ is determined by objective function value

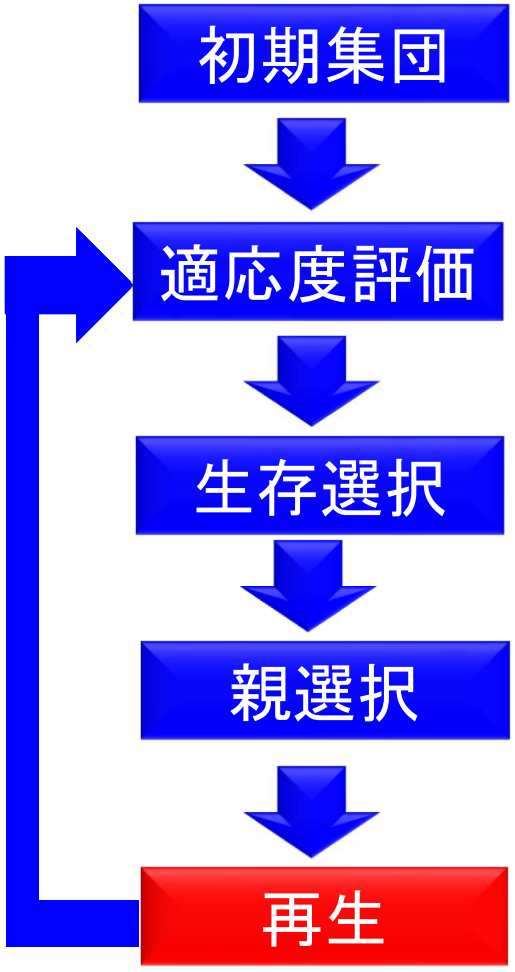



超個体を多数選択してしまうことを避けることが出来る

This avoids selection of super individual many times


初期集団
適応度評価
生存選択
親選択
再生


Pair A
0.81 0.46 0.28 0.07
0.55 0.68 0.87 0.15
Children
0.93 0.47 0.64 0.29
0.08 0.22 0.70 0.36
Pair B
0.19 0.72 0.34 0.90
0.81 0.46 0.28 0.07
Children
0.33 0.49 0.10 0.97
0.91 0.32 0.19 0.62




再生＝交叉＋突然変異

Reproduction = crossover + mutation

交叉　Crossover

最適解がありそうな領域（親の近く）に新しい設計を作る

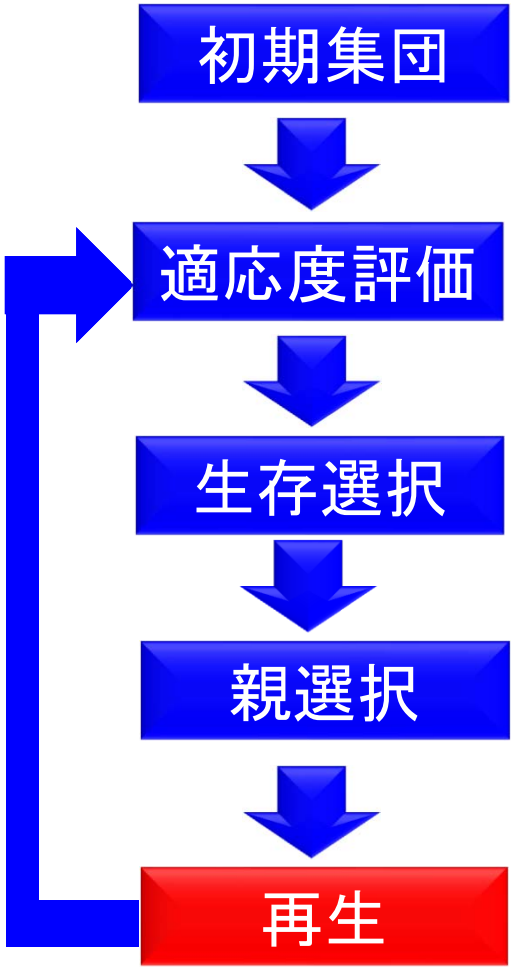
To produce new designs in promising design region (near the parents)

突然変異 Mutation

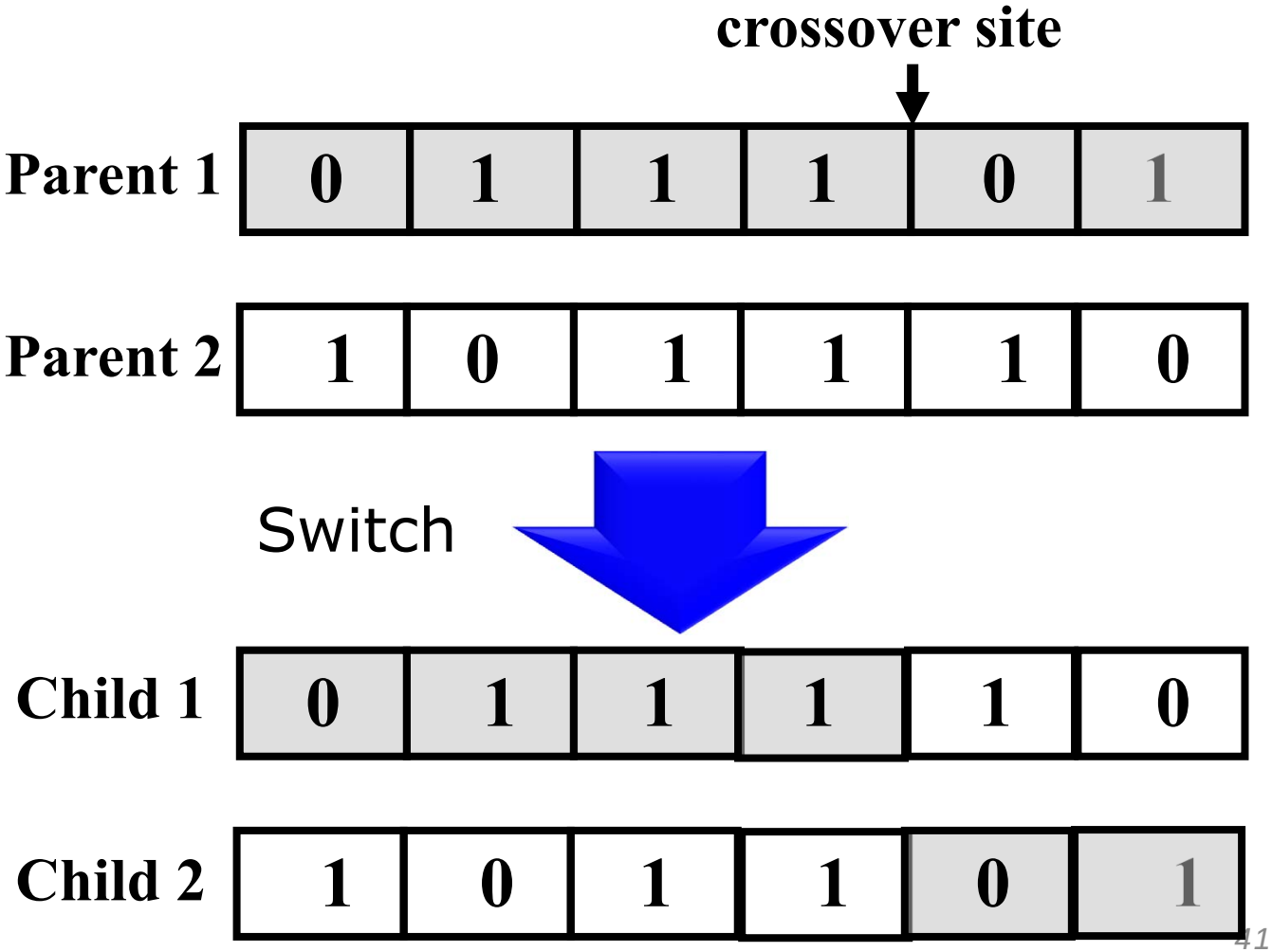
局所解への収束を避ける

To keep diversity in the population and avoid local convergence

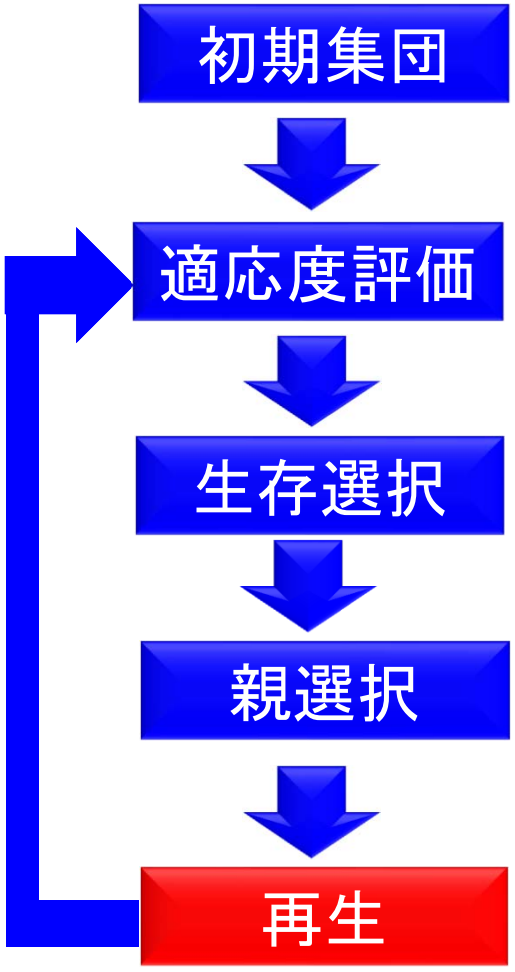
# 一点交叉 One-point crossover



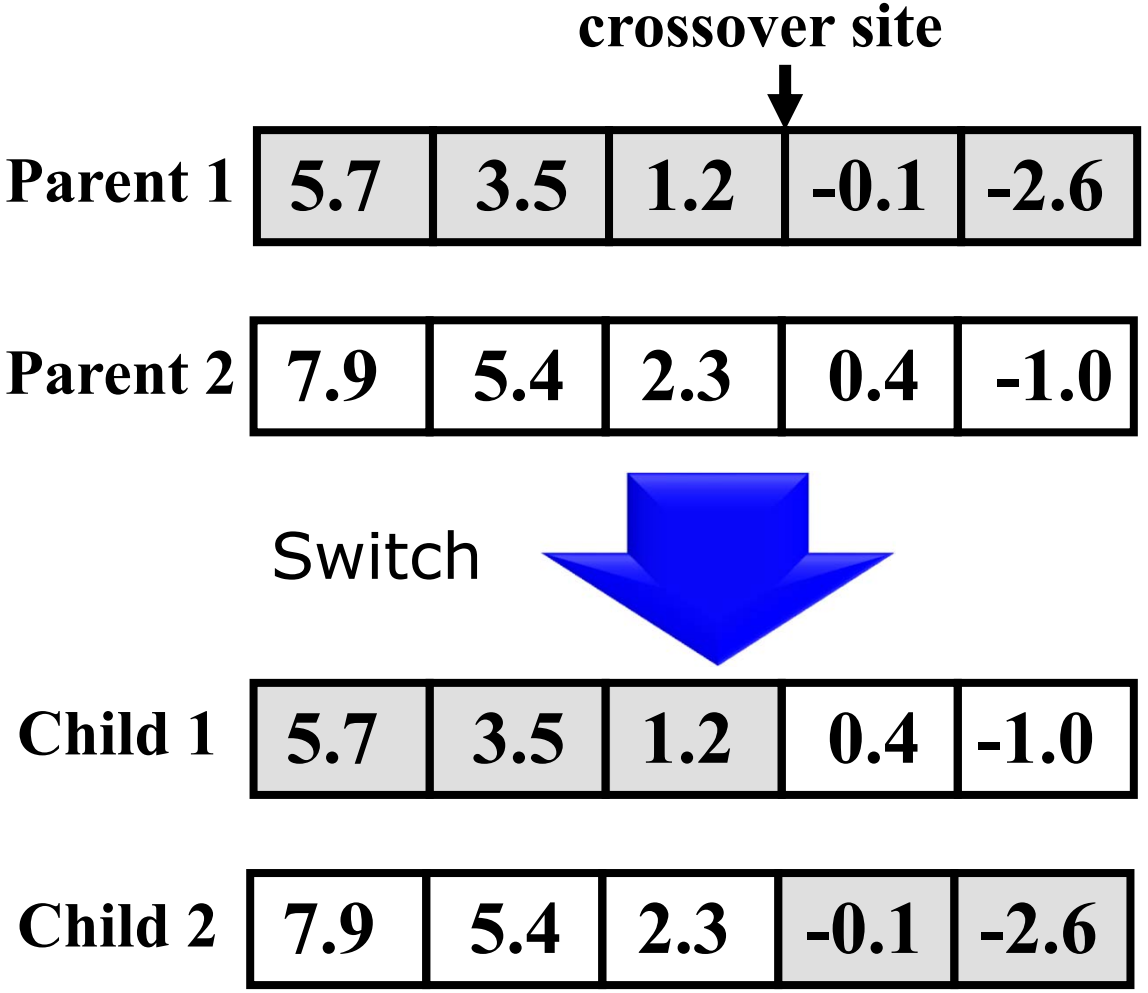

交叉点をランダムに選択
Select crossover site randomly

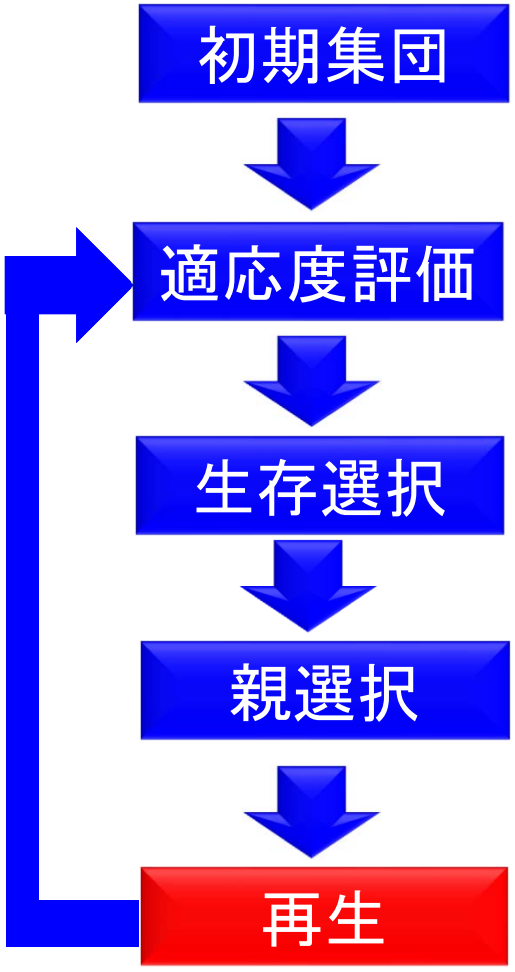


# 一点交叉 One-point crossover




交叉点をランダムに選択
Select crossover site randomly





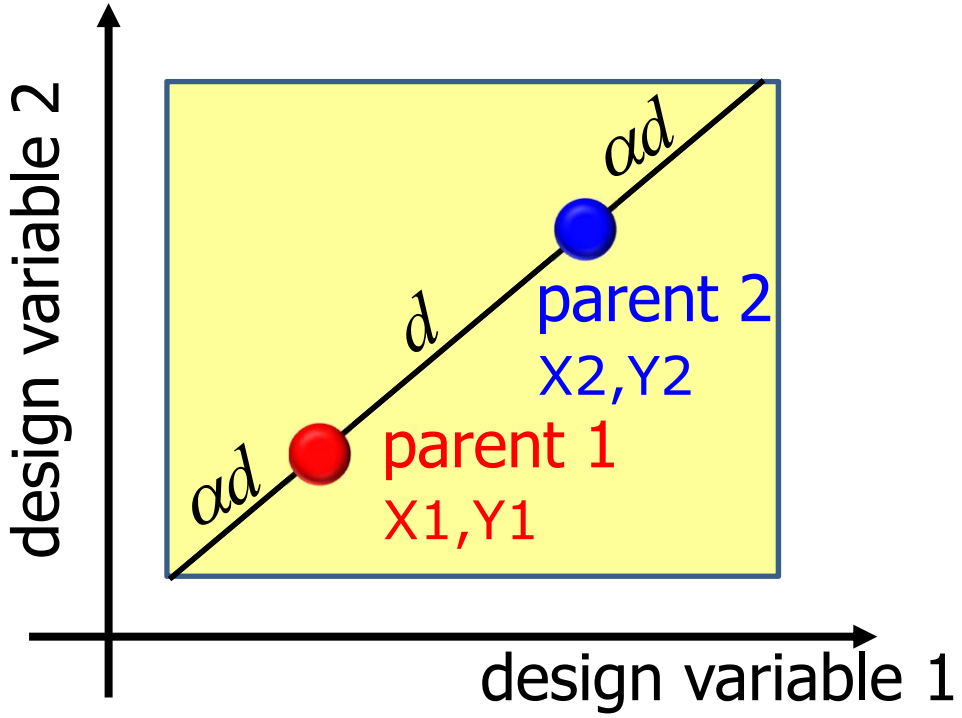

# Blended Crossover

$$X_{child1} = \gamma_1 \times X_{parent1} + (1-\gamma_1) \times X_{parent2}$$
$$Y_{child1} = \gamma_2 \times Y_{parent1} + (1-\gamma_2) \times Y_{parent2}$$

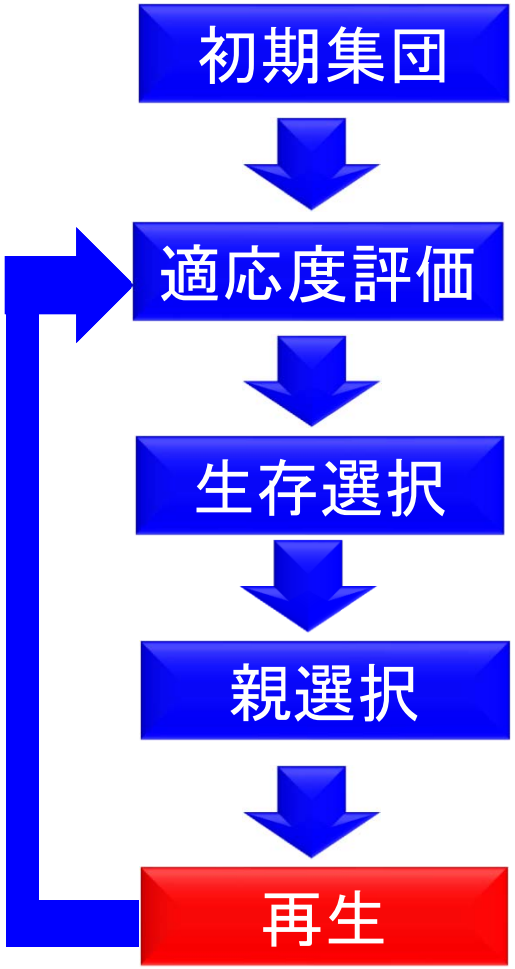
$$\gamma_i = (1+2\alpha) \times rand - \alpha$$

*rand* is random number between 0 and 1

$\alpha$ is usually 0.5



# Blended Crossover




**Algorithm BLX-*a***

select two parents $X^{(1)}$and $X^{(2)}$ from a parent pool

Set alpha $\alpha$=0.5

create two offspring $x^{(1)}$ and $x^{(2)}$ as follows:

for $i$ =1 to $n$ do　（それぞれの設計変数について）

$d_i = | x_i^{(1)} - x_i^{(2)} |$　（距離を求める）

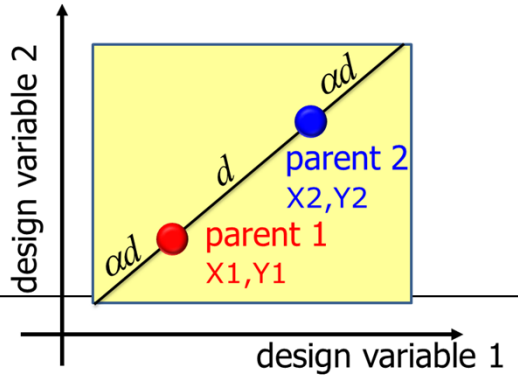
choose a uniform random real number 0<*rand*<1

$\gamma_i = (1+2\alpha) \times rand - \alpha$　（−αから1+αまでの乱数）

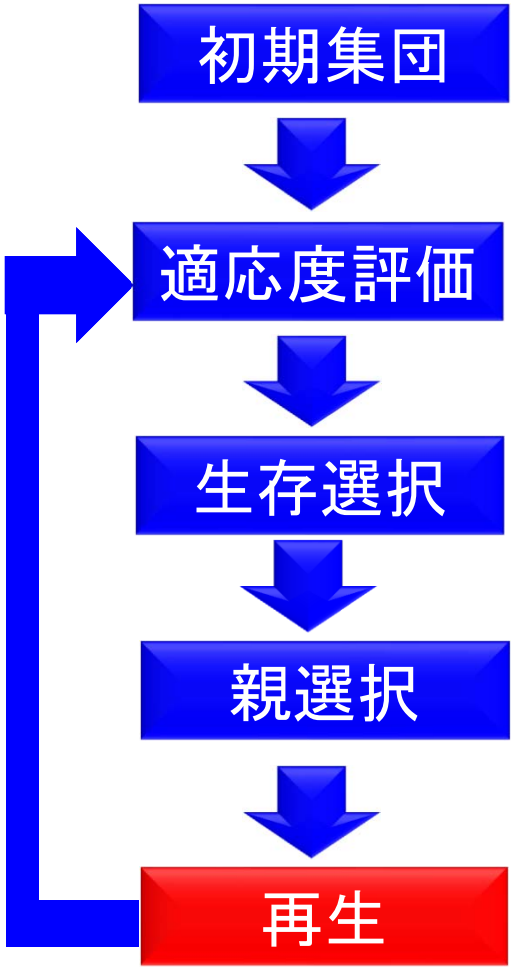
$x_i^{(1)} = \gamma_1 X_i^{(1)} + (1-\gamma_1) X_i^{(2)}$

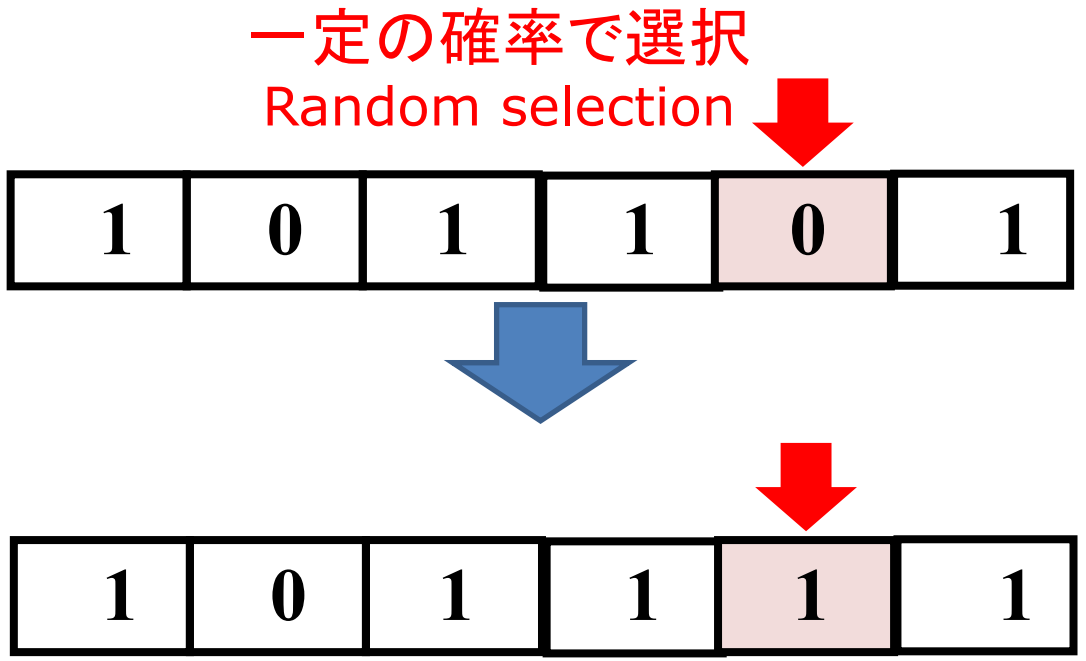
$x_i^{(2)} = (1-\gamma_1) X_i^{(1)} + \gamma_1 X_i^{(2)}$

end do

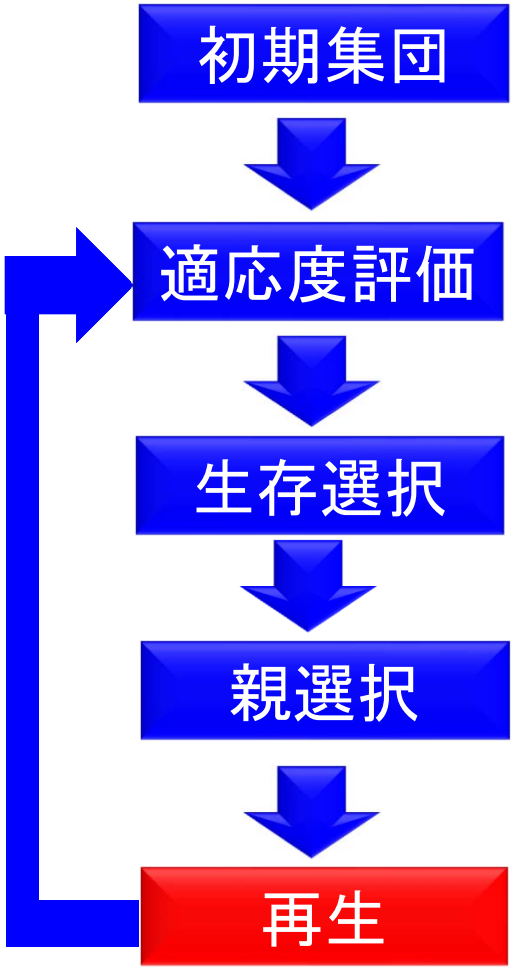


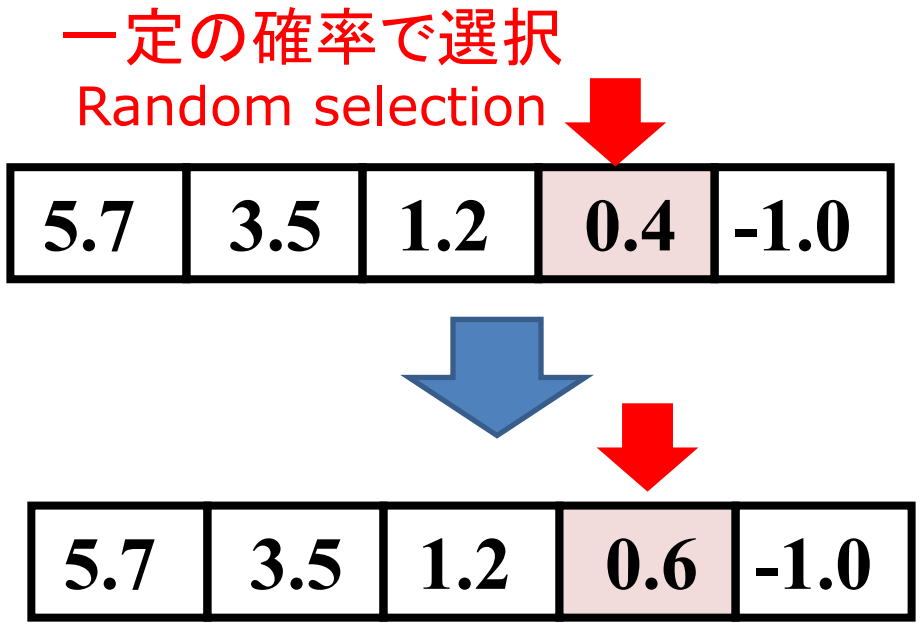
# 突然変異 mutation



# 突然変異 mutation




For each design variable, if *rand* is smaller than *mr*,

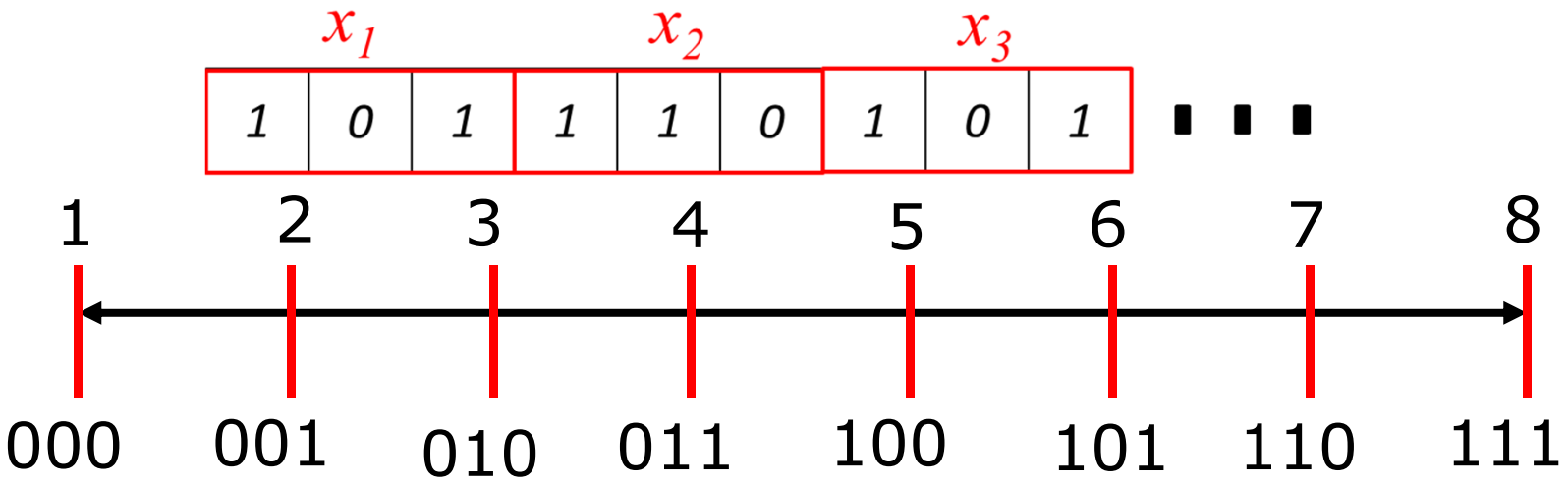
$x_1 = x_1 + rand$

*rand* is random number between 0 and 1

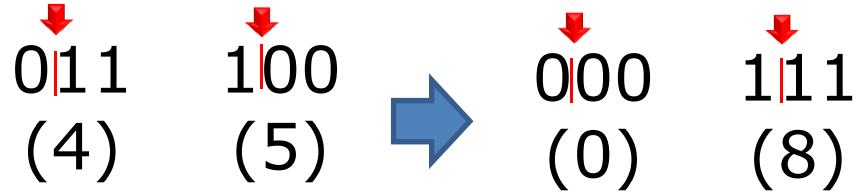
*mr* is mutation rate,

recommended value is 1 / (number of design parameters)

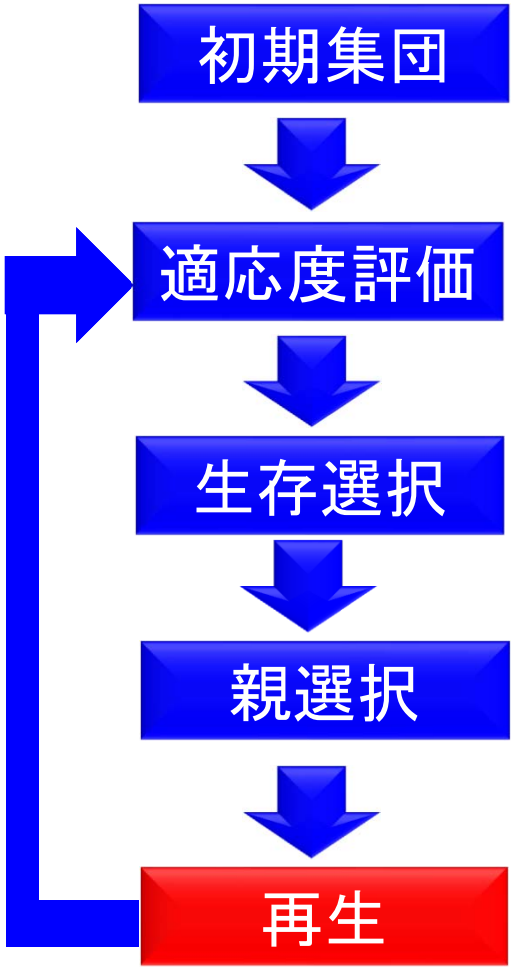
実数最適化問題においては，2進数コーディングは探査効率が悪くなるので，通常，実数コーディングを用いる
In real number optimization problem, real-coding is usually used because optimization efficiency of EA with binary coding is poor

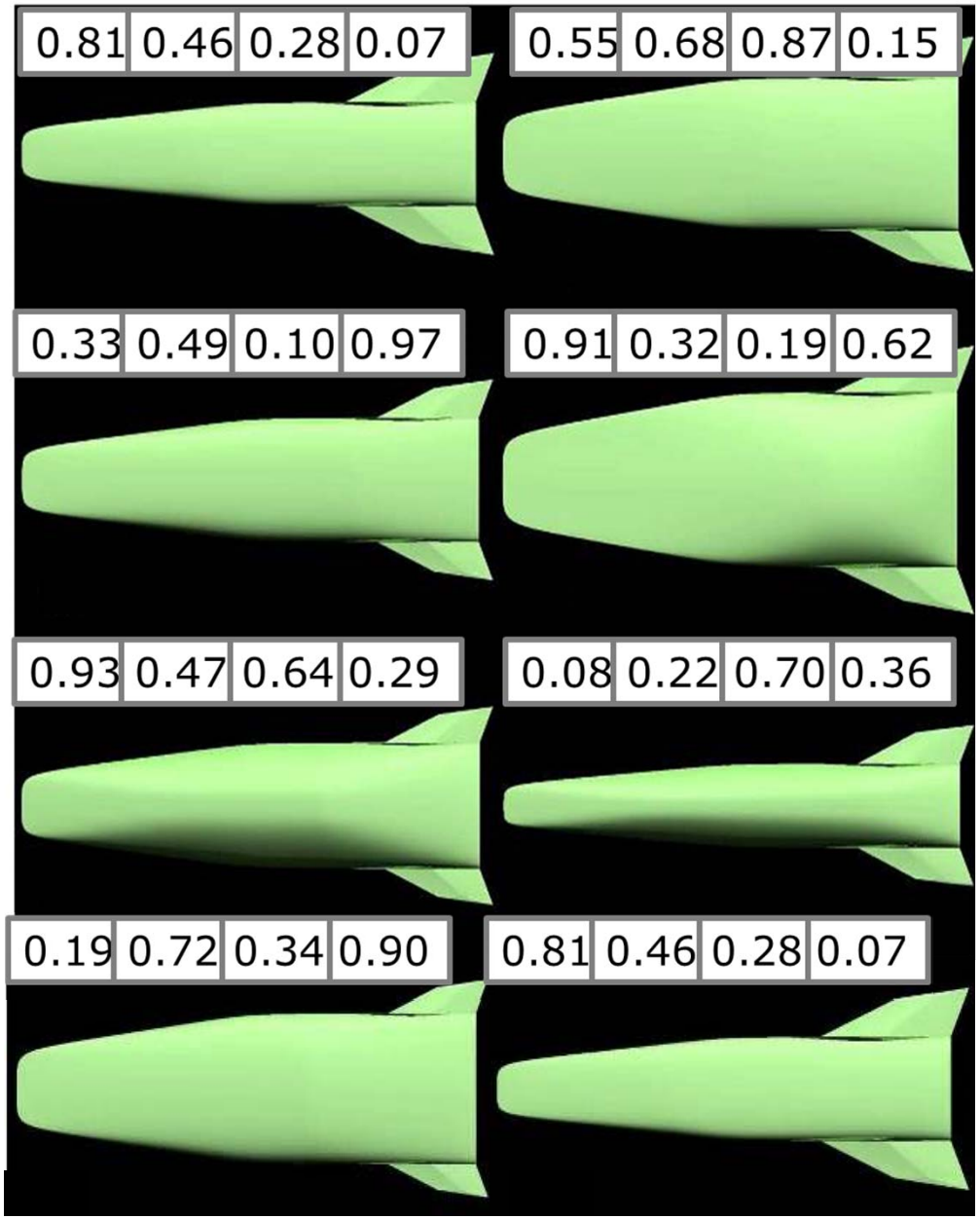



2.5が最適解の場合，最適解にたどり着けない
Can not find optimum if it is 2.5
近い物同士を交叉させても近くに子供が出来るとは限らない
Children of parents



# 次世代の集団 next generation



# BLX交叉
# を計算してみよう
# Let’s try BLX crossover

# 問題 question

親aと親bから子供Aと子供Bを作成し図に示せ

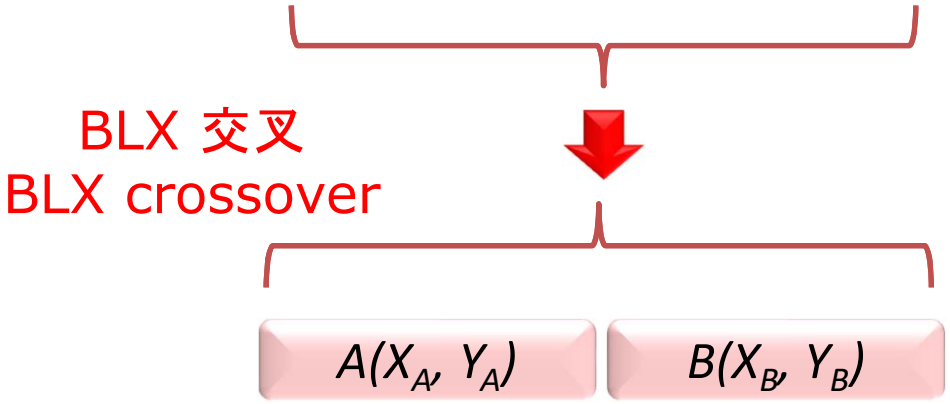
Produce children A and B from Parents a and b. Draw the positions

設計変数 design parameters ：X , Y

集団サイズ population size ：4

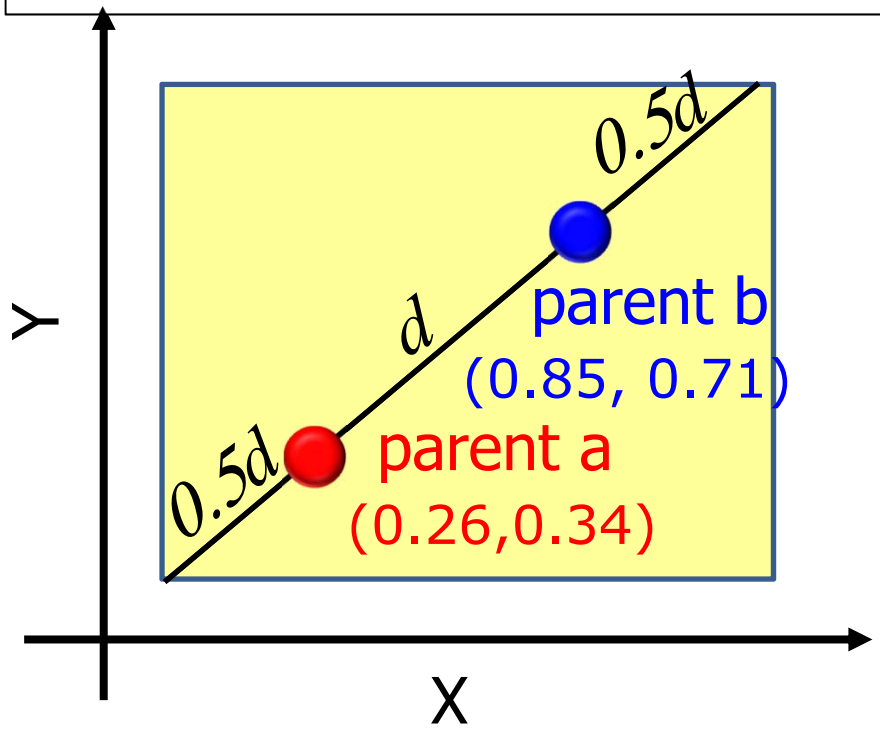
Individuals in the Mating pool

| | design a | design b | design c | design d |
|---|---|---|---|---|
| X | 0.26 | 0.85 | 0.XX | 0.XX |
| Y | 0.34 | 0.71 | 0.XX | 0.XX |



# 交叉にはBlended Crossover-0.5を採用Adopt Blended Crossover-0.5

| | design a | design b |
|---|---|---|
| X | 0.26 | 0.85 |
| Y | 0.34 | 0.71 |




$$X_A = \gamma_1 \times X_a + (1-\gamma_1) \times X_b$$
$$Y_A = \gamma_2 \times Y_a + (1-\gamma_2) \times Y_b$$
$$X_B = \gamma_3 \times X_a + (1-\gamma_3) \times X_b$$
$$Y_B = \gamma_4 \times Y_a + (1-\gamma_4) \times Y_b$$

$$\gamma_i = 2 \times rand - 0.5$$

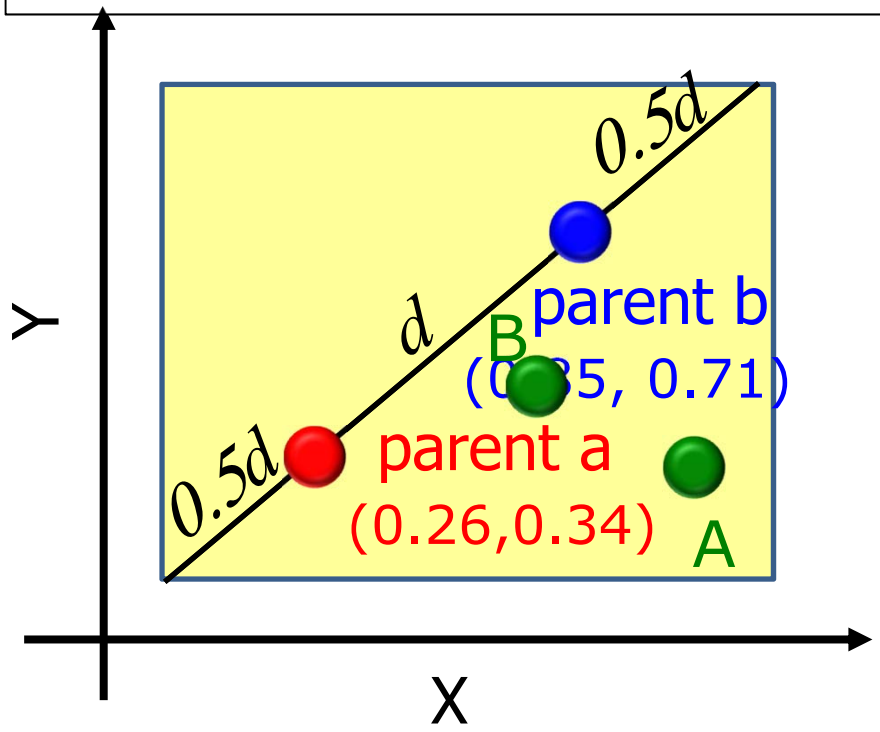
$rand$ is a random number [0,1]

→ $-0.5 < \gamma_i < 1.5$

$$rand = 0.2, 0.8, 0.4, 0.6, \ldots$$

# 交叉にはBlended Crossover-0.5を採用Adopt Blended Crossover-0.5

| | design a | design b |
|---|---|---|
| X | 0.26 | 0.85 |
| Y | 0.34 | 0.71 |




$$X_A = \gamma_1 \times X_a + (1-\gamma_1) \times X_b$$
$$Y_A = \gamma_2 \times Y_a + (1-\gamma_2) \times Y_b$$
$$X_B = \gamma_3 \times X_a + (1-\gamma_3) \times X_b$$
$$Y_B = \gamma_4 \times Y_a + (1-\gamma_4) \times Y_b$$

$$\gamma_i = 2 \times rand - 0.5$$

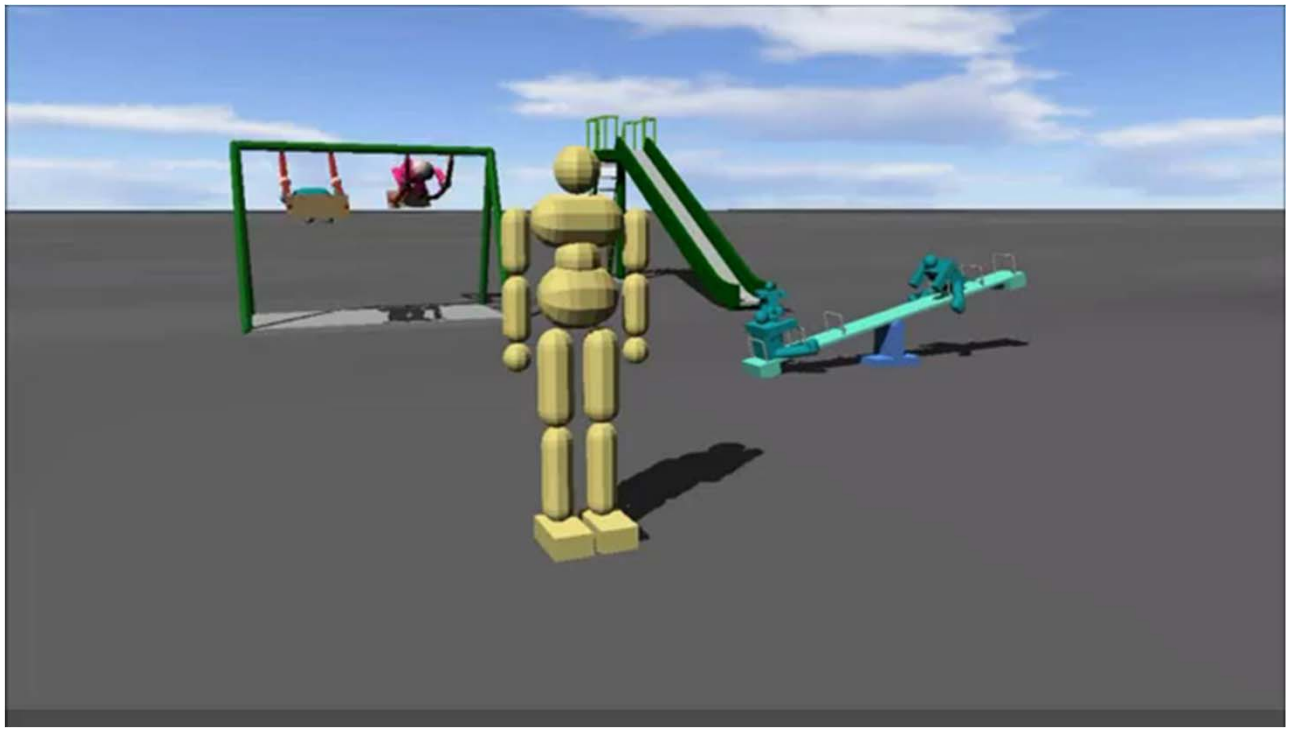
*rand* is a random number [0,1]

→ $-0.5 < \gamma_i < 1.5$

$$rand = 0.2, 0.8, 0.4, 0.6, \ldots$$

| | rand | gamma | |
|---|---|---|---|
| XA | 0.2 | -0.1 | 0.909 |
| YA | 0.8 | 1.1 | 0.303 |
| XB | 0.4 | 0.3 | 0.673 |
| YB | 0.6 | 0.7 | 0.451 |

# 進化アルゴリズムを使ってハイハイを学習させた
Evolutionary algorithm used for learning how to crawling



https://www.youtube.com/watch?v=gWExx-NpimQ

# 進化アルゴリズムの特徴
## features of evolutionary algorithm

- 多点探査

  Multi-point exploration

- 適応度（目的関数値＆設計変数値）に基づく探査（勾配情報を用いない）

  Exploration based on fitness defined by objective function and constraint function values (no gradient information is used)

- 確率的オペレータを使った計算（決定論的手法ではない）

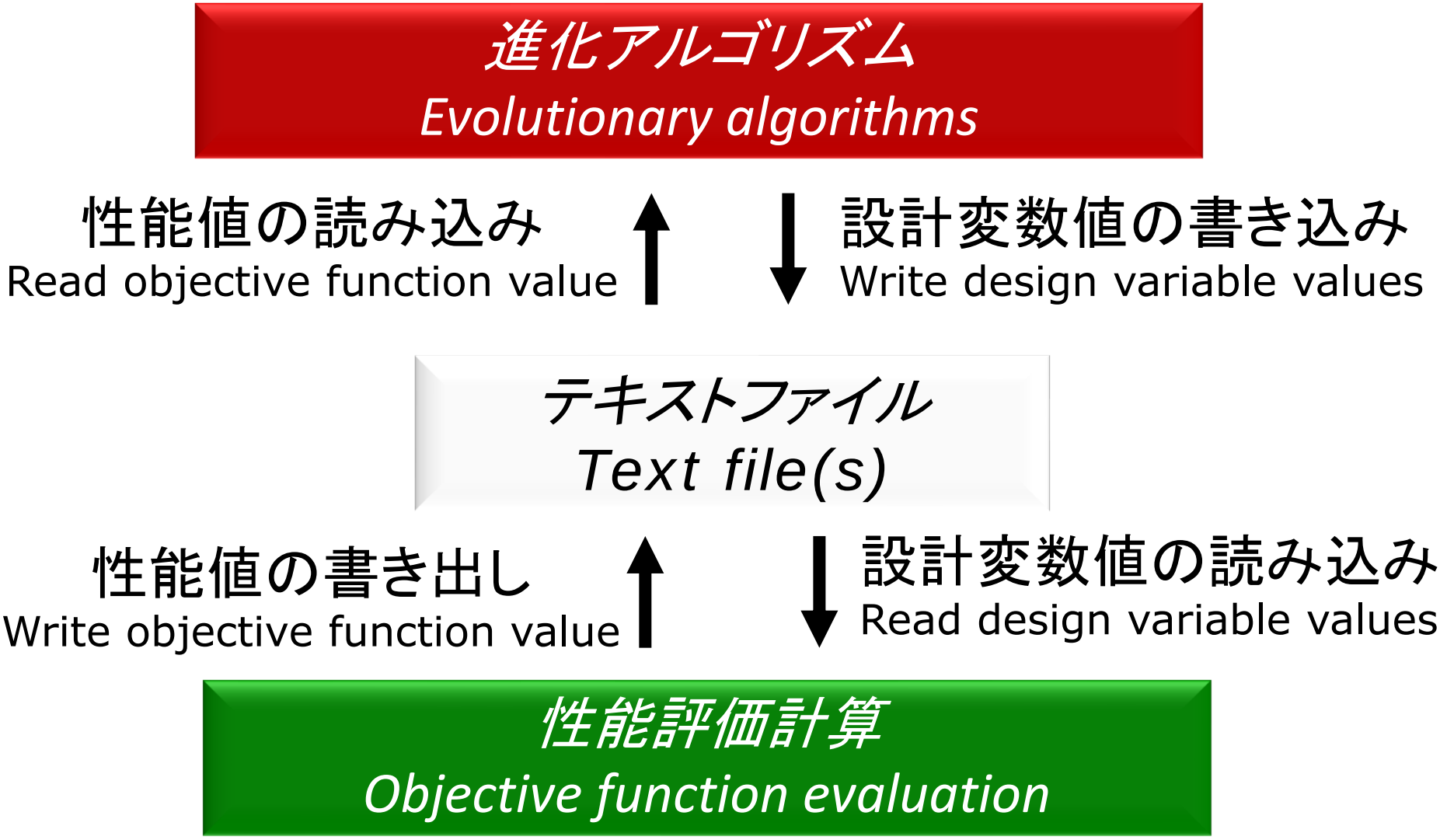
  Probabilistic computation (not deterministic computation)

# (単目的)進化アルゴリズムの利点・欠点

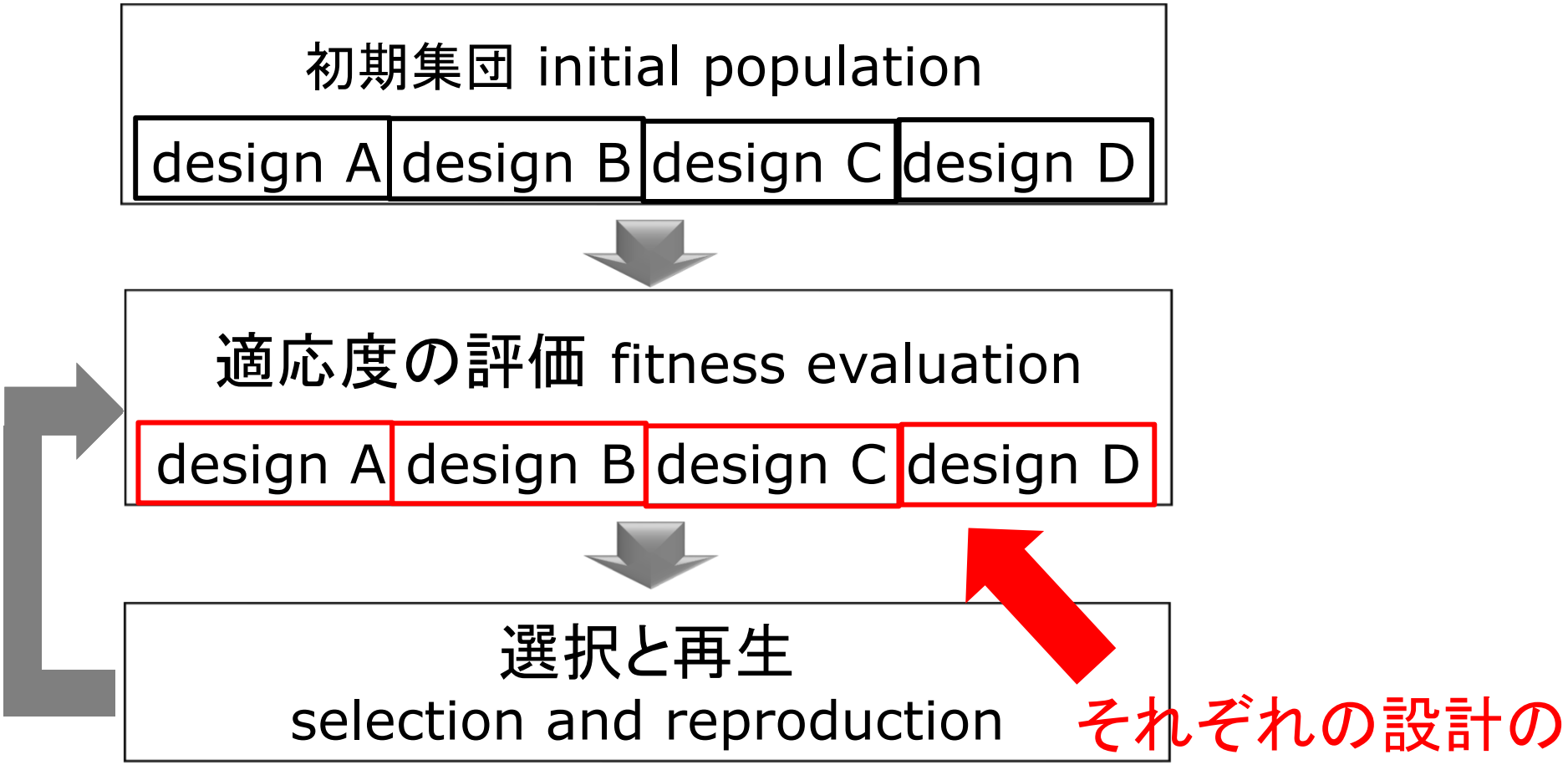
advantages/disadvantages of (single-objective) evolutionary algirithms

- 目的関数分布が多峰的である場合でも，大域的探査が可能
  Global search even if the objective function is multimodal
- 不連続最適化問題も解くことが可能(整数などを含む)
  Applicable to design problems with discontinuous objective function
- 適用が簡単(ブラックボックスを準備すればOK)
  Application is easy
- 並列化効率が高い
  Parallelization efficiency is very high

- 探査効率では勾配法などより劣る(たくさんの評価が必要)
  Less efficient compared with gradient-based method
- 多目的最適化問題に用いる場合は多数あるパレート最適解のうちの1つしか見つけることができない
  Find only one of the Pareto-optimal solutions

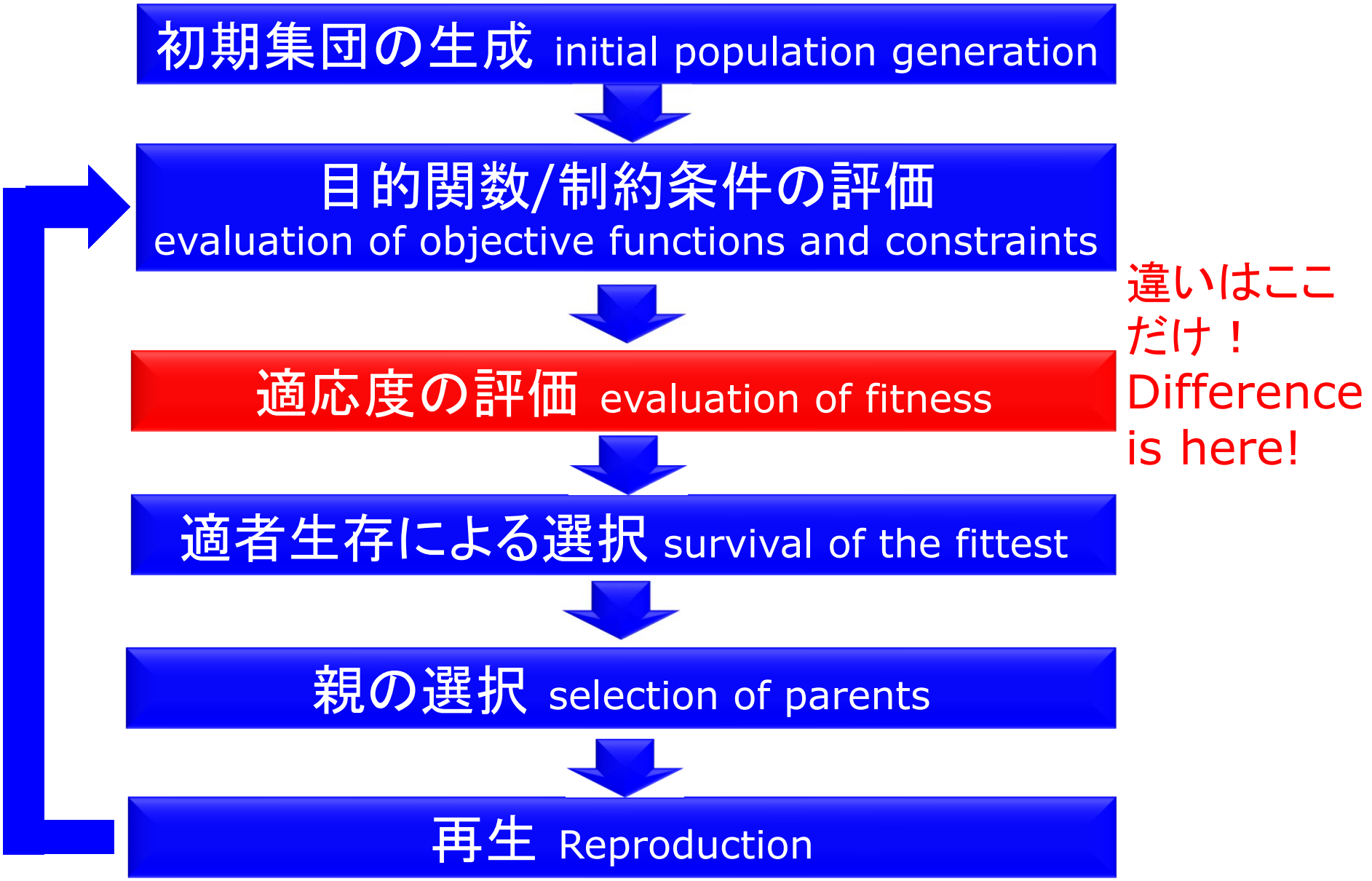



進化アルゴリズムは適用が容易
EA is easy to be implemented


初期集団 initial population
design A
design B
design C
design D
適応度の評価 fitness evaluation
design A
design B
design C
design D
選択と再生
selection and reproduction
それぞれの設計の
適応度評価は
並列実行可能
Evaluation can be
parallelized

# 多目的進化アルゴリズム

multiobjective evolutionary algorithms (MOEAs)


初期集団の生成 initial population generation
目的関数/制約条件の評価
evaluation of objective functions and constraints
適応度の評価 evaluation of fitness
適者生存による選択 survival of the fittest
親の選択 selection of parents
再生 Reproduction
違いはここだけ！
Difference is here!

# 多目的進化アルゴリズムの生い立ち History of MOEA

- 最初の進化アルゴリズム　The first EA algorithms (1970's)
- 最初の多目的進化アルゴリズムThe first MOEC (Schaffer, 1984)

  Vector-Evaluated Genetic Algorithm
- 多目的進化計算への非劣解・劣解の概念の導入(Goldberg, 1989)

  Concept of non-dominated/dominated solutions
- 実用的な多目的進化アルゴリズム

  Multi-Objective Genetic Algorithm (Fonseca and Fleming, 1993)
  - 非劣解・劣解の概念の導入　　Concept of domination
  - 適応度シェアリングの概念の導入　　Concept of sharing
- SPEA2 (Zitzler et al, 2001), NSGA-II(Deb et al., 2002), …
- NSGA-III (Deb, 2013), IBEA, MOEA/D, etc…

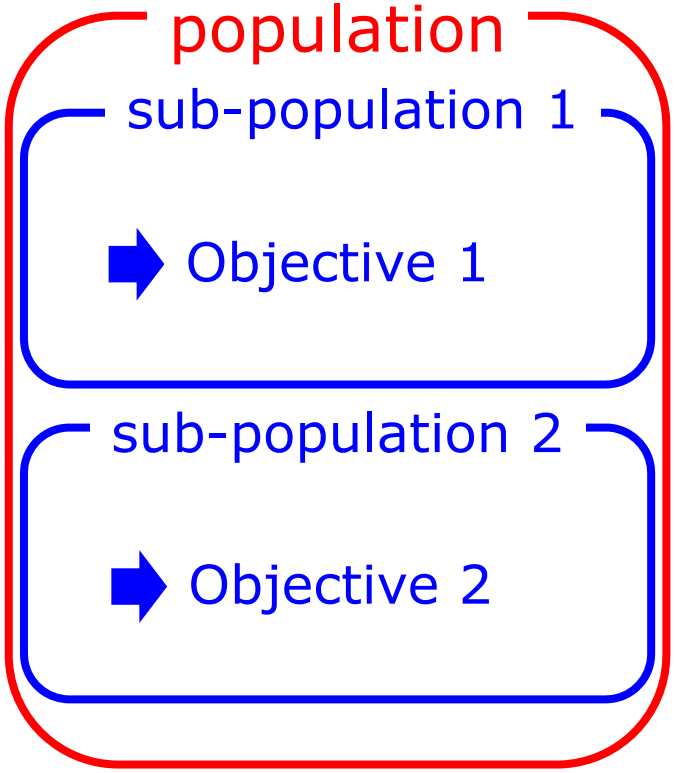
# Vector-Evaluated Genetic Algorithm (Schaffer, 1984)

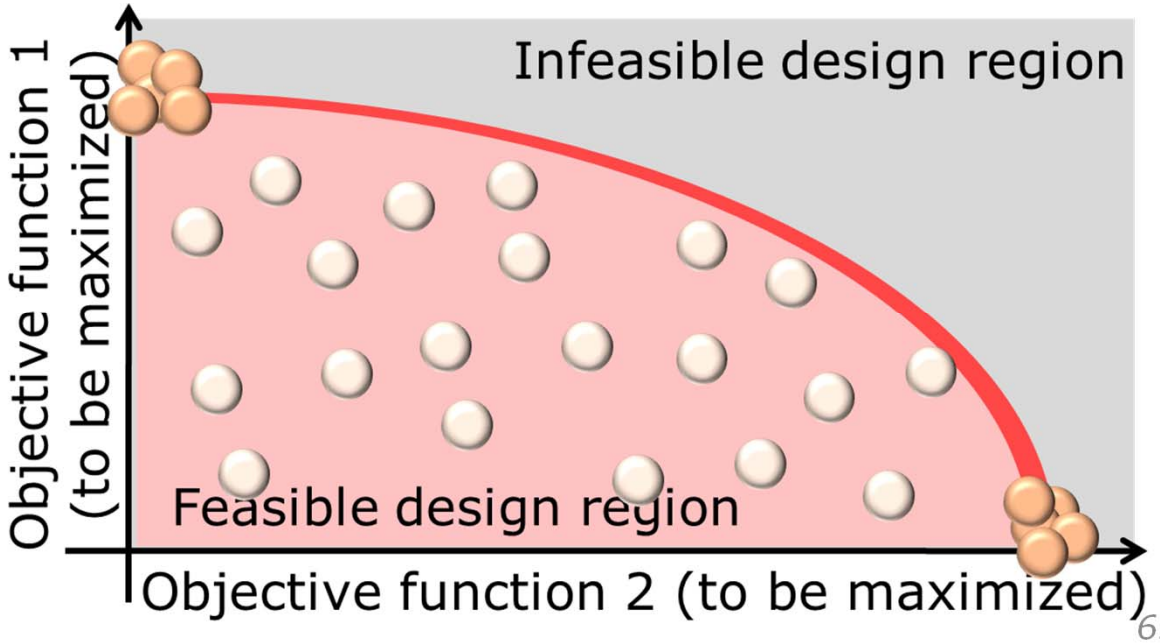
- 目的関数の数に集団を分割し，それぞれの目的関数がよくなるように最適化

  Divide population into subpopulations and each subpopulation optimize one of multiple objectives

- 欠点はパレート面の両端の解しか得られないこと

  Drawback: only extreme solutions are obtained



# 多目的進化アルゴリズムの生い立ち History of MOEA

- 最初の進化アルゴリズム　The first EA algorithms (1970's)
- 最初の多目的進化アルゴリズムThe first MOEA (Schaffer, 1984)

  Vector-Evaluated Genetic Algorithm
- 多目的進化計算への非劣解・劣解の概念の導入(Goldberg, 1989)

  Concept of non-dominated/dominated solutions
- 実用的な多目的進化アルゴリズム

  Multi-Objective Genetic Algorithm (Fonseca and Fleming, 1993)
  - 非劣解・劣解の概念の導入　　Concept of domination
  - 適応度シェアリングの概念の導入　　Concept of sharing
- SPEA2 (Zitzler et al, 2001), NSGA-II(Deb et al., 2002), …
- NSGA-III (Deb, 2013), IBEA, MOEA/D, etc…

# 非劣解・劣解の概念の導入 (Goldberg, 1989)
## Introduction of concept of dominance

- k個の目的関数を持つ最小化問題において，$x_\alpha$ , $x_\beta$ に対して次式が満たされる時，「$x_\beta$は$x_\alpha$に支配されている」あるいは「$x_\beta$は$x_\alpha$の劣解」という

$$F_i(x_\alpha) \le F_i(x_\beta) \qquad i = 1,..,k$$

- 逆に，上式を満たす$x_\beta$が存在しない場合には，「$x_\beta$は支配されない」あるいは「$x_\beta$は非劣解」という.
- 目的関数空間の実行可能領域にある非劣解の集合がパレート最適解となる

---

- If the following relation is satisfied in a minimization problem with k objectives, it is said that "$x_\beta$ is dominated by $x_\alpha$" or "$x_\beta$ is a dominated solution of $x_\alpha$".

$$F_i(x_\alpha) \le F_i(x_\beta) \qquad i = 1,..,k$$

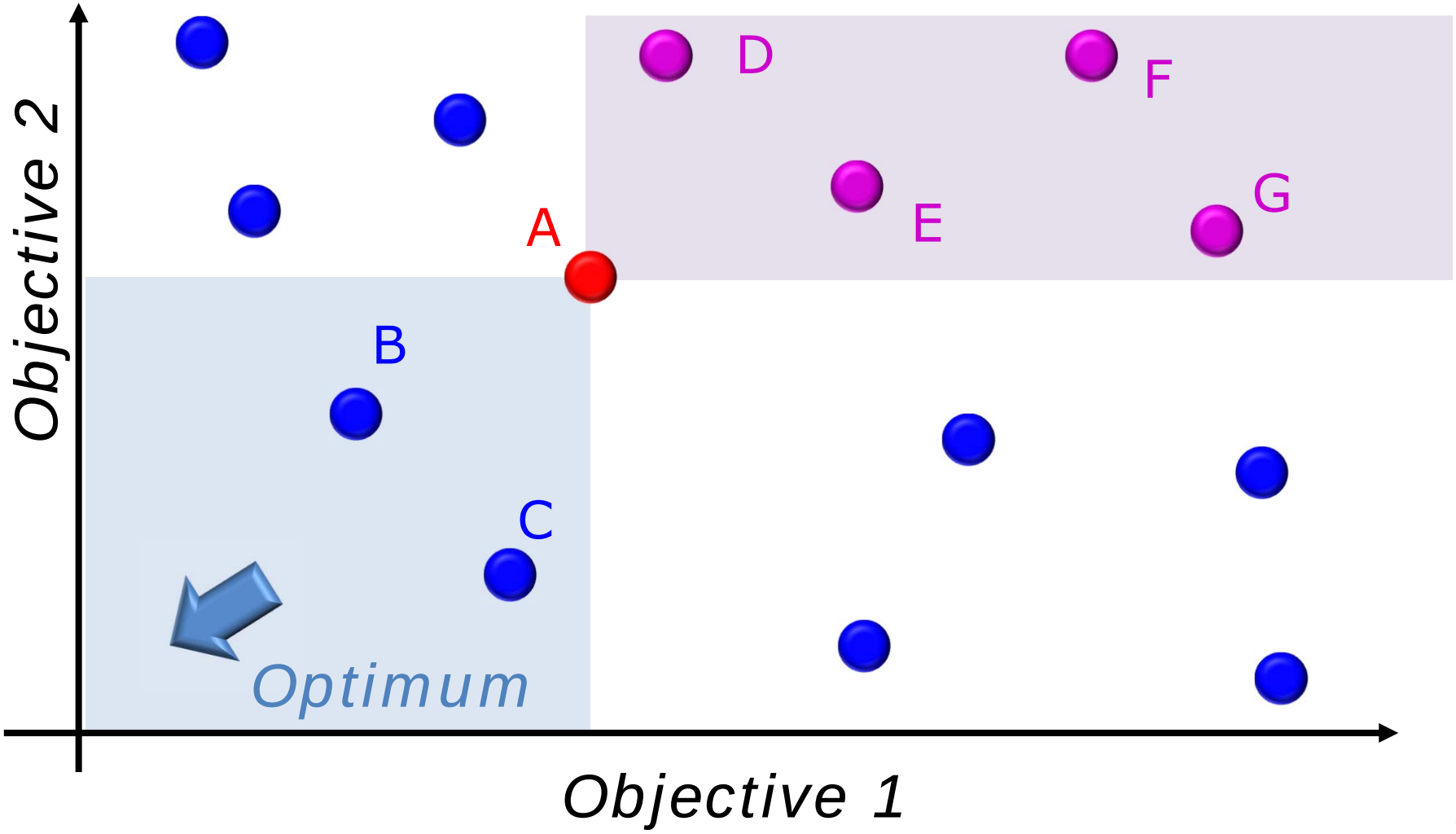
- If there is NO solution that satisfies the above relation, it is said that "$x_\beta$ is NOT dominated" or "$x_\beta$ is a non-dominated solution".
- Set of non-dominated solutions are Pareto-optimal solutions

# 非劣解・劣解の概念の導入 (Goldberg, 1989)

Introduction of concept of dominance

A is dominated by B and C

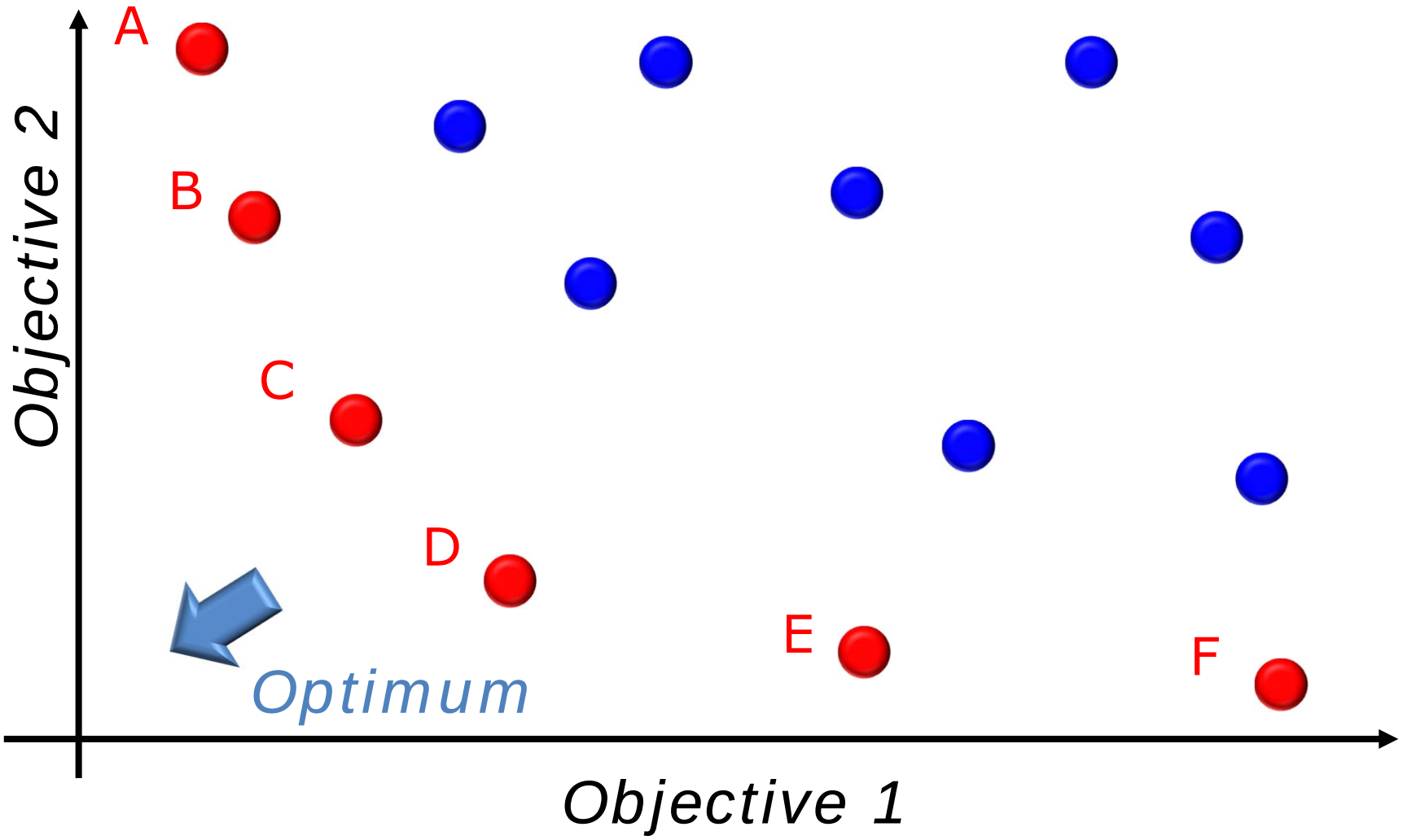
A dominates D, E, F, and G



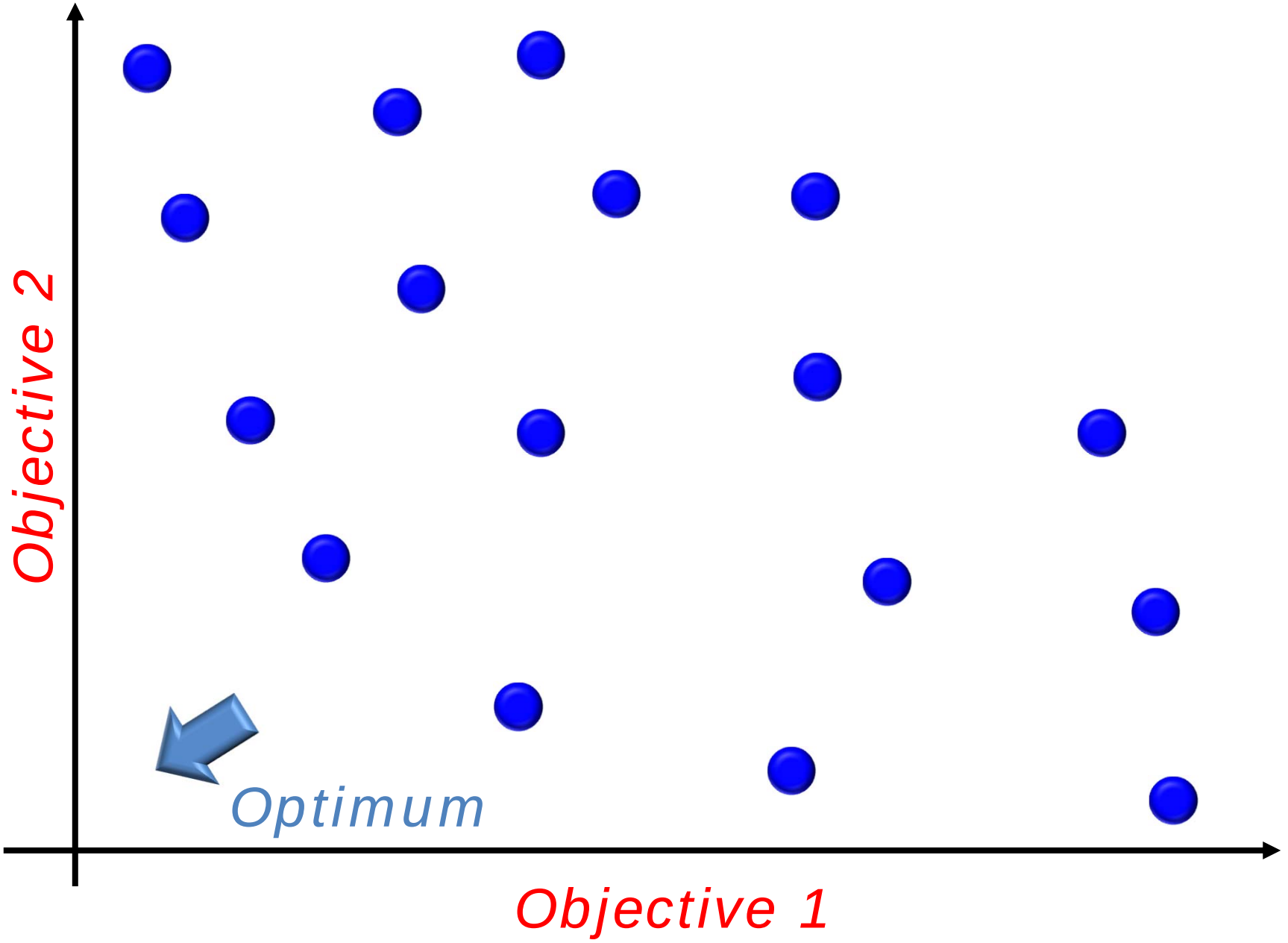
# 非劣解・劣解の概念の導入 (Goldberg, 1989)

Introduction of concept of dominance

A, B, …, F are non-dominated solutions

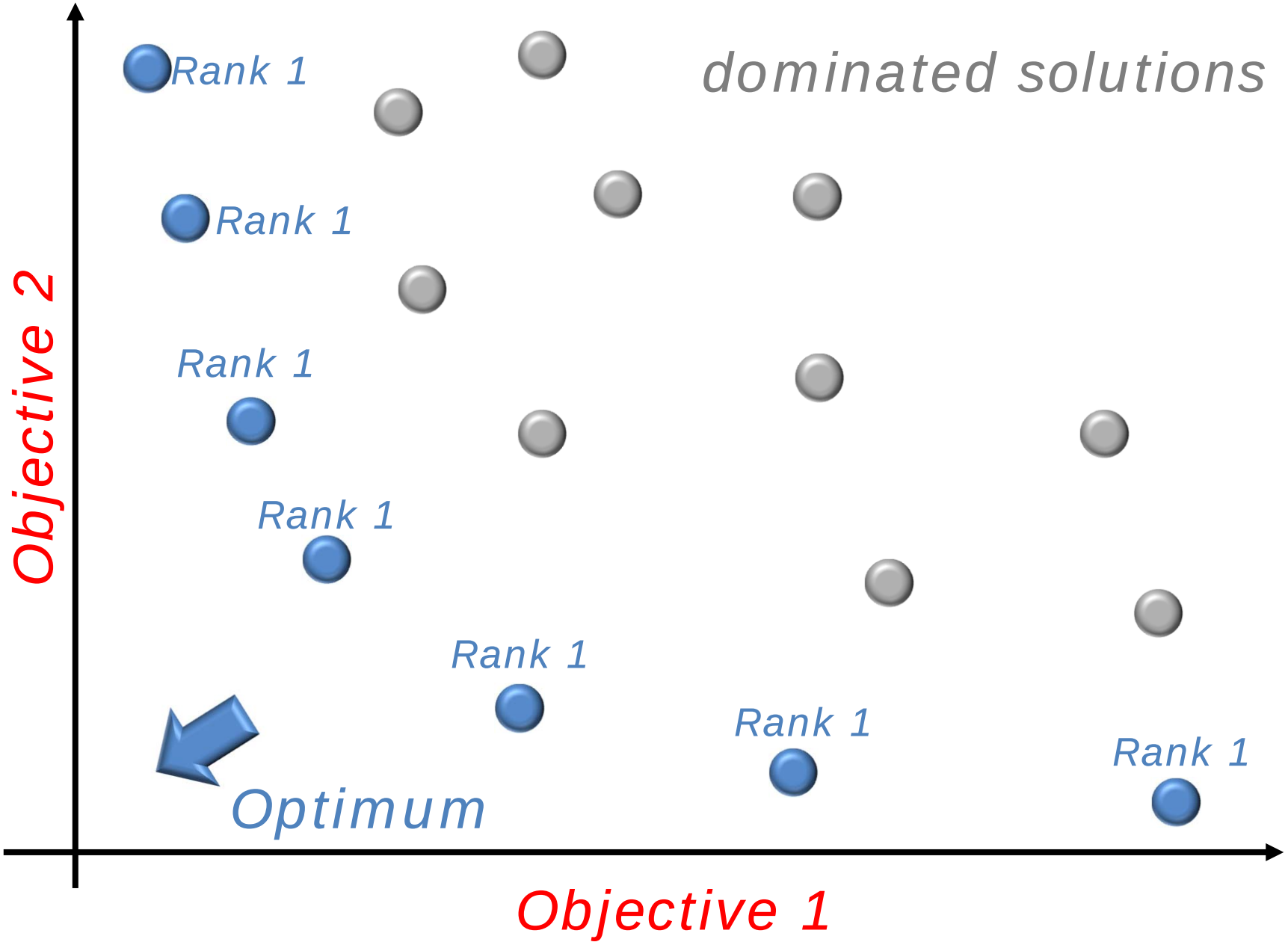


# どうやって適応度を決めるか？ How to define fitness?



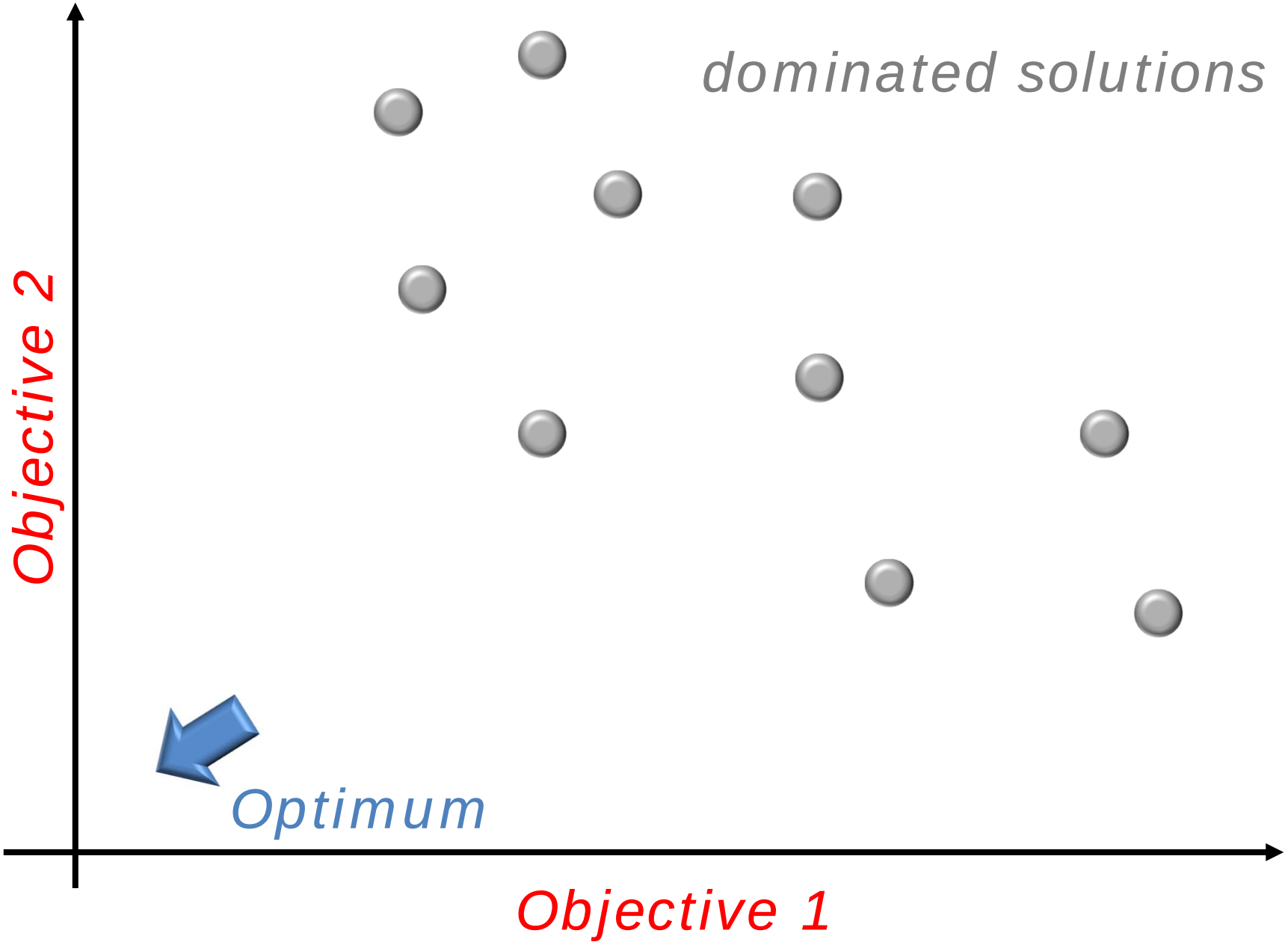
# 非劣解・劣解の概念の導入 (Goldberg, 1989)

Introduction of concept of dominance



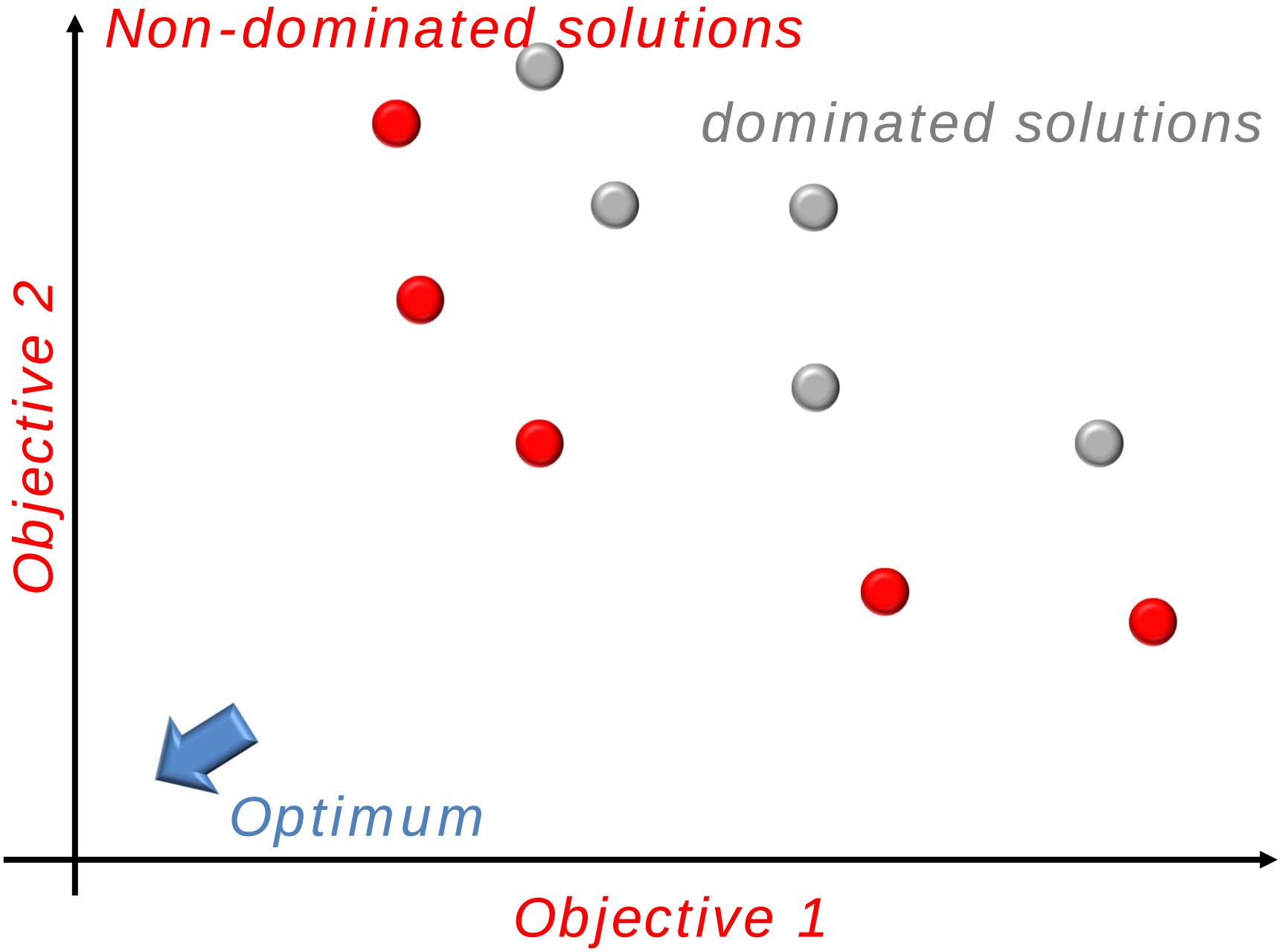
# 非劣解・劣解の概念の導入 (Goldberg, 1989)

Introduction of concept of dominance



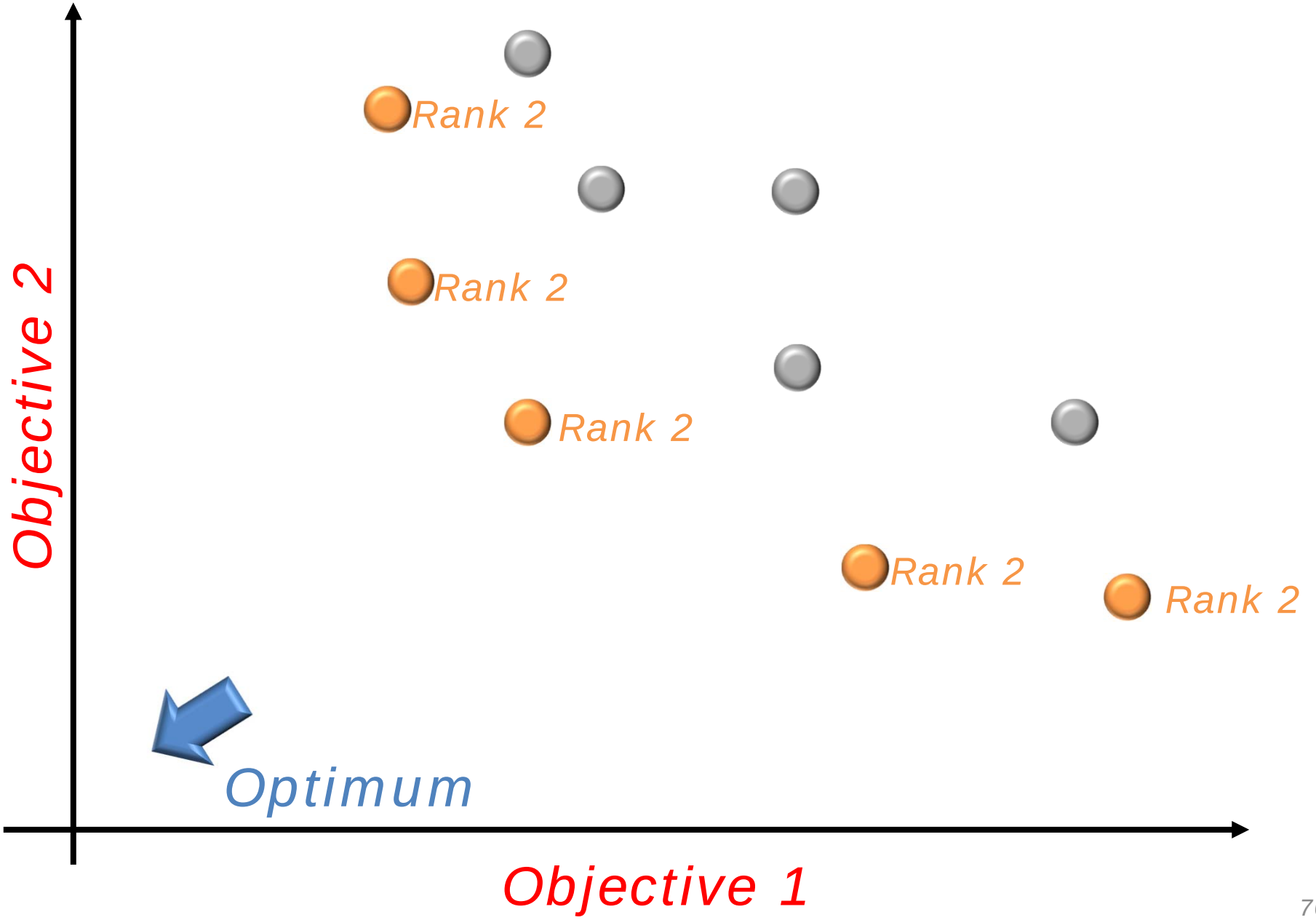
# 非劣解・劣解の概念の導入 (Goldberg, 1989)

Introduction of concept of dominance



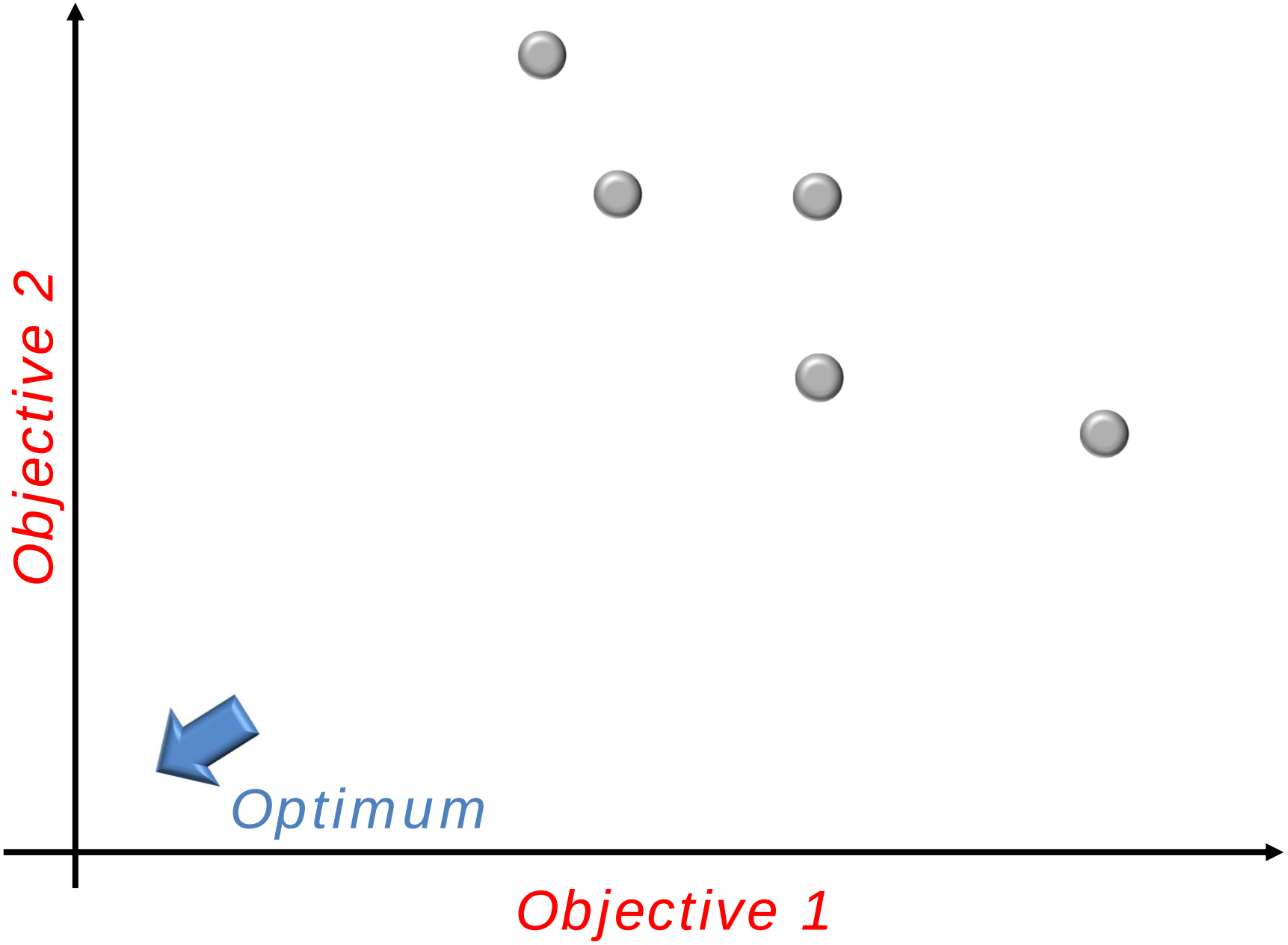
# 非劣解・劣解の概念の導入 (Goldberg, 1989)

Introduction of concept of dominance



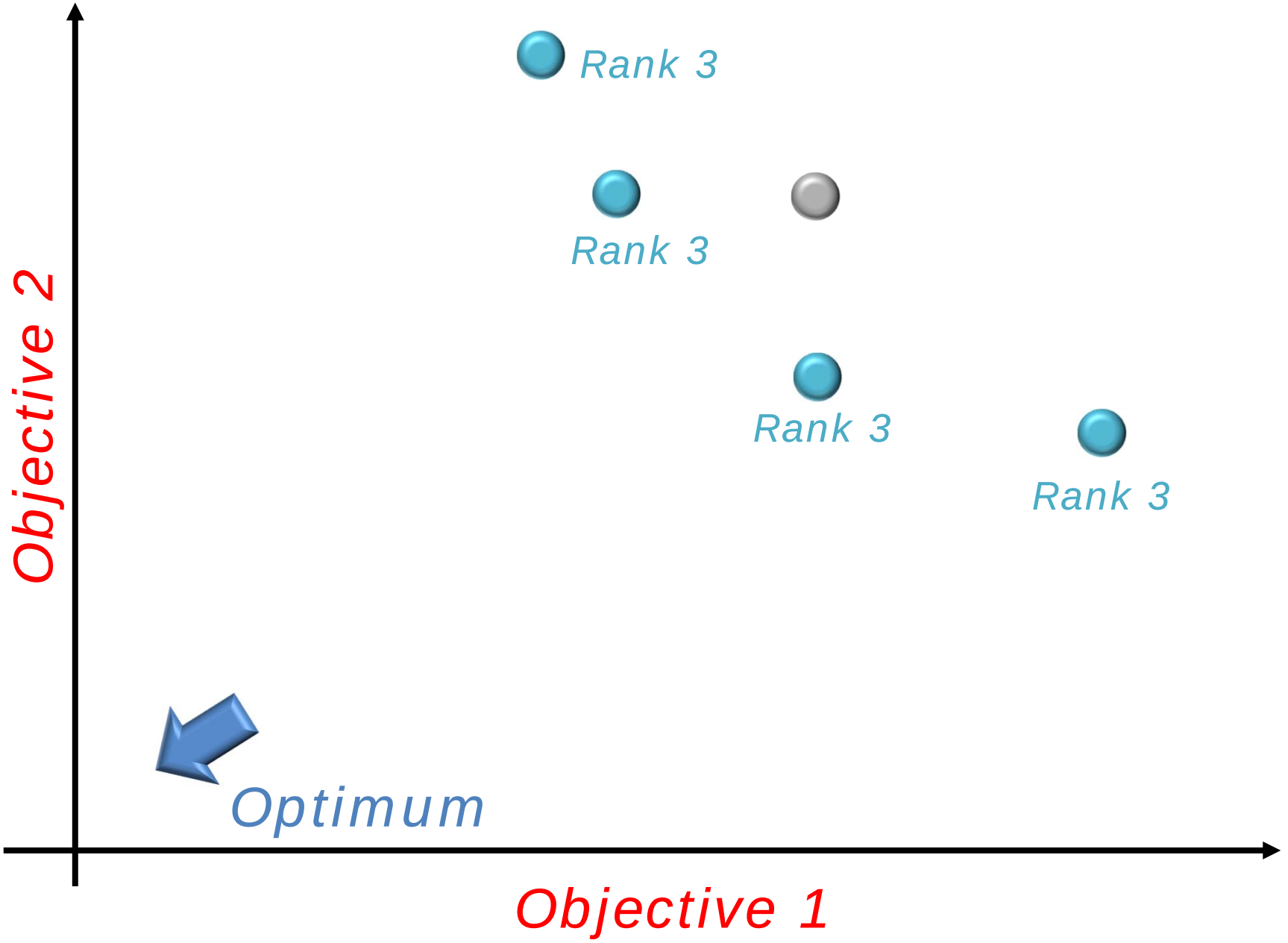
# 非劣解・劣解の概念の導入 (Goldberg, 1989)

Introduction of concept of dominance



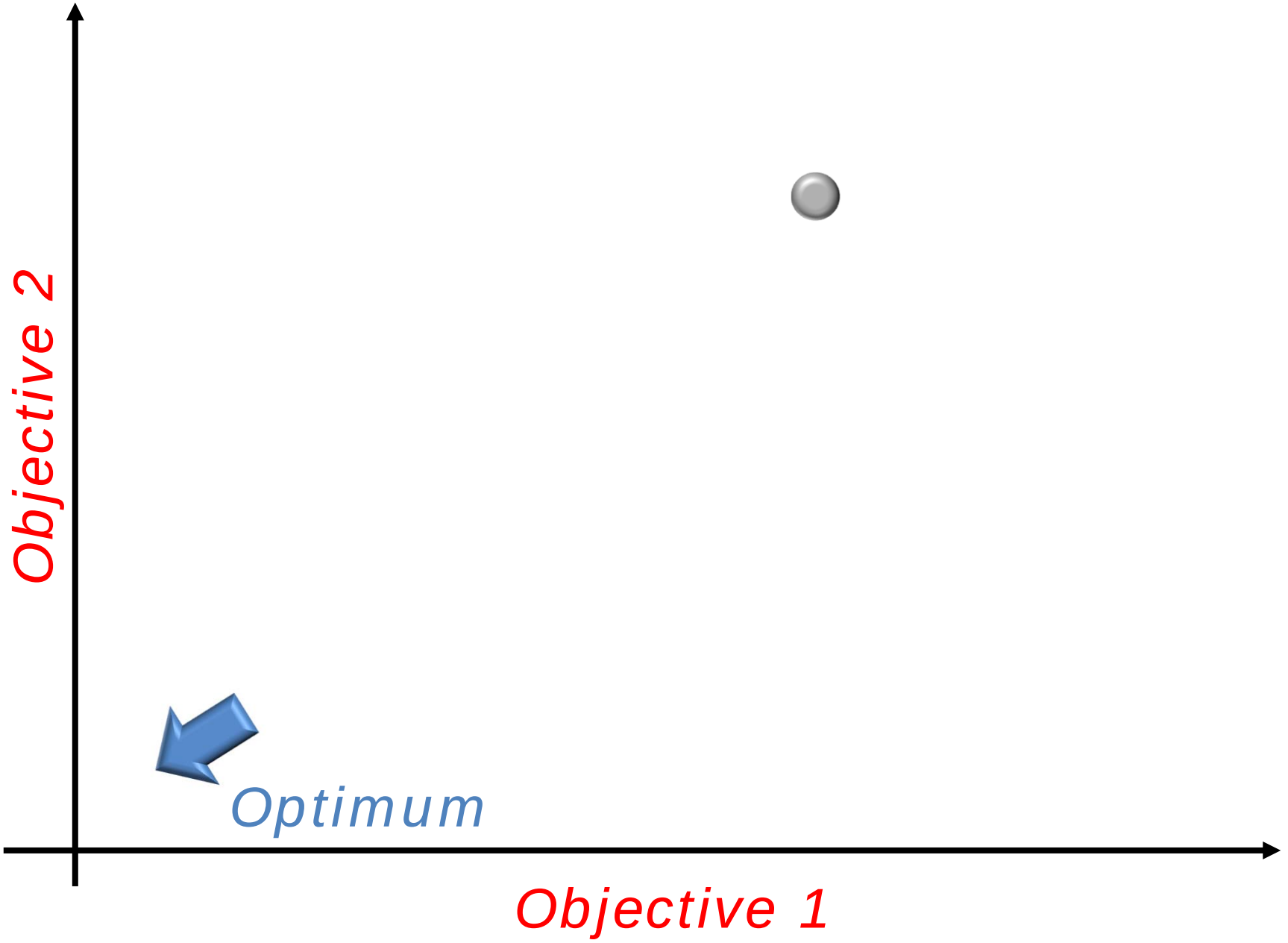
# 非劣解・劣解の概念の導入 (Goldberg, 1989)

Introduction of concept of dominance



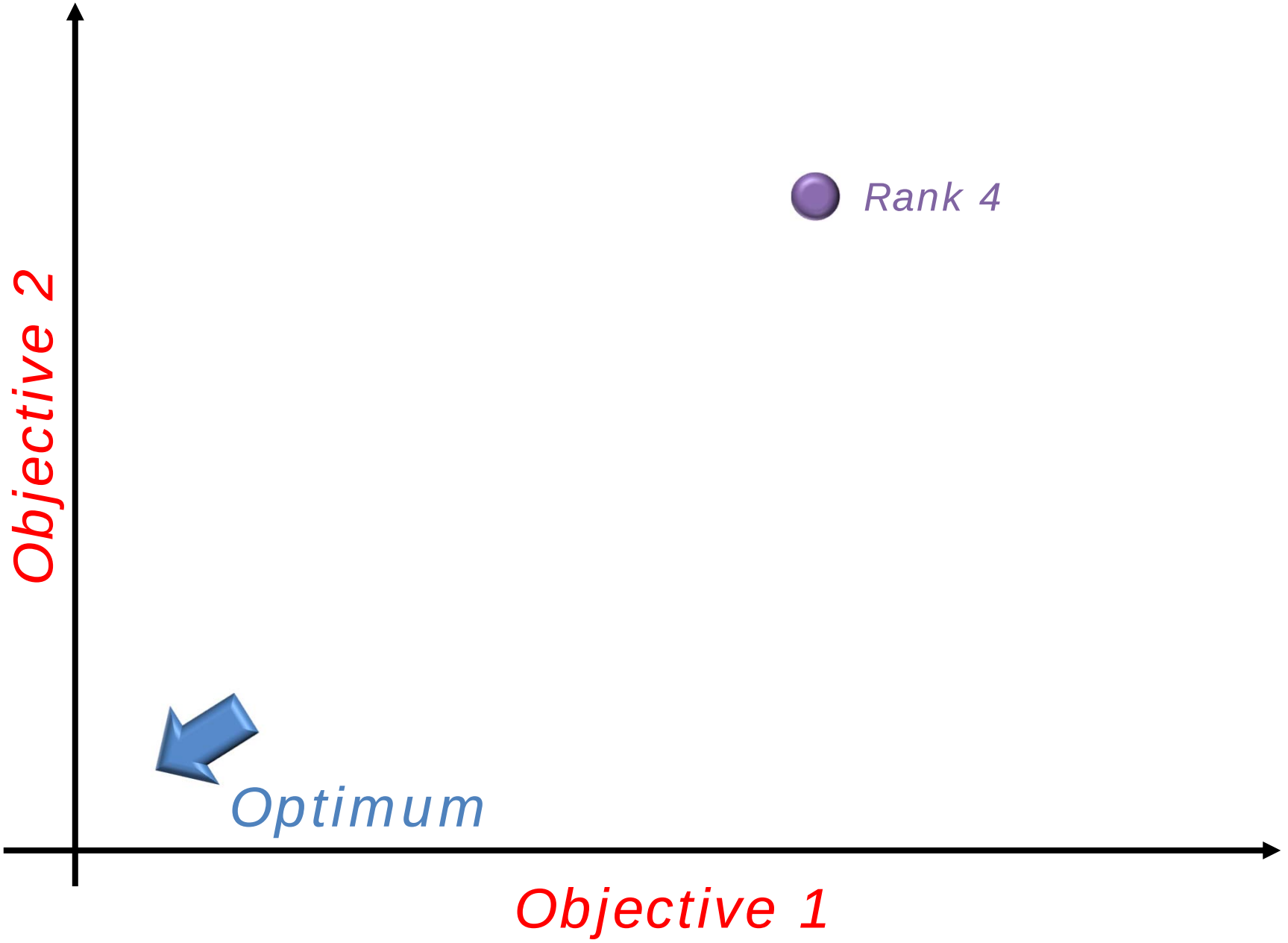
# 非劣解・劣解の概念の導入 (Goldberg, 1989)

Introduction of concept of dominance



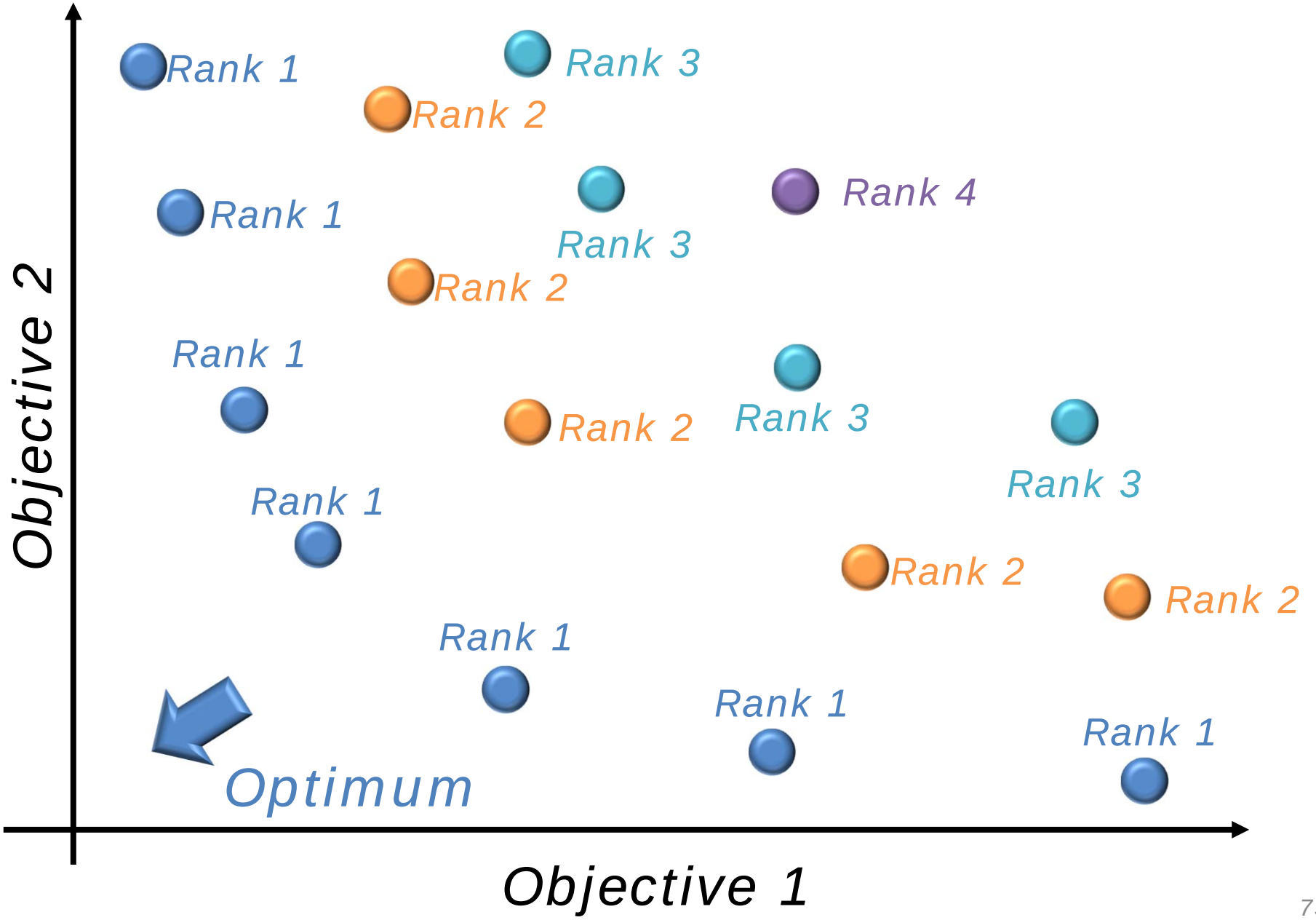
# 非劣解・劣解の概念の導入 (Goldberg, 1989)

Introduction of concept of dominance



# 非劣解・劣解の概念の導入 (Goldberg, 1989)

Introduction of concept of dominance

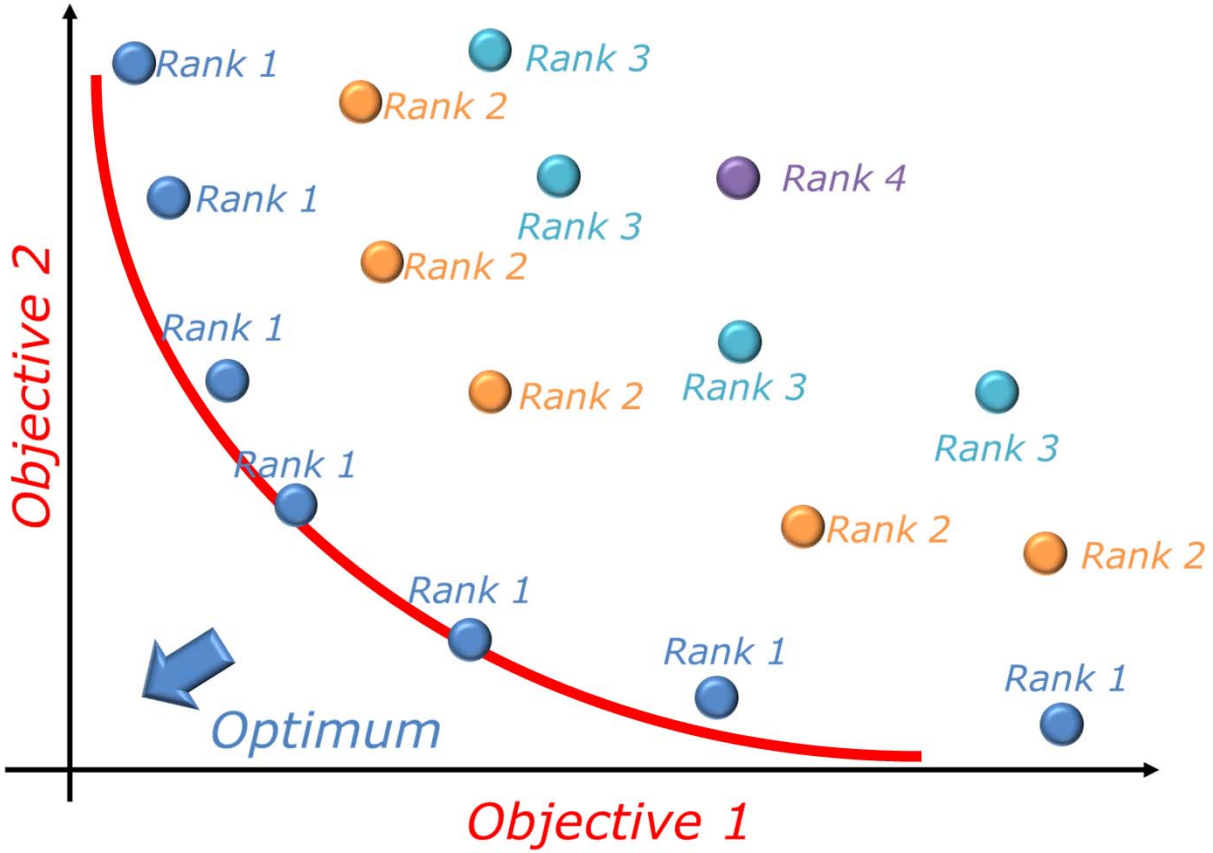


# 非劣解・劣解の概念の導入 (Goldberg, 1989)

Introduction of concept of dominance

## パレート面に近い解によいランクを与える

Give good rank to solutions close to the Pareto-front

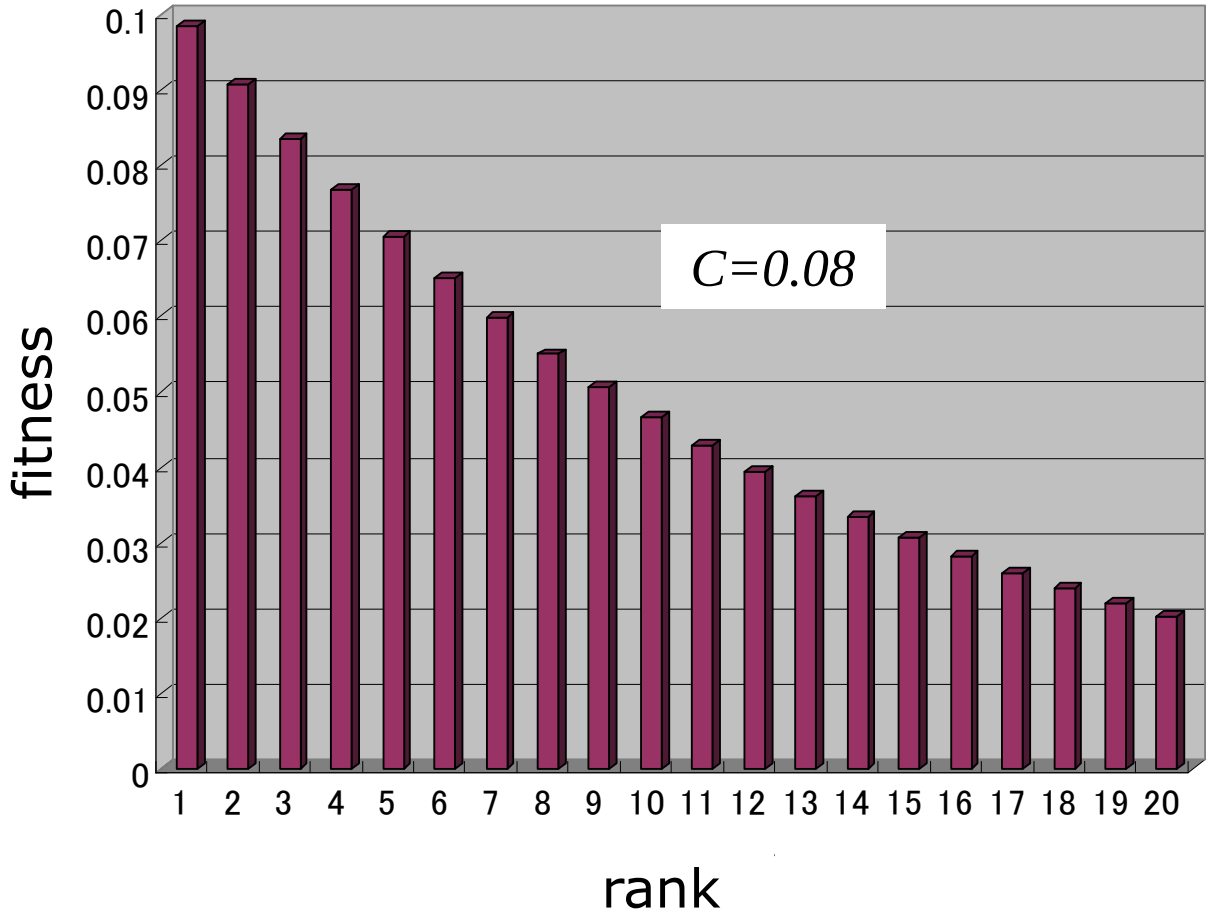


# 適応度の評価　evaluation of fitness

Nonlinear function of Michalewicz

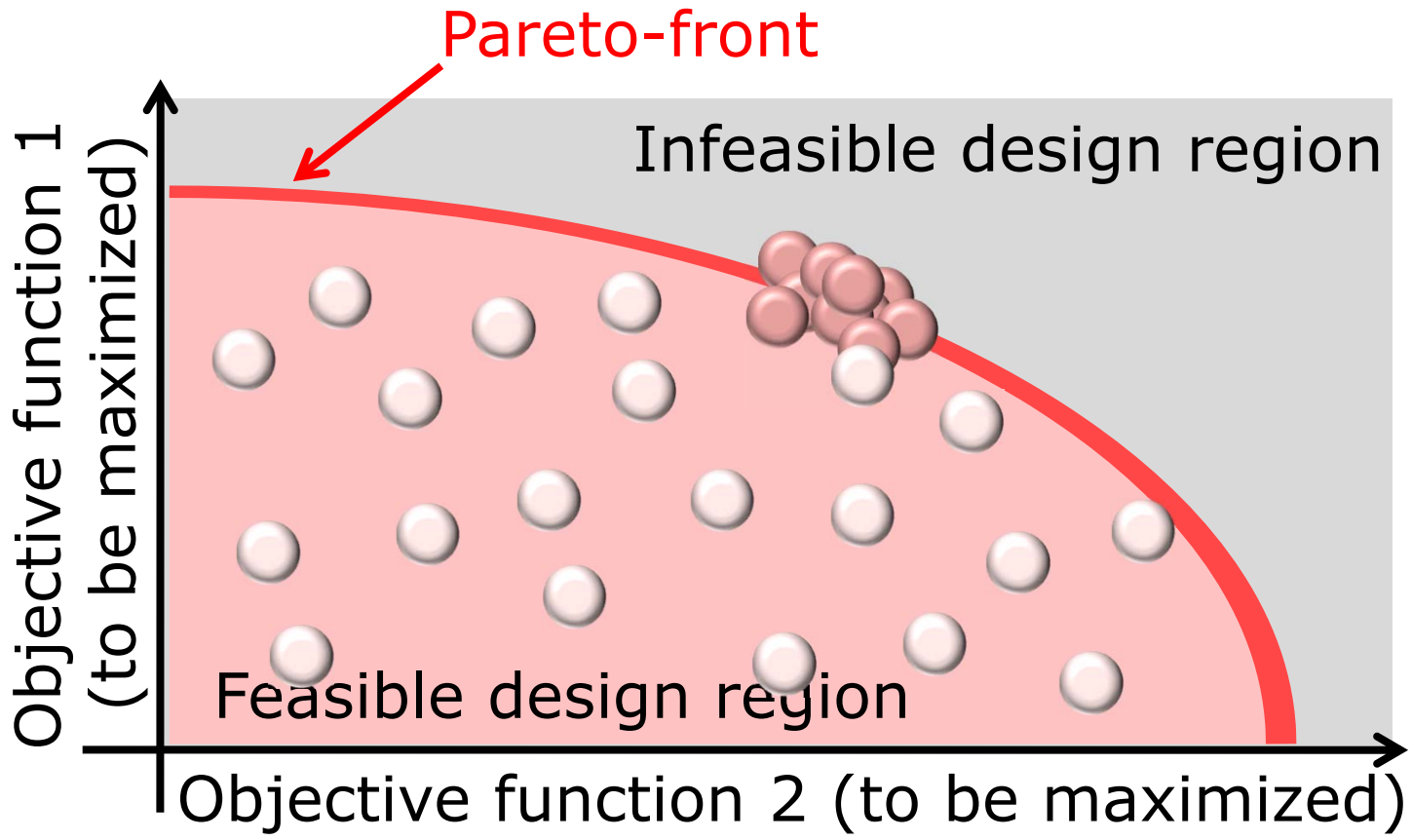
$$Fitness = c \cdot (1-c)^{rank-1}$$

ランクの高い個体にほどよい適応度（より高い生存率）を与える
Give reasonable fitness to each design according to their rank



# パレートランキングの問題点 drawback of Pareto-ranking

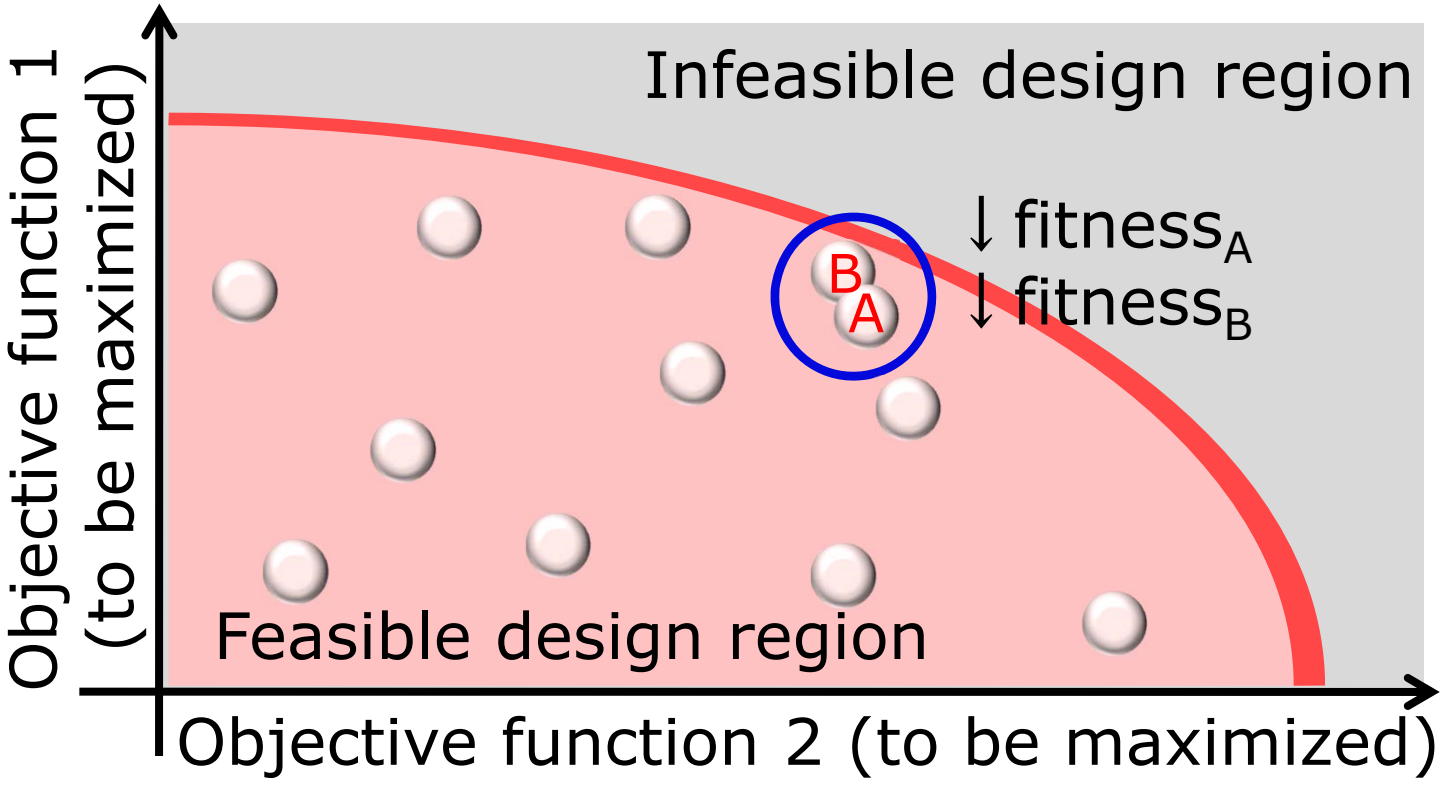
一カ所に集中してしまう(たいがい集まる)
Population tends to concentrate



# ニッチ戦略　Niching

- 近くに別の個体がある個体は適応度がさがるようにする

  When a solution is close to another solution, a penalty is given to its fitness.



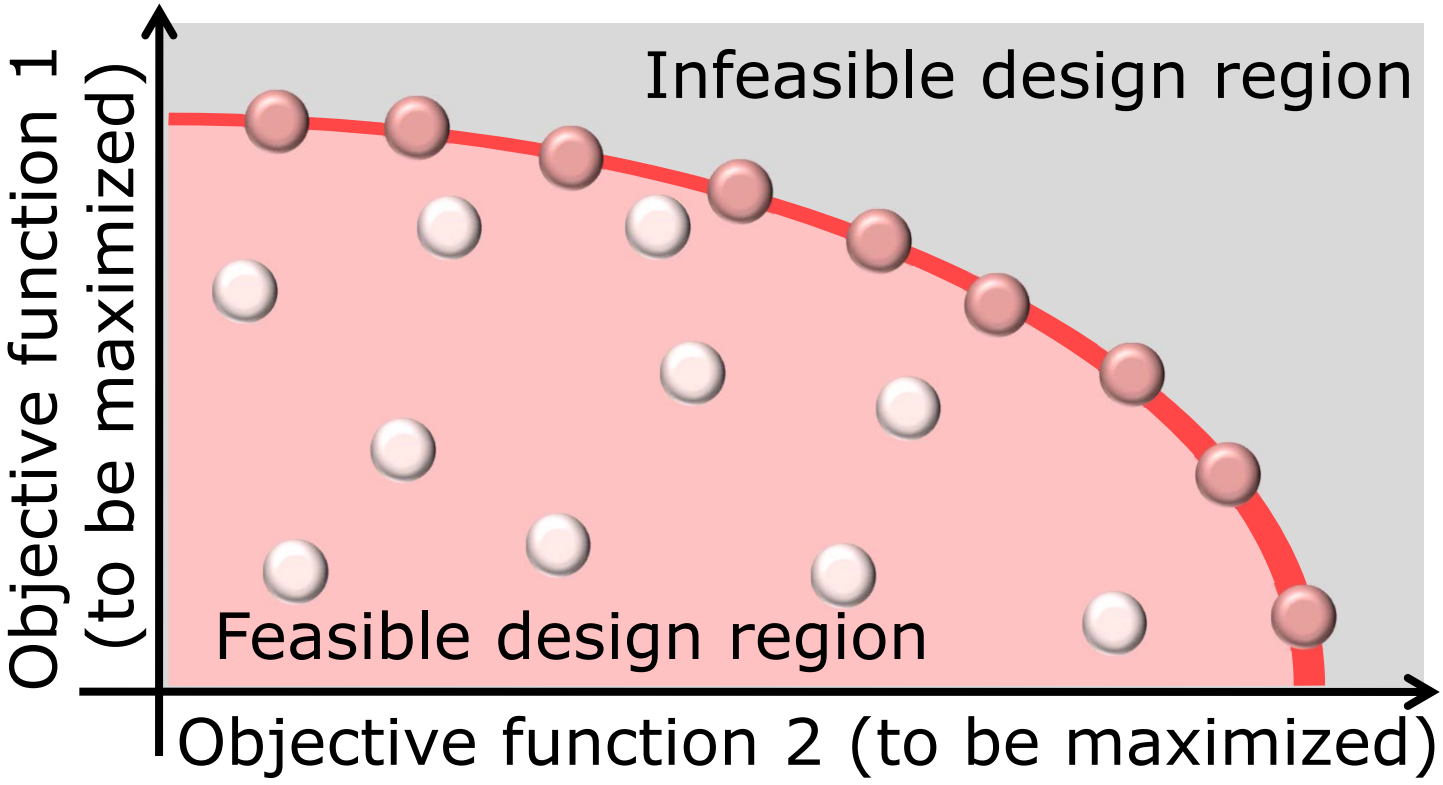
# ニッチ戦略　Niching

- 近くに別の個体がある個体は適応度がさがるようにする

  When a solution is close to another solution, a penalty is given to its fitness.

- 均一にパレート面をとらえることができる

  Non-dominated solutions uniformly distributes to capture entire Pareto-front



# NSGA-II

*Kalyanmoy Deb, et al., "A Fast and Elitist Multiobjective Genetic Algorithm: NSGA-II," IEEE TRANSACTIONS ON EVOLUTIONARY COMPUTATION, VOL. 6, NO. 2, APRIL 2002*

- オープンソースコードとして公開

  https://www.iitk.ac.in/kangal/index.shtml

- 世界で最も普及している多目的最適化手法

- 目的関数が2つあるいは3つの最適化問題に対しては現在でも最も効率的な最適化手法の1つ

# NSGA－II

Non-dominated Sorting Genetic Algorithms -II

# NSGA－IIとは？ What is NSGA-II?

- ウェブサイトからソースコード(C++)をダウンロード可能

  The source code (C/C++) is available on the web

  http://www.iitk.ac.in/kangal/codes.shtml



Kanpur Genetic Algorithms Laboratory

Search KanGAL web-site with Google [Search]

Home | About KanGAL | Announcements and News | Prof. Deb's Homepage | Prof. Deb's Awards | People | Publications | Source Codes | Resources | Projects | KanGAL Books | Bioinformatics at KanGAL | Computing Facilities | Other GA Links | KanGAL in Pictures

>> Home > Source Codes  Back | Forward

**Software Developed at KanGAL**

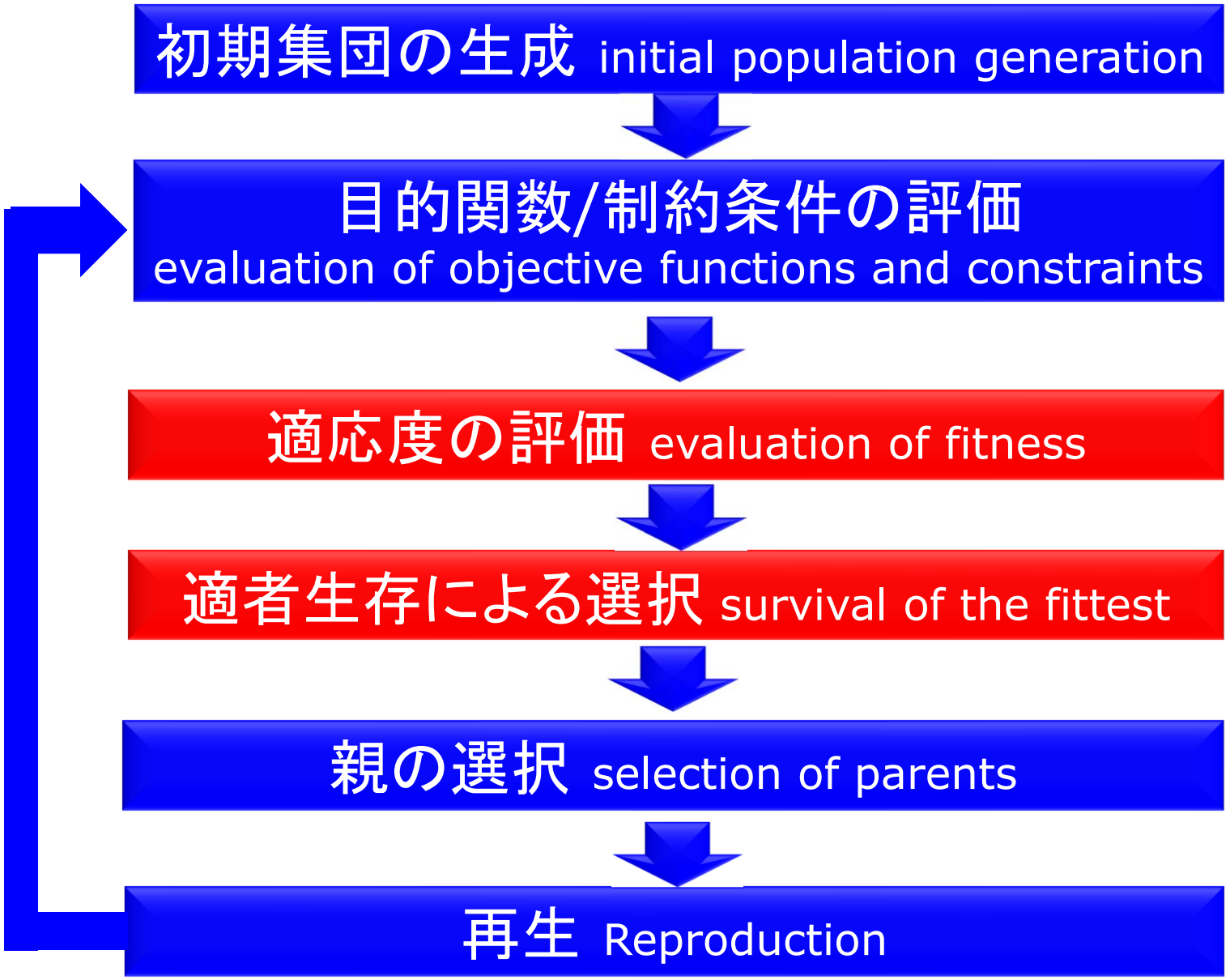
- Simple GA code in C (for Windows and Linux)
  - Binary GA code: SGA in C (and input file)
- G3PCX code in C
  - G3PCX in C
- Single-objective GA code in C (for Windows and Linux)
  - GA in C (Real + Binary + Constraint Handling)
- Multi-objective NSGA code in C (for Windows and Linux)
  - NSGA in C
- Multi-objective NSGA-II code in C
  - Original Implementation (for Windows and Linux): NSGA-II in C (Real + Binary + Constraint Handling)
  - New (10 April 2005) (for Linux only): NSGA-II in C (Real + Binary + Constraint Handling)
  - Revision 1.1 (10 May 2005) (for Linux only): NSGA-II in C (Real + Binary + Constraint Handling) NEW
  - Revision 1.1 (10 June 2005) (for Linux only): NSGA-II in C with gnuplot (Real + Binary + Constraint Handling) NEW
  - Revision 1.1.5 (15 May 2010) (for Linux only- 64-bit bug fixed): NSGA-II in C with gnuplot (Real + Binary + Constraint Handling) NEW
  - Revision 1.1.6 (08 July 2011) (for Linux only- 64-bit bug for binary coding fixed): NSGA-II in C with gnuplot (Real + Binary + Constraint Handling) NEW
- Epsilon-MOEA in C and C++
  - Original: Epsilon-MOEA in C++ (Real + Constraint Handling)
  - New (10 May 2005) (for Linux only): Epsilon-MOEA in C (Real + Binary + Constraint Handling) NEW
- Basic Differential Evolution (DE) in C (Developed by Saku Kukkonen, Lappeenranta University of Technology, Finland)
  - Basic DE in C (Unix/Linux compatible) (June 15, 2005)
  - Basic DE in C (Standard C for Unix/linux/Windows or any other platform) (April 10, 2007) NEW
- Omni-Optimizer (handles single and multiple objectives, unimodal or multi-modal problems) (Published in EJOR (2008) by Kalyanmoy Deb and Santosh Tiwari)
  - Omni-Optimizer code (Unix/Linux compatible) (May, 14, 2011)
- AMGA2 (handles multi-objective optimization problems) (Accepted in Engineering Optimization Journal by Santosh Tiwari, Kalyanmoy Deb and Georges Fadel)
  - AMGA2 code (Unix/Linux compatible) (May, 14, 2011)

Sitemap | About this Site | IITK Home | ME Home | Contact Us

© 2005 Kanpur Genetic Algorithms Laboratory. All rights reserved.

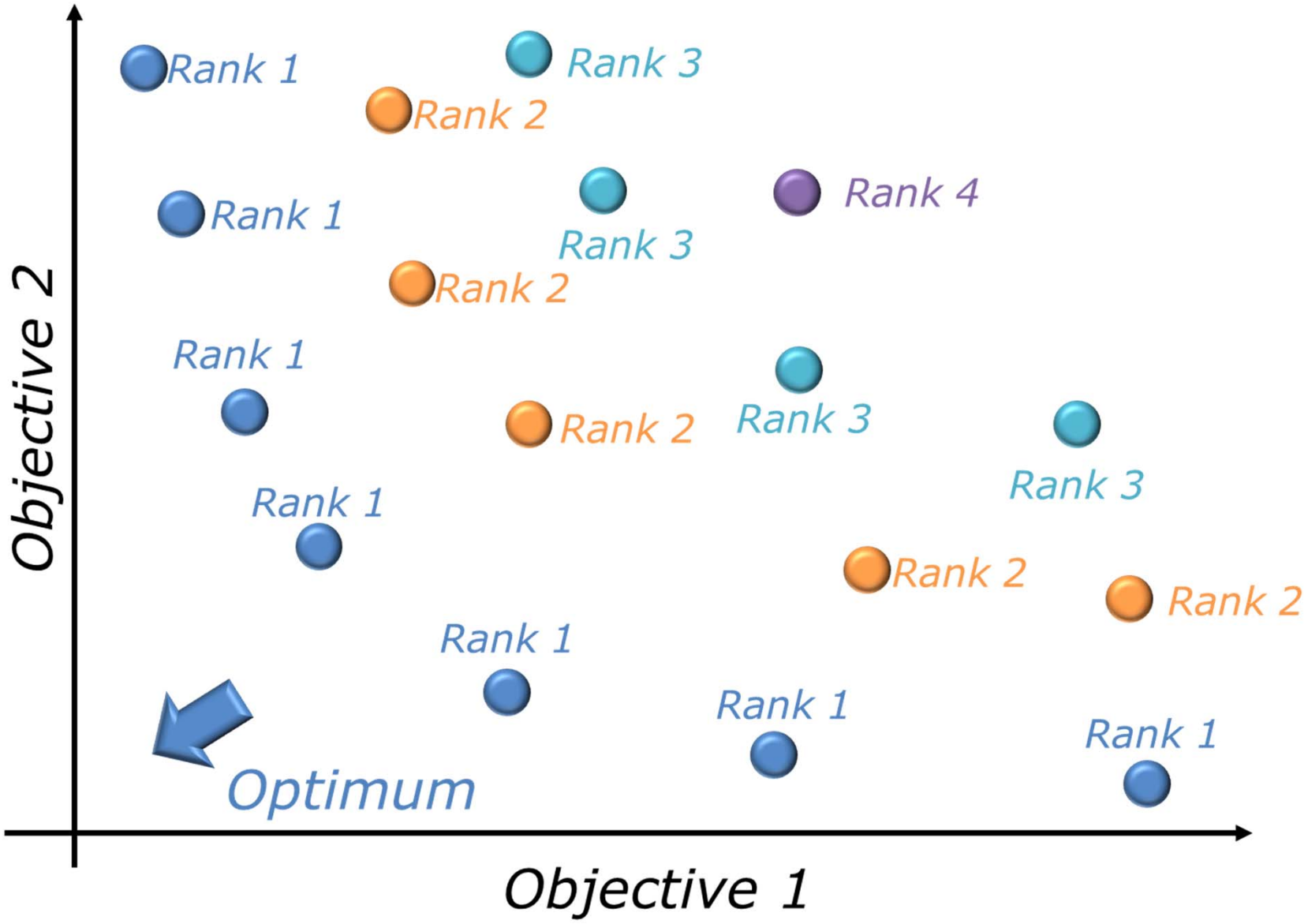
Today is January 8, 2014. Site last updated on Jan 17, 2008.

Number of visitors since July 15, 2005: [an error occurred while processing this directive]


初期集団の生成 initial population generation
目的関数/制約条件の評価
evaluation of objective functions and constraints
適応度の評価 evaluation of fitness
適者生存による選択 survival of the fittest
親の選択 selection of parents
再生 Reproduction

違いはこの
2つ！

# 適応度の評価 evaluation of fitness



# 適応度の評価 evaluation of fitness

Fast nondominated sorting

(1) Set i=1

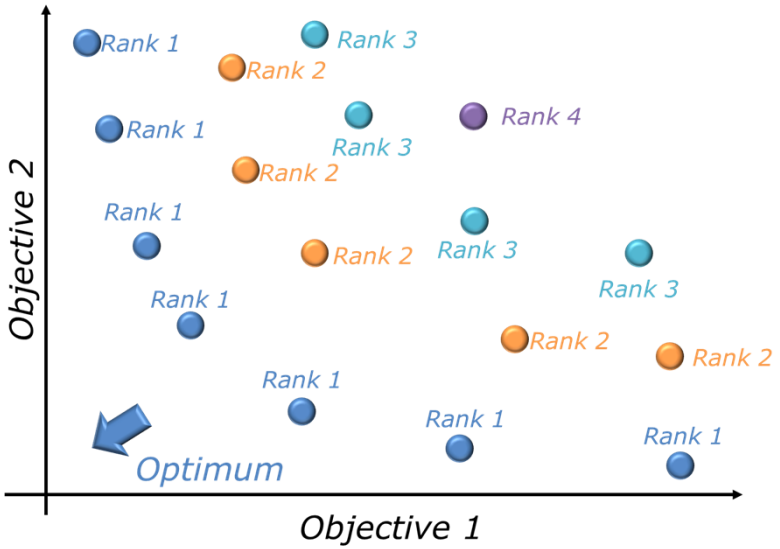
(2) For each individual, count dominate solutions and dominated solutions

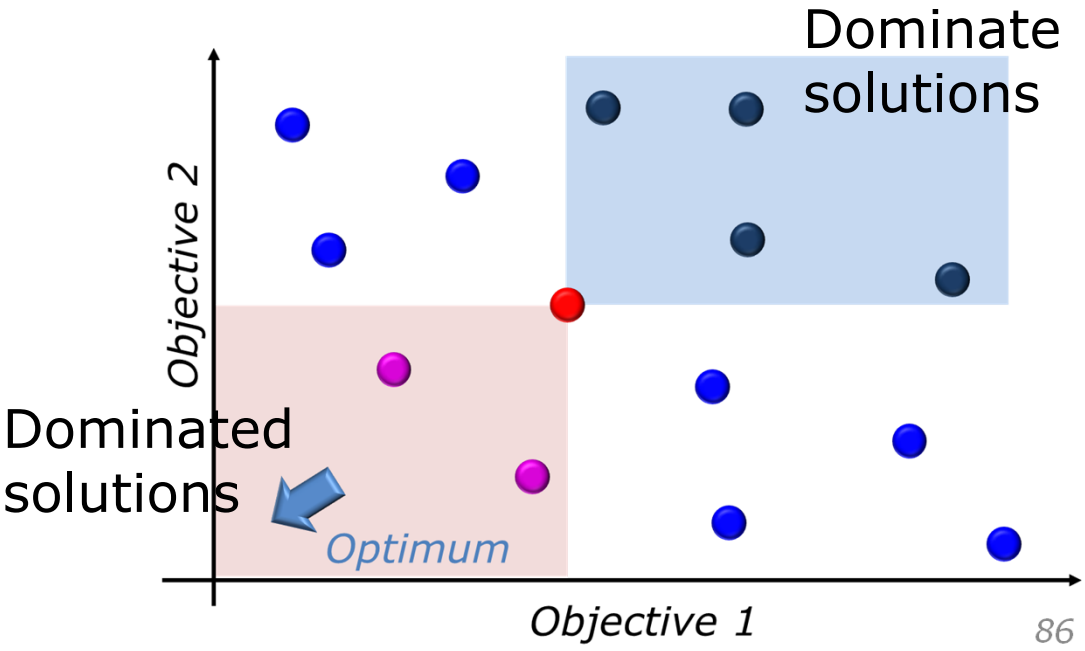
(3) Solutions that have no dominated solutions are rank i

(4) For dominate solutions of rank i solutions, subtract one from their number of dominated solutions

(5) remove rank I from the list

(6) i= i+1 and go to (3)



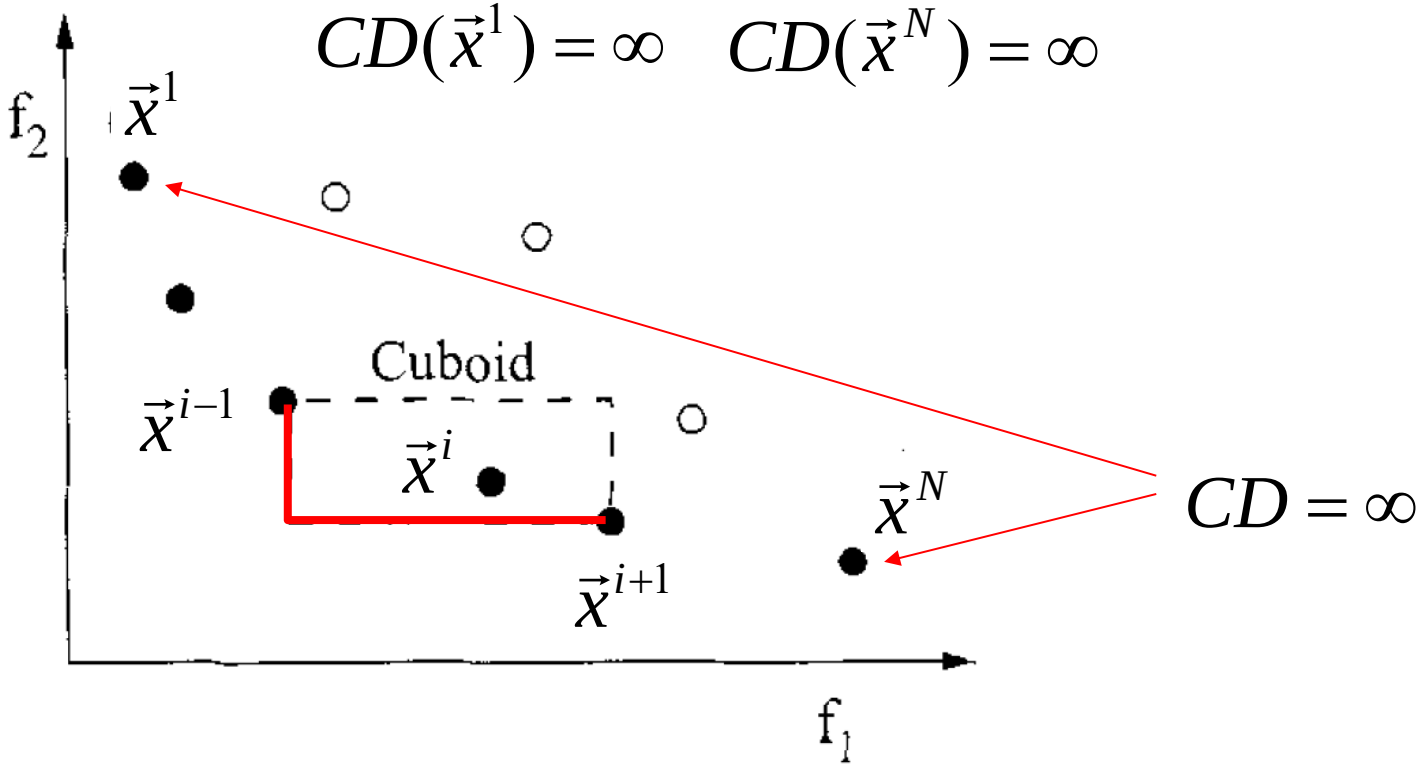
# 適応度の評価 evaluation of fitness

Crowding distance

- It has no parameter to be tuned.

$$CD(\vec{x}^i) = \frac{1}{M}\sum_{j=1}^{M}\left|\frac{f_j(\vec{x}^{i+1}) - f_j(\vec{x}^{i-1})}{f_j(\vec{x}^1) - f_j(\vec{x}^N)}\right|$$

$$CD(\vec{x}^1) = \infty \quad CD(\vec{x}^N) = \infty$$



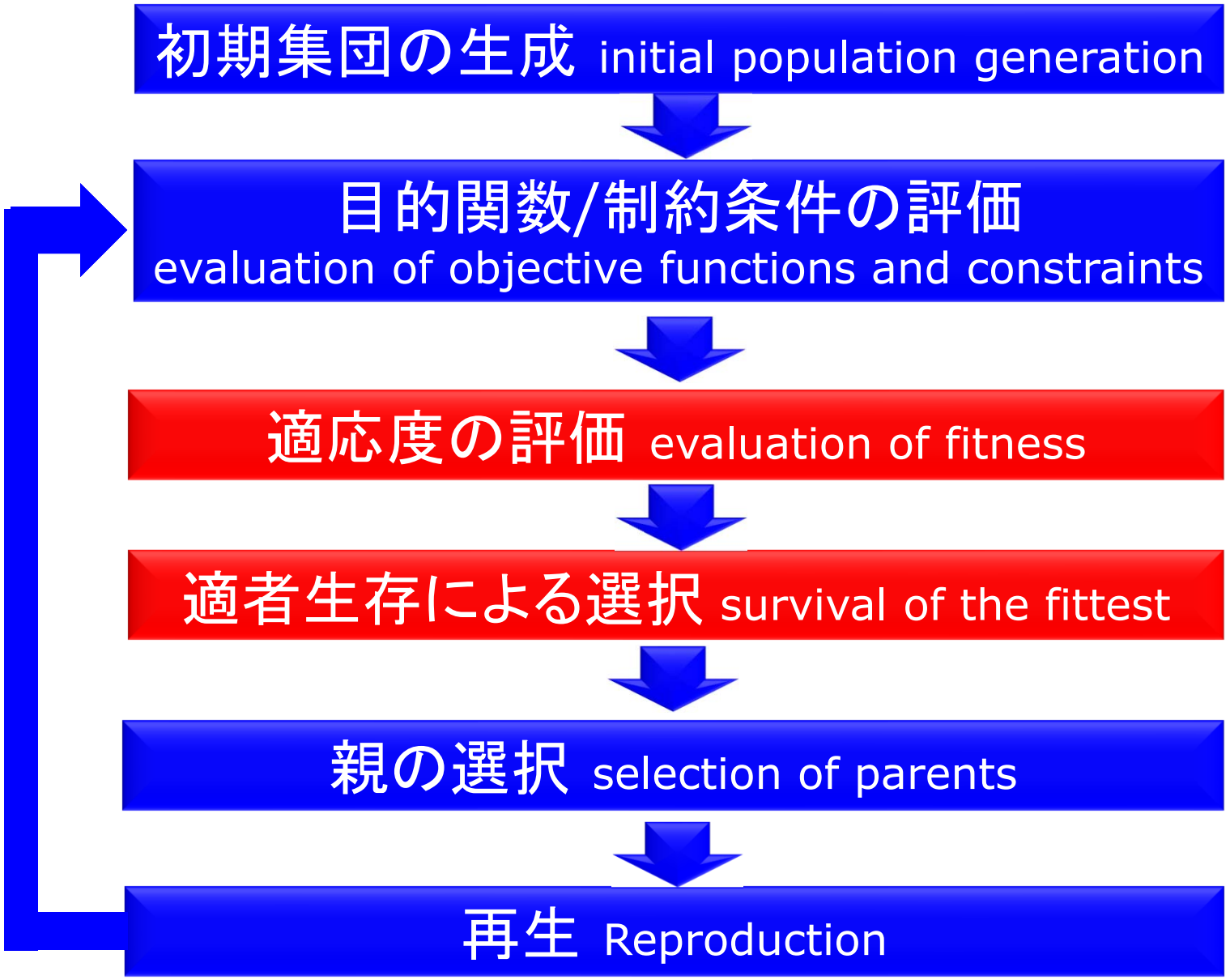

*Kalyanmoy Deb, et al., 2002より*

# 適応度の評価 evaluation of fitness

1. フロント$F_k$に含まれる個体の混雑距離を計算する
   1.1 フロント$F_k$に含まれる個体の数をJとする
   1.2 このフロントに含まれる各個体jに対して初期値設定を行う: $d_j = 0$
   1.3 各目的関数m = 1, 2, . . . ,M に対して,
      1.3.1 目的関数値が悪い順に個体をソートする
      1.3.2 境界個体(目的mが最大と最小の個体) に対して最大距離,
      もしくは無限距離を与える
      1.3.3 境界個体以外の全ての個体(j = 2, . . . , l - 1) に対して以下の式
      に従った混雑度計算を行う.

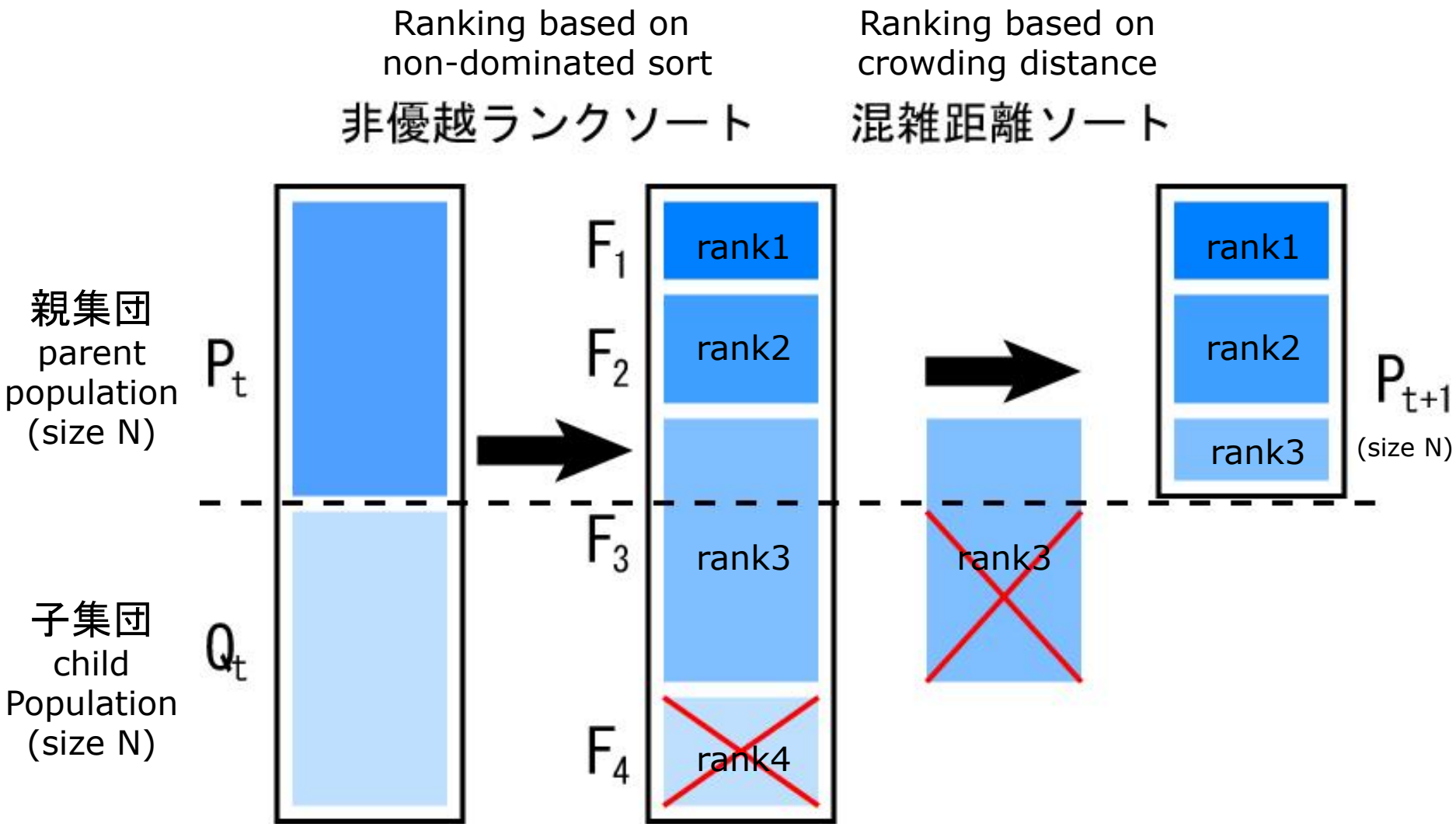
$$d_j = d_j + \frac{f_m^{j+1} - f_m^{j-1}}{f_m^{\max} - f_m^{\min}} \qquad m = 1, \ldots, M$$

2. 混雑距離の値が高い個体を次世代の親集団Pt+1に組み込む


初期集団の生成 initial population generation
目的関数/制約条件の評価
evaluation of objective functions and constraints
適応度の評価 evaluation of fitness
適者生存による選択 survival of the fittest
親の選択 selection of parents
再生 Reproduction

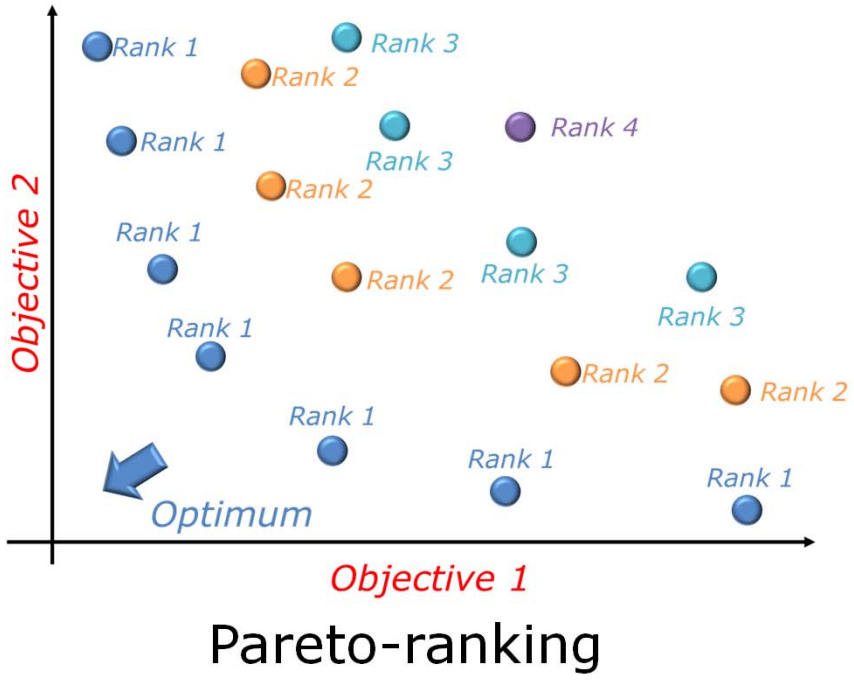
違いはこの
2つ！

# 適者生存による選択 survival of the fittest

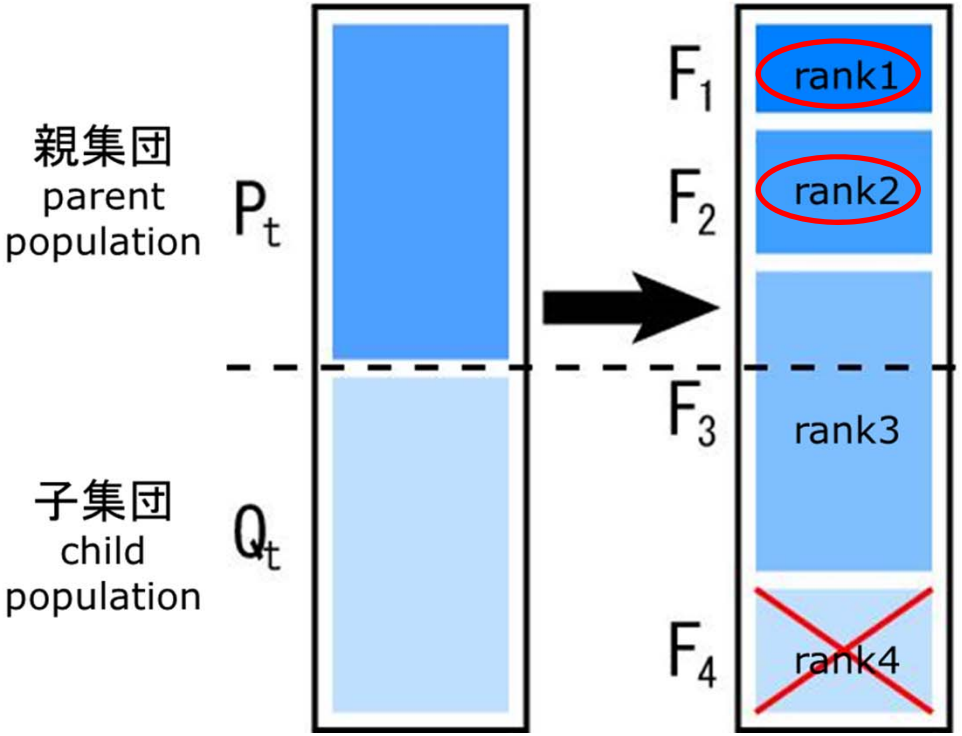


# 適者生存による選択 survival of the fittest

1. 子集団Qt（集団サイズN）と親集団Pt（集団サイズN）の和集合Rt（集団サイズ2N）を作成
2. Rtのすべての解をランクr = 1　とする.
3. 個体群Rtの中から非劣個体を求め, これらの個体をランクr とする.
4. 得られた非劣個体群を個体群Rtから除く
5. （次世代の親集団Pt+1がNを超えない範囲で）得られた非劣個体群をPt+1に加える
6. ランク r = r +1 とし，ステップ3にもどる




Pareto-ranking

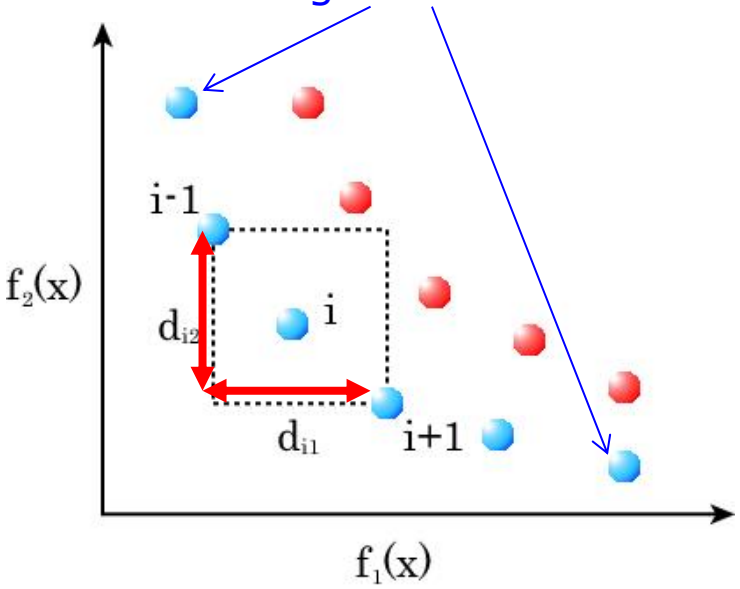


# 適者生存による選択 survival of the fittest

1. フロント$F_k$に含まれる個体の混雑距離を計算する

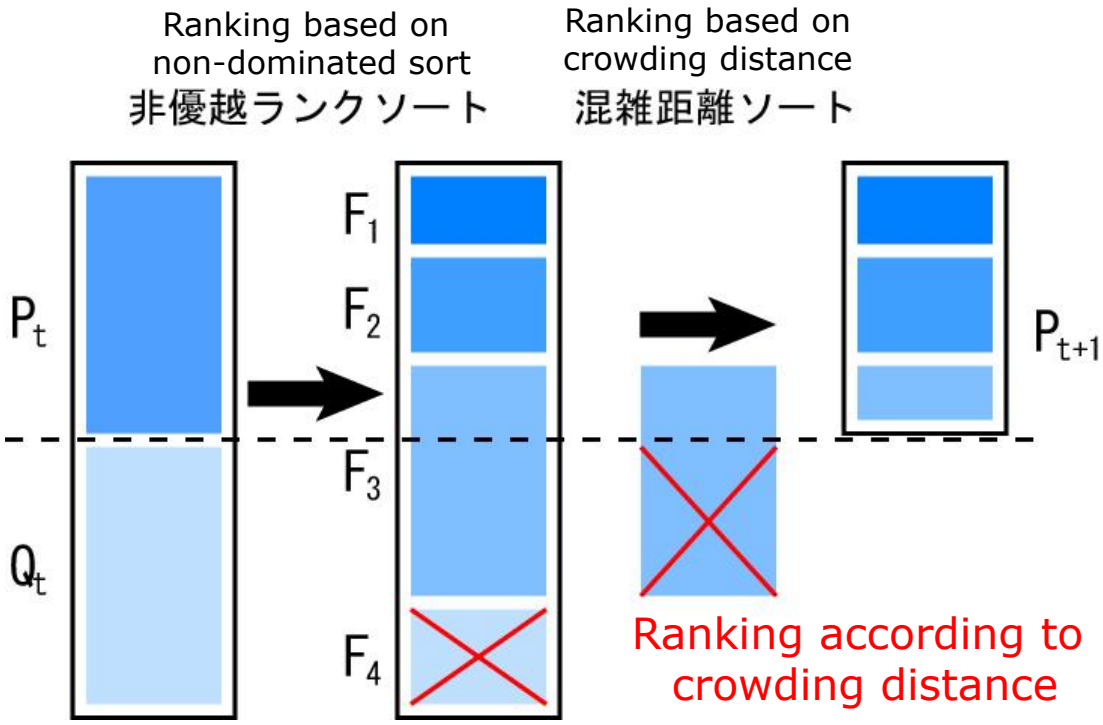
2. 混雑距離の値が高い個体を次世代の親集団Pt+1に組み込む

両端の混雑距離は∞
Crowding distance is ∞




混合距離の概念
Concept of crowding distance

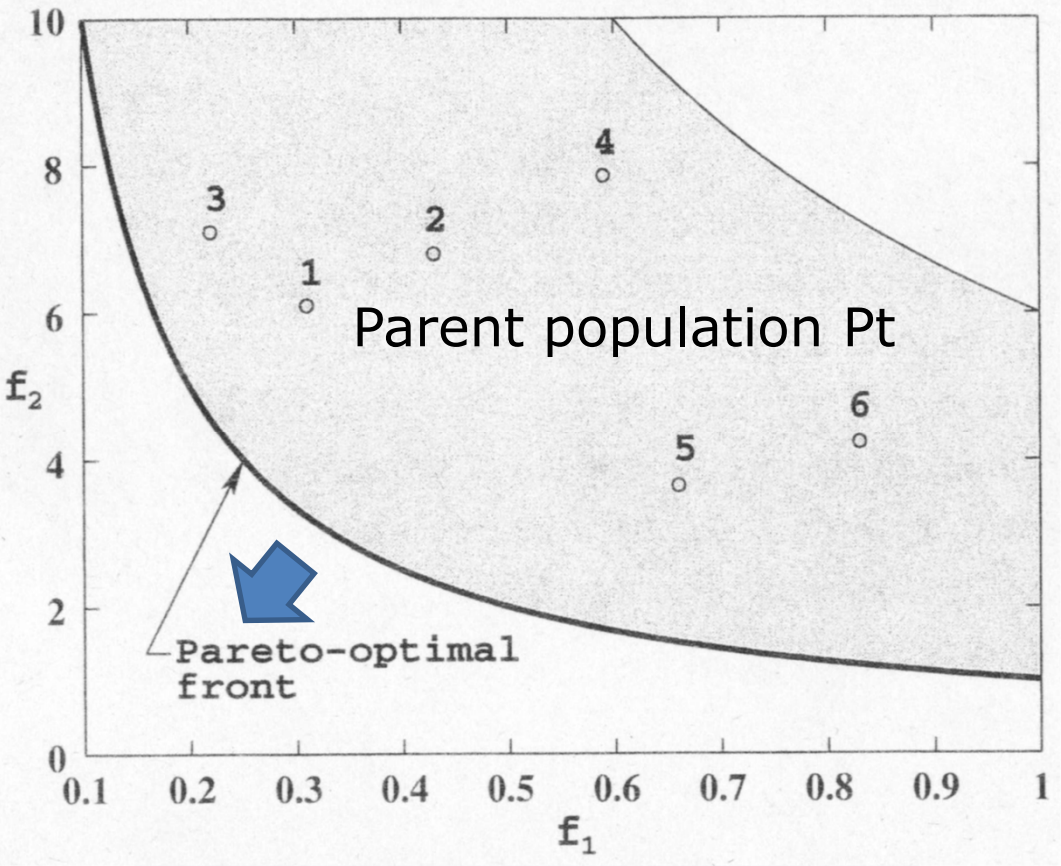
*Crowding distance* = $d_{i1}$ + $d_{i2}$



# NSGA-IIの選択オペレータを実際にやってみよう

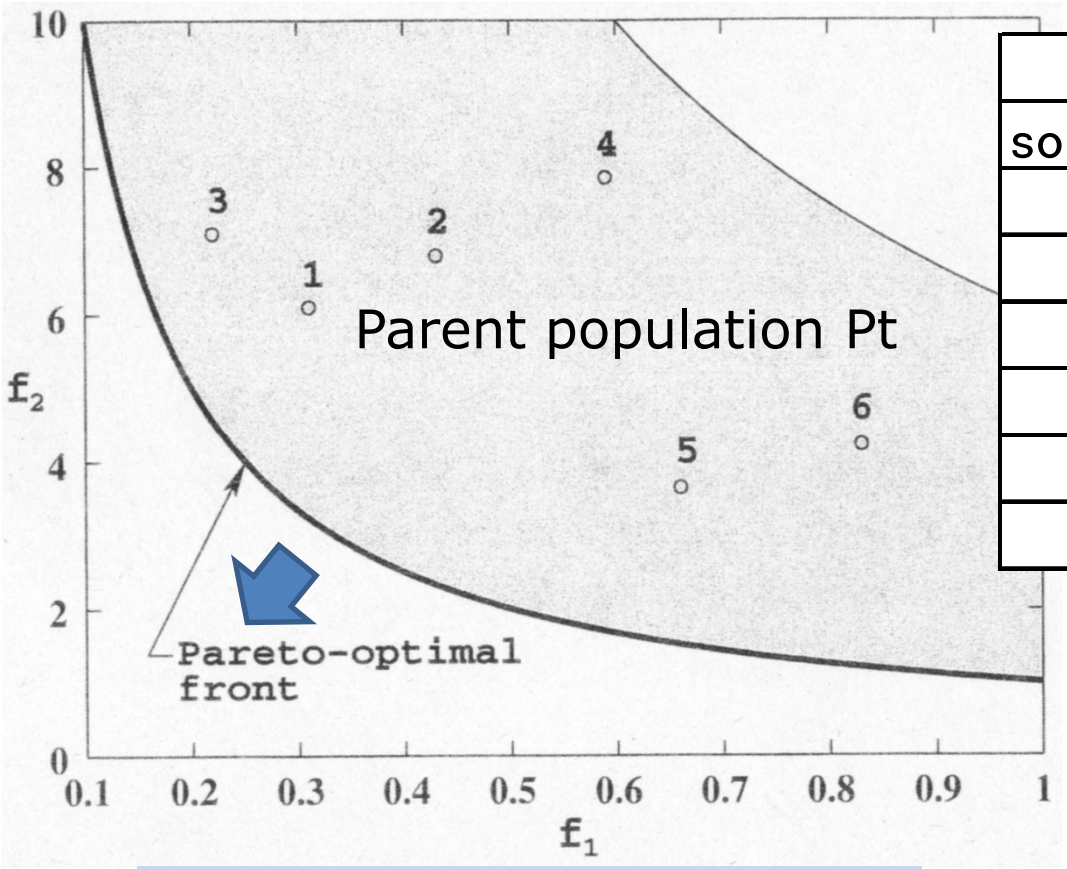
Let’s try selection operator in NSGA-II

# 例題として，Min-Ex問題を考える Suppose Min-Ex problem




| | |
|---|---|
| Minimize | $f_1(x) = x_1$ |
| Minimize | $f_2 = (1 + x_2) / x_1$ |
| subject_to | $0.1 \le x_1 \le 1$ |
| | $0 \le x_2 \le 5$ |

# 例題として，Min-Ex問題を考える Suppose Min-Ex problem




| Parent population, Pt | | | | |
|---|---|---|---|---|
| solution | x1 | x2 | f1 | f2 |
| 1 | 0.31 | 0.89 | 0.31 | 6.1 |
| 2 | 0.43 | 1.92 | 0.43 | 6.79 |
| 3 | 0.22 | 0.56 | 0.22 | 7.09 |
| 4 | 0.59 | 3.63 | 0.59 | 7.85 |
| 5 | 0.66 | 1.41 | 0.66 | 3.65 |
| 6 | 0.83 | 2.51 | 0.83 | 4.23 |

Minimize $f_1(x) = x_1$

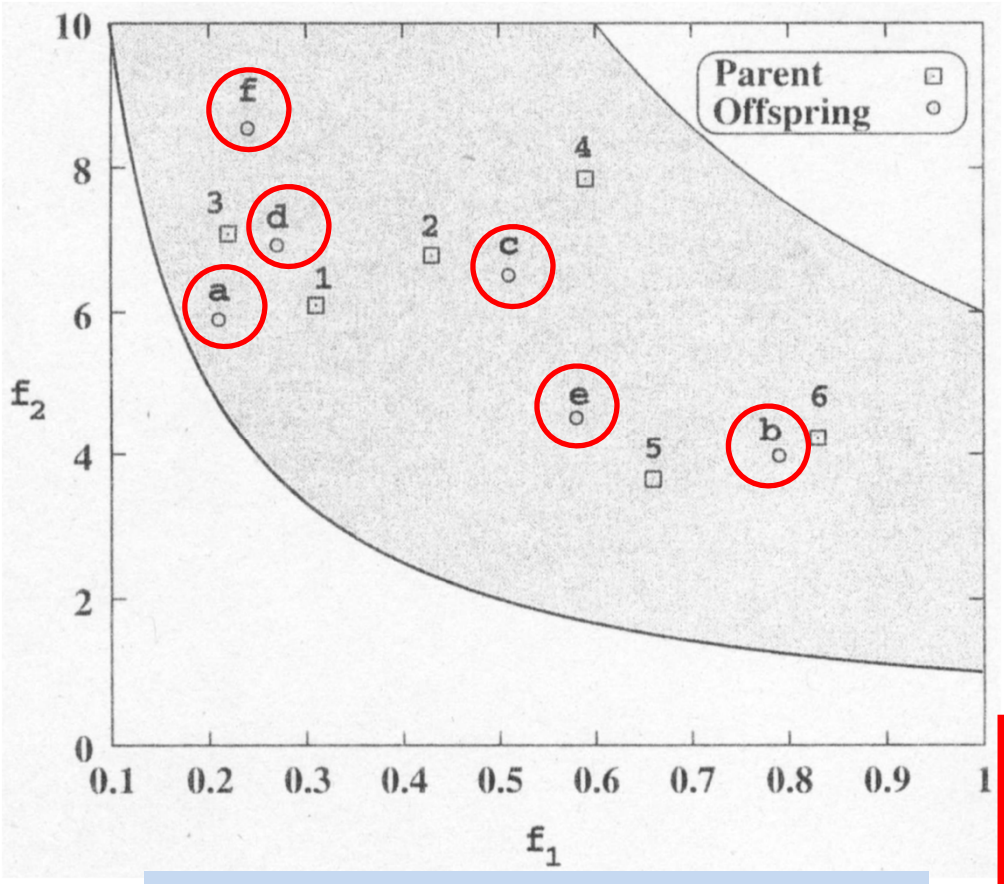
Minimize $f_2 = (1 + x_2) / x_1$

subject_to $0.1 \le x_1 \le 1$

$0 \le x_2 \le 5$

# あるt世代目の親集団と子集団の例

Example of parent and child populations at t-th generation




| Parent population, Pt | | | | |
|---|---|---|---|---|
| solution | x1 | x2 | f1 | f2 |
| 1 | 0.31 | 0.89 | 0.31 | 6.1 |
| 2 | 0.43 | 1.92 | 0.43 | 6.79 |
| 3 | 0.22 | 0.56 | 0.22 | 7.09 |
| 4 | 0.59 | 3.63 | 0.59 | 7.85 |
| 5 | 0.66 | 1.41 | 0.66 | 3.65 |
| 6 | 0.83 | 2.51 | 0.83 | 4.23 |

Minimize $f_1(x) = x_1$

Minimize $f_2 = (1 + x_2)/x_1$

subject_to $0.1 \le x_1 \le 1$

$0 \le x_2 \le 5$

| Offspring population, Qt | | | | |
|---|---|---|---|---|
| solution | x1 | x2 | f1 | f2 |
| a | 0.21 | 0.24 | 0.21 | 5.9 |
| b | 0.79 | 2.14 | 0.79 | 3.97 |
| c | 0.51 | 2.32 | 0.51 | 6.51 |
| d | 0.27 | 0.87 | 0.27 | 6.93 |
| e | 0.58 | 1.62 | 0.58 | 4.52 |
| f | 0.24 | 1.05 | 0.24 | 8.54 |

# 適応度の評価 evaluation of fitness

Fast nondominated sorting

(1) Set i=1

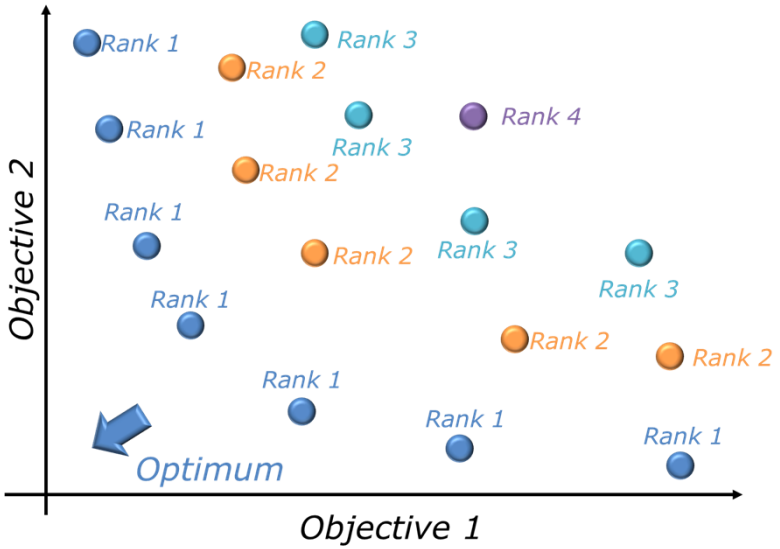
(2) For each individual, count dominate solutions and dominated solutions

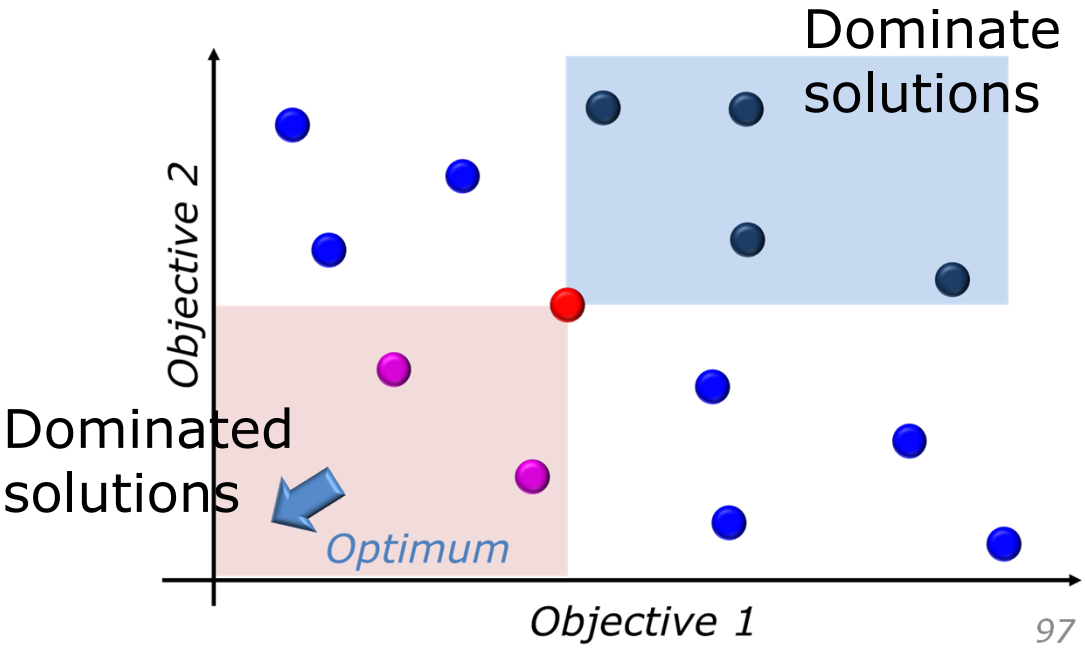
(3) Solutions that have no dominated solutions are rank i

(4) For dominate solutions of rank i solutions, subtract one from their number of dominated solutions

(5) remove rank I from the list

(6) i= i+1 and go to (3)



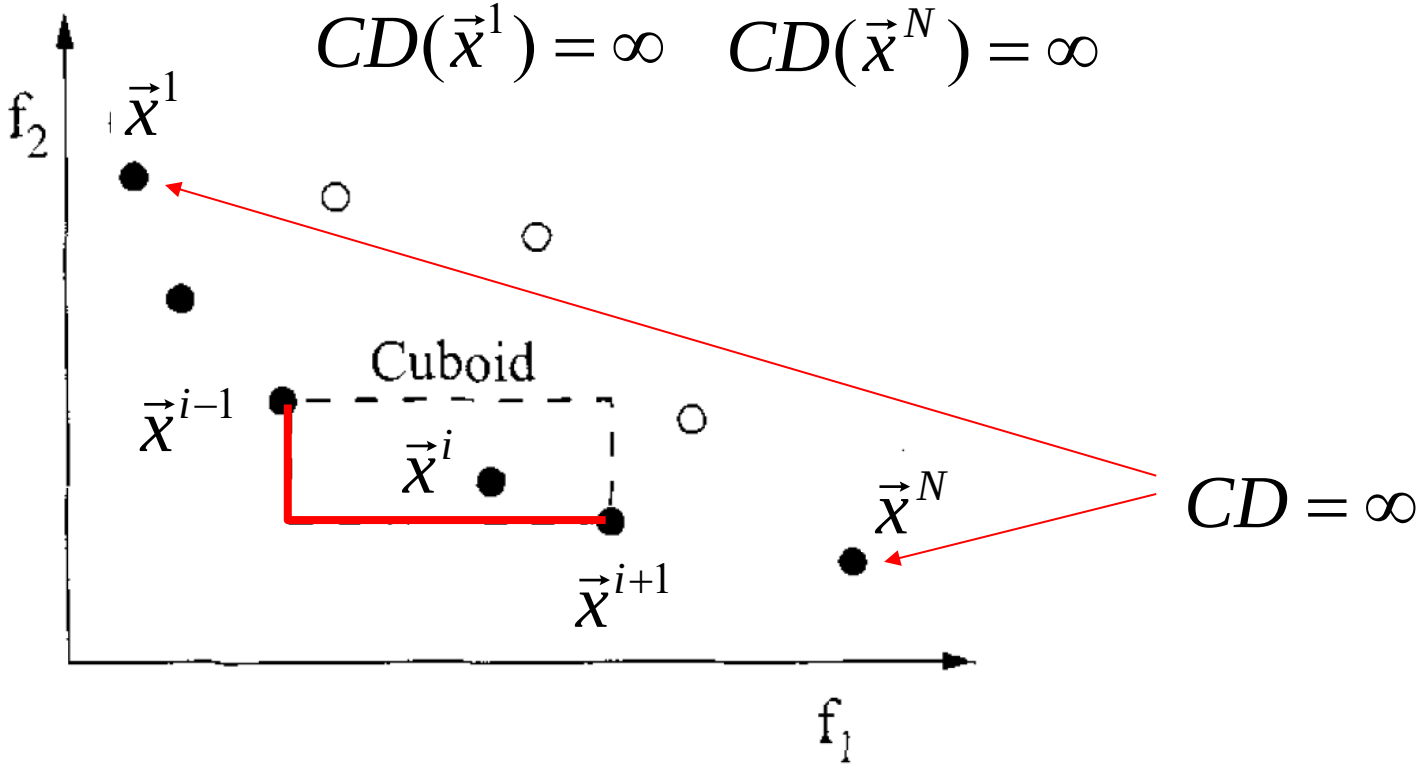
# 適応度の評価 evaluation of fitness

Crowding distance

- It has no parameter to be tuned.

$$CD(\vec{x}^i) = \frac{1}{M}\sum_{j=1}^{M}\left|\frac{f_j(\vec{x}^{i+1}) - f_j(\vec{x}^{i-1})}{f_j(\vec{x}^1) - f_j(\vec{x}^N)}\right|$$

$$CD(\vec{x}^1) = \infty \quad CD(\vec{x}^N) = \infty$$




*Kalyanmoy Deb, et al., 2002より*